Learning Deep Learning

Learning Deep Learning

From Pixels to Perception with PyTorch

William Edward Hahn

Learning Deep Learning: From Pixels to Perception with PyTorch
Copyright © 2026 William Edward Hahn.
All rights reserved.

Online edition, November 3, 2026: https://will.hahn.ai/book/

The code in this book is offered under the MIT License and is available at https://will.hahn.ai/book/code/. Every listing was run before printing; the printed output and the figures are the results of those runs, made on an Apple-silicon Mac with Python 3.9, PyTorch 2.8, torchvision 0.23, NumPy 2.0, SciPy 1.13, scikit-image 0.24 and matplotlib 3.9.

Sample photographs are from the `scikit-image` data collection. MNIST is from Yann LeCun, Corinna Cortes and Christopher Burges; Fashion-MNIST is from Zalando Research; the ants-and-bees images are from the PyTorch transfer-learning tutorial; the pretrained AlexNet weights are distributed with `torchvision`.

While every precaution has been taken in the preparation of this book, the author assumes no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein.

Contents

Preface **xi**

How to Use This Book **xv**

I Why Deep Learning **1**

1 Eight Million Numbers, One Bit **3**
1.1 Skills we cannot write down 4
1.2 You already know the vocabulary 4
1.3 A line is a decision 5
1.4 A few assumptions, and randomness 6
1.5 Computers were people 7

2 The Pascaline Wager **9**
2.1 AI has arrived 10
2.2 Cats and dogs, before and after 2012 10
2.3 Placing the bet 11

3 A Short History of Thinking Machines **13**
3.1 Computation as a natural science 13
3.2 Let us compute 14
3.3 The mechanical kingdom 15
3.4 Turing's surprise 16
3.5 From perceptrons to transformers 17
3.6 Computers for thinking with 18

4 A Jungle of Brain Cells **19**
4.1 Three generations of neural networks 20
4.2 Randomly arranged neuron-like elements 21
4.3 Neurons as animals 22
4.4 Culture as software 22

5 If I Only Had a Brain **25**
5.1 What would a brain have to do? 26
5.2 Whose values? 27

II Images Are Numbers **29**

6 Arrays **31**
6.1 A first array 31
6.2 Two dimensions 33
6.3 Reshaping 34
6.4 Indexing and slicing 35
6.5 Picking with conditions and lists 36
6.6 The dot product 37
6.7 Matrix multiplication 38
6.8 Summing along an axis 39
6.9 Finding the winner 40
6.10 Broadcasting 40
6.11 Random numbers 42

7 Images as Arrays **45**
7.1 A picture made of numbers 45
7.2 Color 47
7.3 A real photograph 48
7.4 Cropping 49
7.5 Taking the channels apart 50
7.6 A trap: overflow 51
7.7 Gray, small, turned and flipped 52
7.8 How many shades? 54
7.9 How many images are there? 56
7.10 Sound is an array too 58

8 Digits as Vectors **61**
8.1 Sixty thousand digits 61
8.2 Many at a glance 64
8.3 From pictures to vectors 65
8.4 A classifier with no learning 67
8.5 A harder set: Fashion-MNIST 69

III Learning from Data **71**

9 Chance **73**
9.1 A recipe for chaos . . . 74
9.2 Flipping coins . . . 77
9.3 Pascal's triangle and the bell curve . . . 78
9.4 The square-root law . . . 81
9.5 Throwing darts . . . 83
9.6 Excursion: when intuition fails . . . 85
9.7 What chance teaches . . . 87

10 Learning by Guessing **89**
10.1 Enter PyTorch . . . 89
10.2 The graphics card . . . 91
10.3 One random guess . . . 92
10.4 Guess a thousand times . . . 93
10.5 Small steps . . . 94
10.6 Survival of the fittest . . . 96
10.7 What guessing teaches . . . 98

11 Following the Gradient **99**
11.1 Slopes for free . . . 100
11.2 The chain rule, backwards . . . 101
11.3 A backward() of your own . . . 102
11.4 Rolling downhill . . . 104
11.5 From scores to probabilities . . . 107
11.6 Cross-entropy . . . 108
11.7 The slope of cross-entropy, by hand . . . 109
11.8 Training a classifier . . . 111
11.9 What did it learn? . . . 113
11.10 Letting PyTorch take the step . . . 115

12 Going Deeper **119**
12.1 Two layers that are really one . . . 120
12.2 The bend . . . 122
12.3 Starting weights . . . 124
12.4 A network with a hidden layer . . . 125
12.5 What the hidden layer sees . . . 126

12.6 Any number of layers . . . 128
12.7 Memorizing versus learning . . . 129
12.8 A harder problem . . . 130
12.9 The same network, the PyTorch way . . . 131

IV Seeing with Convolutions 135

13 Convolution 137
13.1 Finding coins . . . 137
13.2 Correlation and convolution . . . 141
13.3 Kernels as tools . . . 142
13.4 Convolution by hand . . . 144
13.5 Convolution in PyTorch . . . 145
13.6 From filters to networks . . . 147
13.7 Coda: convolution as a rule of nature . . . 148

14 A Convolutional Network 155
14.1 Images with channels . . . 156
14.2 Max pooling . . . 157
14.3 The network . . . 158
14.4 Training . . . 160
14.5 Which mistakes? . . . 162
14.6 What the network learned . . . 163

15 Inside AlexNet 167
15.1 Loading a trained network . . . 167
15.2 Preparing an image . . . 169
15.3 What is in this picture? . . . 170
15.4 Why `eval()` matters . . . 172
15.5 The first layer . . . 173
15.6 Deeper layers . . . 176
15.7 AlexNet by hand . . . 178
15.8 Saving and loading weights . . . 179

16 Teaching an Old Network New Tricks 181
16.1 A small dataset . . . 182
16.2 More data for free . . . 183
16.3 A new head . . . 185

16.4 Training the new layer . . . 187
16.5 Fine-tuning everything . . . 188
16.6 Without the head start . . . 189
16.7 Did it already know? . . . 190
16.8 Looking at the answers . . . 191
16.9 Saving and using the model . . . 193

17 Random Features **195**
17.1 A curve from random bends . . . 195
17.2 Solving instead of training . . . 197
17.3 A random hidden layer . . . 199
17.4 Excursion: too many features . . . 201
17.5 Random kernels . . . 204
17.6 Random features on photographs . . . 207
17.7 Excursion: a random memory . . . 209
17.8 How much must be learned? . . . 215

V Machines That Imagine **219**

18 Autoencoders **221**
18.1 A linear autoencoder . . . 222
18.2 Going up: transposed convolution . . . 226
18.3 A convolutional autoencoder . . . 227
18.4 What the codes are good for . . . 231
18.5 The space of codes . . . 233
18.6 Variational autoencoders . . . 234

19 Two Networks at Play **237**
19.1 One kind of thing . . . 238
19.2 The two players . . . 239
19.3 The contest . . . 240
19.4 Watching it learn . . . 243
19.5 New sneakers . . . 245
19.6 New, or copied? . . . 247

20 Onward **251**
20.1 What we built, and from what 251
20.2 Images as tokens 252
20.3 Attention 255
20.4 How good is it, and what does it look at? 258
20.5 From attention to language models 261
20.6 Questions to invent with 261
20.7 Final thoughts 263

Appendix **265**

A A Project of Your Own **267**
A.1 Warming up 267
A.2 Choosing a problem 268
A.3 Experiments, done fairly 268
A.4 A second project: learning a law of nature 272
A.5 Telling others 275

Notes and Further Reading **277**

Index **281**

Preface

In this book you will build, from scratch, the machinery behind modern artificial intelligence. You will turn photographs into arrays of numbers and handwritten digits into lists of 784 of them. You will get to know chance, then build a machine that learns by guessing, then one that learns by following the slope downhill, then networks with many layers. You will build convolutional networks that see, take apart a famous one, find out how much a network with random weights can already do, and finally build machines that imagine images of their own — ending with the attention mechanism at the heart of today's language models.

Feynman said you have to build something to understand it.[1] I agree, and that is why this is a hands-on book. The first part is about *why*; every part after it is something you will type, run and change yourself.

The one idea

Here is the whole book in a sentence: *every learning machine is a list of weights, a function that uses them, a loss that says how wrong the function is, and a loop that nudges the weights to make the loss smaller.* Everything else is arrangement and scale. The weights are just numbers. The function is built from two parts, weighted sums and a bend called a nonlinearity. The loss is one number. The loop is guessing at first, and then following the slope downhill. Every chapter adds one piece to that picture or puts the same four pieces together in a new way, and the last chapter shows that the machine behind today's chatbots is no exception. When you get lost, come back to this paragraph and ask: which of the four am I looking at?

[1] On his blackboard at Caltech at the time of his death in 1988 Feynman had written, "What I cannot create, I do not understand." Photograph, Caltech Archives, image 1.10-29.

Why pixels, in the age of chatbots?

It is fair to ask why a book written in the era of large language models spends its time on pixels and convolutional networks. The answer is that everything inside a language model — arrays of numbers, weights, layers, gradients, the softmax that turns scores into probabilities, the cross-entropy that measures how wrong a prediction is — is exactly what you will build here. Images just let you *see* it working. By the time we reach attention in the last chapter, it will be assembled from parts you already made with your own hands.

Who this book is for

I believe neural network technology should be taught to students of all ages, as soon as possible, and not only to computer scientists. (For readers who want to see the ideas before any code, younger readers included, the web edition opens with a page of things to try in the browser: draw, label, train and watch.) If you are a student in medicine, law, finance, biology, art or anything else, and you suspect that what you do is about to change, this book is for you. If you are a programmer who has used deep learning libraries without ever looking inside them, this book is for you too.

What you need

The book assumes basic Python: variables, lists, loops and functions. If you have never programmed, learn those first — any introductory Python tutorial will do, and a weekend is enough. Everything else, including NumPy and PyTorch, is introduced as we go.

The mathematics is gentler than you might fear. The basic concepts underlying neural networks need addition, multiplication, and one idea from calculus — the slope — and the book builds that idea from scratch. If you remember $y = mx + b$ from high school, you are ready. What makes neural networks powerful is not difficult mathematics; it is doing a great many simple operations, very quickly, with numbers that the data itself has chosen.

A journey and a guide

This book is a journey and a guide. A journey through the jungles of research into who we are, what makes us tick, and whether a machine can be made in our image. A guide to the intellectual revolutions already well under way, which are changing the ways we live, learn, communicate and create.

First and foremost it is a work of compression: a distillation of the ideas and scaffolding behind our understanding of brains and of the machines we are now building in their image, drawn from computer science, neuroscience, mathematics and physics. Compression is not just the boring thing you do to JPEGs. As Geordie Rose, a co-founder of D-Wave, put it in a talk, "the more that you can compress something the more that you understand it."[2]

Along the way I have tried to leave many of the original voices — Lovelace, Butler, Turing, von Neumann and others — in their own words. I also take an unconventional, and somewhat controversial, point of view: that computers are more than tools, that they are the star of the show. You will not agree with everything I say. Good. You will have to furnish your own map at times, and, like the explorers of old, here there be monsters.

William Edward Hahn
Boca Raton, Florida, 2026

[2]Geordie Rose, public talk; transcribed by the author from a recording.

How to Use This Book

Part I is for reading, and you may skip it and start at Part II, coming back to it later. From Part II on, the book is for doing: every idea comes with code, and the best way to learn it is to type that code in, run it, and then break it. Change a number and see what happens. Delete a line and read the error message. The networks in this book are small enough to train in seconds or minutes on an ordinary laptop, so experimenting is cheap.

What you need

You need Python 3.9 or later and a handful of libraries:

```
pip install torch torchvision numpy scipy scikit-image matplotlib
```

The PyTorch website, https://pytorch.org, gives the exact command for your operating system and hardware. If you would rather not install anything, a free hosted notebook service such as Google Colab already has all of these libraries installed and offers a graphics processor.

A graphics processor (GPU) helps but is not required. Chapter 10 shows how the code picks the best hardware available, whether an NVIDIA card, an Apple-silicon Mac, or the ordinary processor, and everything in the book runs on all three. Only the speed changes.

The datasets, the pretrained AlexNet weights and the sample photographs are all downloaded automatically, by the code that first uses them, into a folder called `data`.

The code and its output

Every listing in this book was run exactly as printed, and the results shown were produced by those runs. Listings that print something are followed by their output. Here is one from Chapter 6, which computes a dot product (that chapter explains what one is):

arr_dot

```
x = np.array([1.0, 2.0, 3.0])     # an input
w = np.array([0.5, -1.0, 2.0])    # a weight for each input
print(np.sum(x * w))
print(x @ w)
```

OUTPUT

```
4.5
4.5
```

All of the plots and images in the figures were produced the same way. The complete code for each chapter, as a single file you can run from top to bottom, is available at https://will.hahn.ai/book/code/.

Each listing's name is printed in small type at its top right, like `arr_dot` above. It is the name of the block in the chapter's code file, and the exercises use it to say which listing to change. Within a chapter, each listing builds on the ones before it, so run them in order. Each chapter starts fresh: it loads its own data and defines what it needs, apart from a few small helper functions that are introduced once and then reused, such as `montage_plot` for drawing many images at once. A chapter that reuses one says so, and the chapter's code file defines it again so that the file runs on its own.

Random numbers play a big part in deep learning, from the starting weights to the order of the training examples. The code fixes the random seed so that it gives the same results every time it runs, but different hardware generates random numbers differently, so your numbers will not exactly match the ones printed here. They should be close, and they should tell the same story.

Python beyond the basics

The book assumes basic Python: variables, lists, loops, `if` and functions. A few other pieces of the language turn up along the way, and here is all you need to know to read them:

- `f'{x:.3f}'` is an *f-string*: the value of `x` is written into the text, here with three decimal places.
- `with torch.no_grad():` runs the indented lines inside a setting, here "do not record for gradients," and switches it off afterwards.
- `[f(x) for x in items]` builds a list in one line; without the brackets, as in `sum(f(x) for x in items)`, the values are made one at a time and used as they come.
- `lambda x: x * 2` is a function without a name, handy when one is needed for a single line.
- `a, b = pair` unpacks two values at once, and `**options` passes on whatever keyword arguments a function was given.
- A `class` bundles data with the functions that work on it; you will only need to read one, in Chapter 11.

Conventions

Code and names from code appear in a `typewriter` font. Pictures and plots are drawn with matplotlib (`plt.figure`, `plt.imshow`, `plt.plot`, `plt.subplot` for a grid of panels); the book uses only a handful of its commands, and you can treat them as recipes. Throughout the book, `X` holds the inputs (images), `Y` the correct answers (labels), and `X_test` and `Y_test` the examples held back for testing. Weights are kept in a list called `w`, and the device the computation runs on is called `device`.

Terms are set in *italics* where they are defined, and the index points to those definitions.

Part I

Why Deep Learning

Chapter 1

Eight Million Numbers, One Bit

A man provided with paper, pencil, and rubber, and subject to strict discipline, is in effect a universal machine.
— Alan Turing, 1948

In my lab there is a small self-driving rover with a camera. Its controls are reduced to three actions: turn left, turn right, go forward. We taught it to drive a scale road course without writing a single rule about roads. First, people drove it around the course, choosing an action for each camera frame. That gave us a large table of pictures, each labeled *left*, *right* or *forward*. Then a neural network studied the table until, shown a new frame, it chose the action a human would have chosen. Run that network many times a second, and the rover drives the course on its own.

That little machine is this whole book in miniature. Pixels go in, a decision comes out, and in between is a function nobody wrote by hand: it was *learned* from examples. Everything that follows — the arrays, the guessing, the gradients, the layers, the convolutions — is about how to build that function, and how to understand what it has learned. You will be able to build the rover's brain yourself by the end of Part IV: it is the recipe of Chapter 16, with three folders of camera frames.

Let's look more closely at what the problem really is: taking images from a camera and turning them into decisions.

Modern cameras record an enormous number of samples. An 8-megapixel photo has eight million pixels, and a color photo stores three numbers for each one — red, green and blue — so that's about 24 million numbers written to memory every time you press the button. The cameras in 2026 phones range from 12 to 200 megapixels. Now suppose we want to make one decision from that photograph: are the subject's eyes closed? If they are, the camera can wait before taking the shot. Eyes open or closed is a single binary digit, 0 for closed, 1

for open. So the camera software has to look at tens of millions of numbers and reduce them to one bit.

That is the real question. How do we recombine millions of numbers into a single bit that tells us whether the people in the scene have their eyes open? This is what deep learning makes possible.

1.1 Skills we cannot write down

For most vision tasks it is simply not practical for humans to write out the rules, whether for driving a car or for recognizing handwriting. These are skills we embody, but we have no explicit access to them in a form we could write down as a program.

We do not teach babies to walk by explaining the dynamics of their ankle muscles, and if you think about your toes too much you're liable to fall down the stairs. Language is the same. We speak and listen in ideas, and it is hard to hear the sounds that make up our words. Repeat a word over and over until you can finally hear the sound you are making. By then it no longer makes sense as an idea; instead of *dolphin*, it is a strange and surprisingly complex set of noises.

There is a famous saying in AI that the hard problems are easy and the easy problems are hard. Building machines that see the way a two-year-old does required a different approach to computer science.

1.2 You already know the vocabulary

The good news is that you already understand many of the ideas you need. Thirty years ago this would not have been the case, but today everyone reading this knows about digital cameras, pixels, memory and file sizes. The vocabulary of consumer electronics is a Rosetta Stone for deep learning. Even a small child knows how to filter, crop and resize a photo, and that is how we will approach deep learning: as a series of filter and resize operations. A filter can pick out the edges in an image; a resize step, which we will call *pooling*, shrinks it while keeping what matters.

The rover from the start of this chapter makes the vocabulary concrete. Its table of labeled frames turns a network into a *policy*: a rule that chooses an action given an image. This is *supervised learning*; the human drivers provide the supervision in the form of labeled frames.

Photo tagging on social networks worked the same way. Before a system could suggest that you were in a picture, it had to have seen photographs labeled as having you in them, and those labels let it recognize you in new photos, even with a new haircut. Deciding which way to turn the steering wheel from a dash-cam image is, in principle, no different from finding a tumor in a brain scan, or a pattern in a medical, financial or legal document.

1.3 A line is a decision

The familiar $y = mx + b$ from high school algebra takes center stage in a modern deep learning system. Use the formula as a recipe to build a table of values, graph it, and you get a line. Now think of that line as a *decision boundary*. On one side we put all the yeses, on the other all the nos. Yes might mean "there's a good chance this photograph shows skin cancer," and no, "nothing to worry about."

Real data can rarely be separated by a single line. The power of deep learning is to combine thousands of these equations into a very complex decision space. No human could tune all those lines. Instead we start with random lines and let the data adjust them as the network learns. The adjusting needs exactly one idea beyond addition and multiplication: the slope, which tells us which way to nudge each number to make the answers less wrong. We will build that from scratch.

How do children learn shapes and colors, or the alphabet? Do you remember learning cursive? We call such tasks child's play, but they are at the heart of what a brain does. Handwriting recognition turned out to be far harder than anyone expected, until people tried to build a machine to reliably read unreliably written handwriting. The MNIST dataset, born of LeCun's work on reading digits, is arguably the most studied dataset in neural network research: many thousands of handwritten numerals, 0 through 9. The task is to name each one, as on *Sesame Street*.

It is hard to believe this problem is hard. We learned it so long ago that we forget, and so much of our brain is devoted to this sort of task that we take for granted doing it in noisy, unfamiliar conditions.

Look at how a handwritten digit becomes data. The original signal was analog, ink on paper, probably from a ballpoint pen. It was

digitized by laying a grid over it, like graph paper, and measuring the amount of ink in each square. Now it is a digital signal a program can use. But if you look at two examples of the same digit as two long lists of numbers, which is closer to how the computer sees them, it is very hard to tell that they are the same digit at all.

Our goal is to reduce all of those squares to a single number, the answer to an unambiguous question like "is this a two?" — 0 if not, 1 if so. What recipes could do that? We could add up all the numbers, or multiply them, or average them, or pick one at random. You can imagine many other ideas. These are *hand-tuned features*: preprocessing steps invented by humans to make the problem easier for the computer. For decades, much of that invention was a compromise with memory. Turing's theoretical machine had an infinite paper tape; real research groups do not, so many clever techniques were developed to shrink problems down to a size a computer could handle.

1.4 A few assumptions, and randomness

Kunihiko Fukushima's Neocognitron and Yann LeCun's convolutional networks took a different path. A convolutional network does not make *no* assumptions about images. It builds in a few strong ones: that nearby pixels matter to each other, and that a pattern means the same thing wherever it appears in the picture. Everything else — which edges, which textures, which shapes — it learns from the data, starting from random weights. Hand-tuned features were replaced by a few good assumptions plus randomness.

Randomness has power in its possibility. For something to be random means it is nothing in particular, which, strangely enough, means it can become precisely what you need. Like Turing's unorganized machines, and like a newborn brain, a neural network starts out randomly connected.

Put another way, you are searching for a function, a mathematical machine, that takes in some input and produces an output that answers your question — a business question, a scientific one, a medical one. It is a strange notion, a mathematical object answering a question, but that is what AI is really about. In a way it was never about computers. It is about finding a mathematical object that we can then instantiate in the physical world with what we call a computer.

1.5 Computers were people

Not long ago, computers were people. Computing was an occupation: just as bakers bake, computers computed, following rules with pencil and eraser, one step at a time — precisely the model Turing used to build his formal theory of computation. A human computer given a formula could apply it all day. Write "add" as a function, $\text{add}(1, 1) = 1 + 1$, and give it a single letter, f, as mathematicians like to, and the job is to evaluate f.

Deep learning turns that job inside out. We know the question we want to ask, and we have examples of answers. What we don't know is the formula connecting them. So we set up a random neural network that, in some sense, could compute almost any function, and let the examples adjust it until it computes the one we need. Once we have the formula, a computer evaluates it in a flash. The power of deep learning is having a computer find the formula, given only examples of the task.

That idea — find the formula from examples — is the engine of everything in this book. The next chapters ask why it matters to you, where it came from, and what it has to do with brains. Then, in Part II, we start building.

Chapter 2

The Pascaline Wager

I do not fear computers. I fear the lack of them.
— Isaac Asimov, as quoted in *Time*, 1978

Blaise Pascal, the seventeenth-century French philosopher, mathematician and physicist, is remembered for two inventions that I think belong together.

The first is his famous wager. Pascal's Wager says that every one of us bets with our life that God either exists or does not. You cannot sit the bet out; you can only decide how to live given the stakes. Pascal was honest about how uncomfortable that is: "seeing too much to deny Him, and too little to assure me, I am in a pitiful state."[1]

The second is the Pascaline, a mechanical calculator. Pascal built it at nineteen, spurred by the arithmetical labor of his father's work as supervisor of taxes in Rouen. It added and subtracted directly, and multiplied and divided by repeated addition and subtraction. It was, in a sense, premature: the mechanical arts of his time could not make it cheaply, accurately and strongly enough for long use, and that was not overcome until well into the nineteenth century.[2]

So here is what I call the *Pascaline Wager*. Either artificial intelligence is going to significantly change your life, or it is not; and either you will help shape it, or you will leave that to others. As with Pascal's original, you cannot decline to bet; not deciding is a decision. And like the Pascaline itself, the machines in question spent a long time looking premature — right up until they weren't.

[1] Blaise Pascal, *Pensées*, Project Gutenberg edition, http://www.gutenberg.org/files/18269/18269-0.txt.

[2] S. Chapman, "Blaise Pascal (1623–1662): Tercentenary of the calculating machine," *Nature* 150 (October 31, 1942): 508–509.

2.1 AI has arrived

Driving a car, reading a CT scan, writing an essay, generating a photograph of something that never existed: AI has "arrived" in a way previous generations only dreamed of, and virtually all of it is due to one algorithmic idea, *artificial neural networks*, loosely modeled on the brain. Even the people who built the modern world were surprised. In January 2017, at the World Economic Forum in Davos, Sergey Brin, co-founder of Google, recalled that, having trained as a computer scientist in the 1990s, "everybody knew that AI didn't work." "The revolution in deep nets has been very profound, it definitely surprised me, even though I was sitting right there."[3]

2.2 Cats and dogs, before and after 2012

To see what changed, look at a problem computers simply could not solve. Around 1990 Marvin Minsky said it plainly: "Right now there's no machine that can look around and tell a dog from a cat."[4] As late as 2007, Microsoft researchers built a security test, a CAPTCHA, on the assumption that a computer could not reliably sort photographs of cats from photographs of dogs.[5] It was not for lack of trying. Some of the smartest people in the world, Minsky among them, threw every mathematical and computational approach they had at visual recognition, and got slow, incremental progress that never worked well enough to use in the real world — on a job a sharp two-year-old does without much trouble.

Then, in 2012, a deep neural network called AlexNet won the ImageNet competition, sorting photographs into a thousand categories far better than any rival method, and deep learning took over computer vision, then speech, translation and, with the transformer, language. The next chapter tells that story.

---

[3] Sergey Brin, "An Insight, An Idea with Sergey Brin," World Economic Forum Annual Meeting, Davos, January 2017; as reported by the World Economic Forum, "Google co-founder Sergey Brin: I didn't see AI coming" (2017).

[4] Marvin Minsky, "Mind as Society," interview by Jeffrey Mishlove, *Thinking Allowed* (undated; the series ran 1986–2002).

[5] Jeremy Elson, John R. Douceur, Jon Howell and Jared Saul, "Asirra: A CAPTCHA that Exploits Interest-Aligned Manual Image Categorization," *Proceedings of the 14th ACM Conference on Computer and Communications Security* (2007).

Ask yourself what it means for the planet when a problem moves from the *impossible* category to the *not that difficult* category. That is what happened to cats and dogs, and it has kept happening, problem after problem.

2.3 Placing the bet

People have always been bad at this wager. In 1876, by Thomas Watson's later account, Bell's backers offered the telephone patent to Western Union for $100,000, and its president, William Orton, turned it down. A famous committee memo dismissing the telephone as a toy is, alas, a twentieth-century invention.[6]

If you do not believe AI is going to be a major force in the world, ask yourself what about nature and science is going to change to stop it. Part of the purpose of this book is to tell you about the emergency we face, and the emergency is not that people are working on AI. It is that *not enough* people are working on AI. We need students, researchers and practitioners in every field to seriously consider what their field will look like when much of what they do now can be automated.

That's the wager. I've placed my bet. The rest of this book is my attempt to help you place yours with your eyes open, and, better still, to give you the means to invent what comes next rather than wait for it.

[6]The memo first appeared in print in 1968; see David Malki, "True Stuff: Alexander Graham Bell vs. Western Union," *Wondermark*.

Chapter 3

A Short History of Thinking Machines

We shall find it impossible to refrain from asking ourselves what the end of this mighty movement is to be.
— Samuel Butler, 1863

It is hard to appreciate how new software is — even the idea of it. When Edsger Dijkstra, later a Turing Award winner, applied for a marriage license in 1957, the authorities asked for his occupation. He wrote "Programmer." It was rejected: there was no such job in the Netherlands at the time.

3.1 Computation as a natural science

Donald Knuth has said: "Science is what we understand well enough to explain to a computer. Art is everything else we do."[1] That cannot be said without controversy, so I will add some of my own. In this decade, if you are not using a computer, you are probably not doing science; it is hard to imagine a modern study that does not use one for collecting, analyzing, visualizing or communicating its data. This book takes the view that computers are more than tools. They are the star of the show: a new framework within which humanity can build its next generation of understanding.

In his original work Alan Turing presented a new mathematical object, as fundamental as the vector or the derivative: the computer program. It took decades for mathematicians and scientists to agree that programs are mathematical objects at all. Peter Denning goes further: "Computing is now a natural science," he wrote in 2007; it "is no longer a science of just the artificial. It is the study of information processes, natural and artificial."[2] Physicists find information processes

[1] Donald E. Knuth, foreword to Marko Petkovšek, Herbert S. Wilf and Doron Zeilberger, $A = B$ (Wellesley, MA: A K Peters, 1996), vii.

[2] Peter J. Denning, "Computing Is a Natural Science," *Communications of the ACM* 50, no. 7 (2007): 13–18.

in quantum waves, biologists in DNA, cognitive scientists in patterns of brain activity.

The early giants — Babbage, Turing, von Neumann — had no training in computer science; the subject did not exist. They were mathematicians and natural scientists. In my opinion, one of the unfortunate turns in academic history was the creation of separate computer science departments, which treated computing as its own discipline, cut off from natural science, mathematics, philosophy and language. I suspect it came down partly to cost: I can imagine department heads forced to choose between lab rats and workstations, and when I look at academic labs today I don't see many that have both. Neuroscience, the field that most needs to think about computation, is the one that has most often ignored it. Richard Feynman is reported to have said, in a lecture at Bell Laboratories, "I don't believe in computer science."[3] I believe in it — as a natural science, a branch of physics and mathematics. And as the rest of this chapter shows, the pioneers were thinking about brains all along.

3.2 Let us compute

Gottfried Wilhelm Leibniz dreamed of settling disputes by calculation. Instead of dueling, as Gregory Chaitin retells it, two people should be able to "just sit down and say, 'Gentlemen, let us compute!' "[4]

Charles Babbage, Lucasian Professor of Mathematics at Cambridge from 1828 to 1839, was tasked with eliminating errors in mathematical tables, and is said to have exclaimed, "I wish to God these calculations had been executed by steam!"[5] He planned a machine called the Analytical Engine. It was never completed, yet it drove the world's

[3] Recorded talk, usually dated to 1985 and captioned as given at Bell Laboratories. He went on: "To me, science is the study of the behavior of nature. And engineering or applied things is the behavior of things we make."

[4] Gregory Chaitin, "The Perfect Language," *Inference: International Review of Science* 1, no. 3 (2015). Leibniz's own word was the Latin *calculemus*, "let us calculate."

[5] The episode dates from 1821, when Babbage and John Herschel were checking astronomical tables; the words are from Babbage's account in H. W. Buxton, *Memoir of the Life and Labours of the Late Charles Babbage Esq. F.R.S.*, ed. Anthony Hyman (Cambridge, MA: MIT Press, 1988), 46, as cited by Doron Swade, "Automatic Computation: Charles Babbage and Computational Method," *The Rutherford Journal* 3 (2010).

imagination. One admirer wrote that Babbage "had taught wheelwork to think."[6] According to the historian Doron Swade, Babbage boasted that his machines "would produce the correct result or would jam but that they would never deceive."[7]

Ada Lovelace saw further than anyone, at least a century ahead of her time. In 1844 she wrote that "it does not appear to me that cerebral matter need be more unmanageable to mathematicians than sidereal & planetary matter & movements," and that she hoped "to bequeath to the generations a calculus of the nervous system."[8] A calculus of the nervous system is not a bad description of what you will build in this book.

3.3 The mechanical kingdom

In 1863 Samuel Butler connected Darwin's new theory of evolution to the machines spreading everywhere around him.[9] He looked at the "primordial types of mechanical life" — the lever, the wedge, the inclined plane, the screw and the pulley — and then at the machinery of the steamship *Great Eastern*, and found himself "almost awestruck" at how fast the mechanical world advanced compared with the slow progress of animals and plants. "In what direction is it tending? What will be its upshot?"

Butler's image was a new branch on the tree of life: as the vegetable kingdom developed from the mineral, and the animal from the vegetable, "so now in these last few ages an entirely new kingdom has sprung up." He described a pocket watch the way a naturalist describes a turtle, as "the beautiful structure of the little animal," descended from the cumbrous clocks of the thirteenth century, and predicted that clocks might one day go extinct like the earlier saurians. It is a

[6] Buxton, *Memoir*, written 1872–1880. The full sentence: "the marvelous pulp and fibre of a brain had been substituted by brass and iron, he had taught wheelwork to think."

[7] Doron D. Swade, "Redeeming Charles Babbage's Mechanical Computer," *Scientific American*, February 1993.

[8] Ada Lovelace, letter to Woronzow Greig, 15 November 1844, in Betty Alexandra Toole, *Ada, the Enchantress of Numbers* (Mill Valley, CA: Strawberry Press, 1992). The sentence on cerebral matter is from her correspondence of the same period.

[9] Samuel Butler, "Darwin among the Machines," *The Press* (Christchurch, New Zealand), 13 June 1863.

metaphor, not biology, but a useful one: if the machines really are a new branch on the tree of life, we should expect the world to adapt to them on the scale of earlier upheavals, like the rise of the animals.

3.4 Turing's surprise

By 1950 the idea of a digital computer was, in Turing's words, already "an old one."[10] His most detailed knowledge of the Analytical Engine came from Lady Lovelace's 1843 memoir, which states: "The Analytical Engine has no pretensions to originate anything. It can do whatever we know how to order it to perform." Hartree had added in 1949 that this did not rule out electronic equipment that would "think for itself."

Turing took the objection that a machine can never surprise us and turned it around. "Machines take me by surprise with great frequency," he wrote, largely because he did not do enough calculation to know what to expect. The belief that machines cannot surprise us, he argued, rests on the false assumption that "as soon as a fact is presented to a mind all consequences of that fact spring into the mind simultaneously with it." His goal was to "investigate the question as to whether it is possible for machinery to show intelligent behaviour,"[11] and with Turing, it is my contention "that machines can be constructed which will simulate the behaviour of the human mind very closely."[12]

John McCarthy, who coined the term *artificial intelligence*, saw two roads: imitate the nervous system or human psychology, insofar as we understand them, or study the problems the world presents to anything trying to achieve goals. "I would say that the computer-science approach is the one that so far has had the most success."[13] Machine learning is nearly as old: Arthur Samuel's 1959 "Some Studies in Machine Learning Using the Game of Checkers" reported a program that learned "to play a better game of checkers than can be played by the person who wrote the program." A learning machine need not even be electronic. In 1961 Donald Michie built MENACE, a

[10] Alan Turing, "Computing Machinery and Intelligence," *Mind* 59, no. 236 (1950): 433–460.

[11] Alan Turing, "Intelligent Machinery," 1948.

[12] Alan Turing, "Intelligent Machinery, A Heretical Theory," c. 1951.

[13] John McCarthy, "Artificial Intelligence," interview by Jeffrey Mishlove, *Thinking Allowed* (undated; the series ran 1986–2002).

noughts-and-crosses player made of 304 matchboxes of colored beads: each box stood for a position, each bead for a move, and the machine learned by adding beads after a win and taking them away after a loss.[14] Konrad Zuse, who independently pioneered much of early hardware and software, later recalled that he had "realized that there is no border-line" between calculating and thinking.[15]

3.5 From perceptrons to transformers

The neural network road had its own history, and it was not a straight one. In 1958 Frank Rosenblatt introduced the *perceptron*, a machine that learned to classify patterns by adjusting the strengths of its connections. It was greeted with great excitement. A year later Oliver Selfridge described *Pandemonium*, a model of perception as a crowd of "demons": simple ones shouting when they see a feature, such as a line or a corner, and higher ones listening to them and shouting in turn, until a decision demon picks the loudest. Layers of feature detectors, each built on the one below, are exactly what a deep network learns. In 1969 Marvin Minsky and Seymour Papert published *Perceptrons*, a book that analyzed precisely what a single layer of such units can and cannot compute. Its limits were real, and the book became a major reason that funding and interest drained away from neural network research for more than a decade — one of the "AI winters."

The spring came in the 1980s. John Hopfield showed how ideas from physics could describe networks of neurons, and in 1986 David Rumelhart, Geoffrey Hinton and Ronald Williams popularized *backpropagation*, a way of training networks with many layers by sending the error backward through them. Meanwhile, in 1980, Kunihiko Fukushima had built the Neocognitron, a layered network for recognizing visual patterns inspired by the visual cortex. Yann LeCun combined that layered, convolutional design with backpropagation and, from the late 1980s through 1998, developed the convolutional

[14] Donald Michie, "Experiments on the Mechanization of Game-Learning. Part I: Characterization of the Model and Its Parameters," *The Computer Journal* 6, no. 3 (1963): 232–236.

[15] Konrad Zuse, "Some Remarks on the History of Computing in Germany," lecture, Los Alamos, 1976; in N. Metropolis, J. Howlett and G.-C. Rota, eds., *A History of Computing in the Twentieth Century* (New York: Academic Press, 1980).

networks known as LeNet, which read handwritten digits well enough to be put to practical use. The MNIST dataset of handwritten digits that came out of that work is the one you will use in Part II.

What these networks lacked was data and speed. Both arrived. ImageNet gathered millions of labeled photographs in some 22,000 categories, a thousand of which were used in an annual competition. In 2012 AlexNet — essentially LeNet's design made much larger, trained on graphics processors — won that competition by a wide margin, and the field turned on a dime. In 2017 the transformer was built around a mechanism called *attention*, and transformers became the engine of the language models that reached the public in late 2022.

Rosenblatt's perceptron, backpropagation, LeCun's digit reader, AlexNet, attention: you will build a version of every one of them in this book. The history is the syllabus.

3.6 Computers for thinking with

There is a second history running alongside the first, about what computers are *for*. In 1960 J. C. R. Licklider described "man-computer symbiosis": people and machines working together, each doing what it does best. Douglas Engelbart set out in 1962 to build tools for "augmenting human intellect," and in a famous 1968 demonstration showed the mouse, hypertext, shared screens and video conferencing, decades before most people had any of them. Seymour Papert, the same Papert who helped write *Perceptrons*, spent much of his life on children: with colleagues he created the Logo programming language, in which children teach a "turtle" to draw, and in *Mindstorms* (1980) he argued that children learn powerful ideas best by building things with them. Alan Kay, building on Papert, imagined a personal computer for children of all ages, the Dynabook, a medium for thinking rather than a calculator.

This book belongs to that second history as much as to the first. Its method is Papert's: you learn how a learning machine works by building one, breaking it and building it again. And the machines themselves are tools for thinking. A network that has learned to see is not only a product to use; it is an experiment you can open up, a theory of vision you can test.

Chapter 4

A Jungle of Brain Cells

When we talk mathematics, we may be discussing a secondary language, built on the primary language truly used by the central nervous system.

— John von Neumann, *The Computer and the Brain*, 1958

It is difficult to describe the brain, given that I am a brain and cannot be objective in the matter. In this chapter I will focus on one part of the story, the information processing: how a brain learns to adapt itself.

We should be humble about our metaphors. As John Daugman observes, "invariably the explanatory metaphors of a given era incorporate the devices and the spectacles of the day." The water technology of antiquity underlies the Greek idea of the soul as *pneuma* and Galen's four humours; Enlightenment clockwork ticks inside La Mettrie's *L'Homme machine*; Victorian steam and hydraulics churn beneath Freud's unconscious; the telegraph gave Helmholtz his metaphors.[1] So yes, "the brain is a computer" is our era's metaphor. The difference, I will argue, is that this metaphor can be run, tested and built.

Francis Crick called it "the Astonishing Hypothesis": that "you," your joys and sorrows, memories and ambitions, "are in fact no more than the behavior of a vast assembly of nerve cells and their associated molecules."[2] Neural network models rest on that hypothesis, and on an older discovery: Ramón y Cajal's "neuron doctrine," recognized with the 1906 Nobel Prize he shared with Golgi, which established that the brain is built from separate cells. If the brain is made of units, it can be modeled as a network of units.

[1] John G. Daugman, "Brain Metaphor and Brain Theory," in William Bechtel et al., eds., *Philosophy and the Neurosciences: A Reader* (Oxford: Blackwell, 2001), 23–24.

[2] Francis Crick, *The Astonishing Hypothesis: The Scientific Search for the Soul* (New York: Scribner, 1994).

4.1 Three generations of neural networks

In 1943 Warren McCulloch, a neuroscientist, and Walter Pitts, a logician, developed the first generation of neural networks as simplified models of nervous function. Real neurons are so complex that fully simulating the chemistry of even one cell is beyond the fastest computers, so a neural network models only a neuron's *information processing*. The McCulloch–Pitts neuron was a switch, either all on or all off. Networks of such switches could reproduce simple logic, and perhaps, it seemed, the brain was a system for implementing logic in switches, like the early telephone exchanges.

The second generation gives each neuron a continuous, graded output, and almost all of the networks in use today, including every network in this book, are of this kind. Its key development was backpropagation, which adjusts a network's weights to shrink a single number measuring how wrong its answers are. You will build exactly this, by hand, in Part III.

The third generation, as Wolfgang Maass defined it in 1997, uses *spiking* neurons, which communicate with brief pulses timed in continuous time, much as real neurons do. Running alongside all three generations is a thread from physics: John Hopfield's networks of the 1980s borrowed the mathematics physicists use for magnetic materials and described a network's computation as rolling downhill on an energy landscape. That view of neural computation as a physical process suggests networks might one day be built directly in hardware rather than simulated on digital computers.

Deep learning's big idea is *learning features from data instead of designing them by hand*, layer upon layer. Despite decades of research a broad theory of the brain remains elusive, and when Demis Hassabis, co-founder of DeepMind, said in 2010 that "if you last looked seriously at neuroscience circa 2005 — you are out of date," he might have been talking to most of us.[3]

[3] Demis Hassabis, "A Systems Neuroscience Approach to Building AGI," Singularity Summit, San Francisco, August 2010.

4.2 Randomly arranged neuron-like elements

Turing anticipated much of this. As Jack Copeland and Diane Proudfoot put it, "few realize that Alan Turing wrote a blueprint for much of the connectionist project as early as 1948."[4] Turing suggested that training could make some pathways through such a network effective and others ineffective, and, so far as is known, he "was the first person to consider building computing machines out of trainable networks of randomly arranged neuron-like elements."

Randomly arranged. Keep that word in mind. A child born in France learns French, and a child born in Spain learns Spanish. Our DNA cannot contain instructions for a machine that understands French or Spanish; there is too much information for that. Instead it encodes a semi-general-purpose learning machine whose particular connection strengths are not specified at birth. They start from something close to a random initial condition, and the brain develops by strengthening the connections that solve problems.

There is evidence that the same learning machinery serves many purposes. The Common Cortical Algorithm hypothesis holds that much of perception may come from a single learning algorithm. Ferrets whose visual inputs have been rewired to their auditory cortex can learn to see, and in hamsters rewired to their touch cortex, the touch neurons begin to respond like visual ones.[5]

---

[4] B. Jack Copeland and Diane Proudfoot, "Turing's Neural Networks of 1948," AlanTuring.net (2000); see also Copeland's introduction to "Intelligent Machinery" in Copeland, ed., *The Essential Turing* (Oxford: Clarendon Press, 2004).

[5] Laurie von Melchner, Sarah L. Pallas and Mriganka Sur, "Visual Behaviour Mediated by Retinal Projections Directed to the Auditory Pathway," *Nature* 404 (2000): 871–876; Christine Métin and Douglas O. Frost, "Visual Responses of Neurons in Somatosensory Cortex of Hamsters with Experimentally Induced Retinal Projections to Somatosensory Thalamus," *PNAS* 86 (1989): 357–361.

4.3 Neurons as animals

The living jungle of brain cells we all carry can be thought of as a microcosm of evolution on a much faster time scale. Each brain cell is like a small animal. It has no idea it is a brain cell, but it lives in an ecosystem of other cells that must cooperate and compete for nutrients and information; without input, neurons atrophy. In the forest, animals compete for places to live and food to eat. In the brain, neurons compete to represent ideas and memories. In the first word of the previous sentence you saw the capital I as a vertical bar, and you have neurons that detect that feature. From it and its neighbors you concluded the word was "In," and used that to understand the rest of the sentence. The cells that represent that vertical bar had to compete for the job. In ecology we would say they found a niche, the way a squirrel in a mature forest has found its niche among the acorns. Neither the squirrel nor the oak decided to enter into their arrangement; the forest evolved into it. In the same way, the brain starts as a collection of animals that must evolve, over seconds, days and years, into a mature ecosystem.

4.4 Culture as software

Perception is built as a hierarchy: the eye and ear pass signals up through stage after stage of processing, each extracting something more abstract than the last, from edges to stripes to a tiger. We pay attention only at the top of this hierarchy. You cannot attend to the stripes on the tiger; you don't even want to think about the tiger very much. You need to think about running away. If we could hear the language of our eardrums it would make no sense to us; if we could see what falls on our retinas, we could not make sense of it. Our conscious experience lives at the top of a deep-learning-style hierarchy, among symbols, concepts and emotions.

I once asked some students to describe the taste of strawberries in English. After a second, one blurted out "heaven." There is no good way to put experience into words. How, then, can we do what Turing suggested and teach a computer to "enjoy strawberries and cream"?[6]

[6] Alan Turing, "Computing Machinery and Intelligence," *Mind* 59, no. 236 (1950): 433–460.

Much of what we can do comes from culture — software, so to speak — riding on top of our perceptual system, and it is nearly impossible to separate what our brains do from what our culture does. No human is raised in isolation; we need a steady stream of meaningful information to train our neural networks. If that is true of us, it will be true of the machines we build.

Chapter 5

If I Only Had a Brain

I could think of things I never thunk before.
— The Scarecrow, *The Wizard of Oz* (1939)

In the 1939 classic *The Wizard of Oz*, the Scarecrow sings about what he would do if he "only had a brain." If we want to understand the near future of AI, we need only listen to the Scarecrow.

"I could while away the hours, conferrin' with the flowers." Computer vision could let robotic gardeners tend each plant individually, making mixed plantings practical and perhaps reducing the need for chemicals. Ironically, if the Scarecrow gets a brain, he may find himself right back in the cornfield.

"With the thoughts I'd be thinkin' I could be another Lincoln." Software already searches evidence in litigation. What are the legal and ethical considerations of letting it make important decisions?

"I'd unravel every riddle for any individ'le, in trouble or in pain." In 2011 IBM's Watson beat the *Jeopardy!* champions Ken Jennings and Brad Rutter, and within days IBM announced plans to apply it to medicine.[1] Deep learning vision systems have since become competitive with human radiologists on some narrow tasks, tasks that radiologists train for many years to perform. The work of doctors and lawyers is going to change profoundly, and few of the schools that train them teach these ideas yet.

[1] IBM and Nuance Communications announced the healthcare collaboration on February 17, 2011, the day after the final *Jeopardy!* broadcast. IBM sold off its Watson Health business in 2022.

5.1 What would a brain have to do?

It is much less clear what the Scarecrow's brain would have to *do*. How do little bursts of chemistry add up to a system that can learn to play the piano, or catch a football?

One of the most illuminating answers came from a board game. In 2016 DeepMind's AlphaGo beat Lee Sedol, one of the strongest Go players in the world, years before most experts expected a machine to win.

We usually praise computers for predicting exactly what will happen after a sequence of moves. That works for simple games, but in Go the number of possibilities explodes too fast to look very far ahead. So alongside its search, AlphaGo used two neural networks. One, a *policy* network, suggested promising moves — a policy is just a rule for choosing an action in a situation. The other, a *value* network, gave something we are all familiar with: a gut feeling. Given a position, it returned a single number saying how likely this side was to win. Like the plus or minus grades of elementary school, the signal carries little information, but its meaning is clear: does this feel good or bad? Shifting from *what will happen?* to *how does this situation feel?* was a major part of the breakthrough.

How do you teach a gut feeling? Imagine a whole game as a filmstrip: the first frame is the empty board, the last frame shows the winner. Pull frames at random from the middle and quiz the computer: who do you think will win? Because the last frame holds the answer, every frame becomes a training example. With millions of such frames the network learns what it is about a position that makes one side win.

5.2 Whose values?

Getting machines to share human values — what researchers now call *alignment* — has grown into a large field, and it should be. The famous thought experiment asks a machine to make paper clips, and it melts down the planet to make as many as it can. The story is as old as the genie's lamp: we never get what we asked for, because we were not careful enough to say what we actually cared about. Building machine culture deserves as much care as building machine intelligence.

There is a darker possibility, too: a world in which very few companies or countries understand AI. Imagine an app that can tell you whether the mole on your skin is cancer. It could save your life, but like most applications it will be protected by a password. When life-saving medicine is an application, who controls the passwords? That is why I believe AI should be more than open source; it needs to become part of our popular culture. My advice to every student is to learn AI programming early and often.

Does the good outweigh the bad? Asking that implies we have a great deal of control. When I imagine convincing the world to simply stop, I picture the plants in some prehistoric meadow agreeing that they don't want the animals to start showing up. We will need intellectual tools this century just as we needed mechanical tools in the previous ones.

The Scarecrow is getting his brain. The question is whether we will understand it.

Understanding starts with building. Before a network can turn millions of numbers into one bit, we need to get comfortable with the numbers themselves: what an image looks like to a computer, and how a handwritten digit becomes a list of 784 of them. Pick up your pencil and eraser. It's time to compute.

Part II

Images Are Numbers

Chapter 6

Arrays

Everything a neural network sees, it sees as a list of numbers. A photograph is a grid of brightness values, a sound is a sequence of air-pressure readings, and a sentence, by the time a network reads it, has been turned into rows of numbers too. The network itself is also just numbers: its "knowledge" is a large collection of weights. Learning to think about deep learning therefore begins with learning to think in *arrays*: lists, grids, and stacks of numbers that we can add, multiply and reshape all at once.

This chapter introduces NumPy, the Python library that made array programming ordinary. It is short on purpose. We will meet only the handful of operations that the rest of the book leans on, and almost every one of them will reappear, under a different name, inside PyTorch.

6.1 A first array

arr_first

```
import numpy as np
np.set_printoptions(precision=3, suppress=True)   # tidier printing

a = np.array([1, 2, 3, 4, 5])
print(a)
print(a + 5)
print(a * 2)
print(a.shape, a.dtype)
```

OUTPUT

```
[1 2 3 4 5]
[ 6  7  8  9 10]
[ 2  4  6  8 10]
(5,) int64
```

(The `set_printoptions` line only tidies the printing, showing three decimal places and no scientific notation. Every chapter starts with

something like it, so that the numbers you see match the ones printed here.)

Notice what did *not* happen: there is no loop. `a + 5` adds five to every element, and `a * 2` doubles every element. Operations on arrays act on all of their elements at once. This style is called *vectorized* code, and it is not just a convenience. The loop still happens, but it happens inside NumPy's compiled code (and, later, on a graphics card), where it runs hundreds of times faster than a Python loop would.

Every array has a `shape` and a `dtype`. The shape `(5,)` says there is one axis with five elements; the `dtype` says they are 64-bit integers. When something goes wrong in deep learning code, the first question to ask is almost always "what shape is this?"

6.2 Two dimensions

A grid of numbers is a two-dimensional array, which a mathematician would call a *matrix*.

arr_2d

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m)
print(m.shape)
print(m.T)
```

OUTPUT

```
[[1 2 3]
 [4 5 6]]
(2, 3)
[[1 4]
 [2 5]
 [3 6]]
```

The shape `(2, 3)` means two rows and three columns, always in that order. `m.T` is the *transpose*: rows become columns. Arithmetic works element by element exactly as before, and NumPy provides the usual mathematical functions:

arr_elementwise

```
a = np.array([1.0, 2.0, 3.0])
b = np.array([4.0, 5.0, 6.0])
print(a + b)
print(a * b)
print(np.sqrt(b))
print(np.mean(a), np.max(b))
```

OUTPUT

```
[5. 7. 9.]
[ 4. 10. 18.]
[2.    2.236 2.449]
2.0 6.0
```

6.3 Reshaping

The same twelve numbers can be arranged as a row, as a 3×4 grid, or as a $2 \times 2 \times 3$ block. Reshaping never changes the numbers or their order; it only changes how we index them.

arr_reshape

```
x = np.arange(12)
print(x)
print(x.reshape(3, 4))
print(x.reshape(2, 2, 3).shape)
print(x.reshape(-1, 6).shape)   # -1 means "work it out for me"
```

OUTPUT

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
(2, 2, 3)
(2, 6)
```

The -1 in the last line asks NumPy to work out that dimension from the others: twelve numbers in rows of six means two rows. This trick will be everywhere in the chapters ahead, because a 28×28 image and a list of 784 numbers are the same data, reshaped.

6.4 Indexing and slicing

Square brackets pick out parts of an array. For a two-dimensional array the first index is the row and the second is the column. A colon on its own means "everything along this axis," and `a:b` means "from `a` up to, but not including, `b`."

arr_index

```
m = np.arange(12).reshape(3, 4)
print(m)
print(m[0, 0], m[2, 3])   # single elements: [row, column]
print(m[1])               # a whole row
print(m[:, 2])            # a whole column
print(m[0:2, 1:3])        # rows 0-1, columns 1-2
```

OUTPUT

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
0 11
[4 5 6 7]
[ 2  6 10]
[[1 2]
 [5 6]]
```

The last line, which takes a rectangular block, is exactly how we will crop images in the next chapter.

6.5 Picking with conditions and lists

Two more ways of indexing will save us a great deal of work later. The first uses a condition. Comparing an array with a number gives an array of `True` and `False`, one per element, called a *mask*. Indexing with a mask keeps exactly the elements marked `True`. The condition can even come from a different array of the same length, which is how we will pick out, say, all the images of the digit 7 from a dataset using its list of labels.

arr_mask

```
scores = np.array([3, 8, 1, 9, 4])
print(scores > 4)              # a True/False answer for every element
print(scores[scores > 4])      # keep only the elements marked True

labels = np.array([0, 1, 0, 1, 1])
print(scores[labels == 1])     # select using a condition on another array

both = (scores > 2) & (scores < 9)     # & means "and", | means "or"
print(both, both.mean())       # the mean of True/False is a fraction
```

OUTPUT

```
[False  True False  True False]
[8 9]
[8 9 4]
[ True  True False False  True] 0.6
```

Two conditions combine with `&` (“and”) and `|` (“or”); the parentheses around each condition are needed. And because `True` counts as 1 and `False` as 0, the mean of a True/False array is the fraction of elements that are `True`: here three of five, 0.6. We will measure accuracy exactly this way, as the mean of `guesses == labels`.

The second uses a list of positions. Indexing with a list picks those elements in that order, repeats allowed. With one list per axis, the lists are paired up: the first row index with the first column index, and so on.

arr_fancy

```
print(scores[[0, 3, 3]])       # pick elements by a list of positions

m = np.arange(12).reshape(3, 4)
print(m[[0, 1, 2], [3, 0, 1]])  # elements (0,3), (1,0) and (2,1)
```

OUTPUT

```
[3 9 9]
[3 4 9]
```

Pairing like this picks one element from each row, a trick we will use in Chapter 11 to look up, for every image, the score of its correct answer.

6.6 The dot product

Here is the single most important operation in this book. Take two lists of numbers of the same length, multiply them pair by pair, and add up the results. That is the *dot product*.

arr_dot

```
x = np.array([1.0, 2.0, 3.0])     # an input
w = np.array([0.5, -1.0, 2.0])    # a weight for each input
print(np.sum(x * w))
print(x @ w)
```

OUTPUT

```
4.5
4.5
```

Read `x` as an input and `w` as a set of *weights*, one per input. The dot product is a weighted vote: inputs with large positive weights push the total up, inputs with negative weights pull it down, and inputs with weights near zero are ignored. An artificial neuron, stripped to its essentials, computes exactly this sum. NumPy writes it with the `@` operator.

6.7 Matrix multiplication

If one weighted sum is a neuron, then several weighted sums computed from the same input are a *layer* of neurons. Stack the weight vectors side by side as the columns of a matrix, and `@` computes all the dot products at once:

arr_matvec

```
x = np.array([1.0, 2.0, 3.0])          # 3 inputs
W = np.array([[0.5,  1.0],
              [-1.0, 0.0],
              [2.0,  1.0]])            # 3 inputs -> 2 outputs
print(W.shape)
print(x @ W)
```

OUTPUT

```
(3, 2)
[4.5 4. ]
```

Three inputs went in and two outputs came out, because the weight matrix has shape `(3, 2)`. Now stack several inputs as the rows of a matrix, one example per row, and a single multiplication processes all of them:

arr_matmul

```
X = np.array([[1.0, 2.0, 3.0],
              [0.0, 1.0, 0.0],
              [2.0, 2.0, 2.0],
              [1.0, 0.0, -1.0]])      # 4 examples, 3 numbers each
print(X.shape, W.shape)
print(X @ W)
print((X @ W).shape)
```

OUTPUT

```
(4, 3) (3, 2)
[[ 4.5  4. ]
 [-1.   0. ]
 [ 3.   4. ]
 [-1.5  0. ]]
(4, 2)
```

This is *matrix multiplication*, and its shape rule is worth memorizing:

$$(n \times k) \mathbin{@} (k \times m) \longrightarrow (n \times m).$$

The inner sizes must agree, since each row of the left matrix is dotted with each column of the right, and they disappear from the result. Four examples of three numbers each, times a 3×2 weight matrix,

give four examples of two numbers each. When the inner sizes do not agree, NumPy refuses:

arr_shape_error

```
try:
    W @ X
except ValueError as err:
    print(err)
```

OUTPUT

```
matmul: Input operand 1 has a mismatch in its core dimension 0, with gufunc
    ↪ signature (n?,k),(k,m?)->(n?,m?) (size 4 is different from 2)
```

The message is cryptic, so let's decode it once. "Operand 1" is the second matrix (NumPy counts from zero), `X` with shape (4, 3). Its "core dimension 0" is its first size, 4, and that should have matched the last size of the first matrix, which is 2. You will see this error many times. It is a friend: it means the shapes in your head and the shapes in the computer have drifted apart, and it tells you which sizes disagree.

6.8 Summing along an axis

Many operations can collapse an array along one axis. The `axis` argument names the axis that disappears.

arr_axis

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m.sum())
print(m.sum(axis=0))   # collapse the rows: one total per column
print(m.sum(axis=1))   # collapse the columns: one total per row
```

OUTPUT

```
21
[5 7 9]
[ 6 15]
```

6.9 Finding the winner

`np.argmax` returns the *position* of the largest value rather than the value itself. If the four numbers in `scores` were a network's confidence in four possible answers, `argmax` would be its answer. With `axis=1` it picks a winner for every row: one answer per example.

arr_argmax

```
scores = np.array([0.1, 2.3, -1.0, 0.7])
print(np.max(scores), np.argmax(scores))

S = np.array([[0.1, 2.3, -1.0, 0.7],
              [3.0, 0.2,  0.5, 0.1],
              [0.0, 0.1,  0.2, 0.9]])
print(np.argmax(S, axis=1))
```

OUTPUT

```
2.3 1
[1 0 3]
```

Put `@` and `argmax` together and you have the skeleton of a classifier: multiply the inputs by a weight matrix to get a score for each class, then pick the class with the highest score. The whole of Part III is about finding good weights for that matrix.

6.10 Broadcasting

When two arrays have different shapes, NumPy will often stretch the smaller one to fit, a rule called *broadcasting*. A single number is stretched to the whole array; a row is repeated down every row.

arr_broadcast

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m * 10)
print(m * np.array([1, 10, 100]))
print(m - m.mean(axis=0))
```

OUTPUT

```
[[10 20 30]
 [40 50 60]]
[[  1  20 300]
 [  4  50 600]]
[[-1.5 -1.5 -1.5]
 [ 1.5  1.5  1.5]]
```

The last line subtracts each column's mean from that column, a one-line version of the kind of data normalization we will do repeatedly.

The precise rule is this. NumPy compares the two shapes starting from the *right*. Two sizes are compatible if they are equal or if one of them is 1, and a size of 1 is stretched to match the other. Missing sizes on the left count as 1. So shape (2, 3) works with (3,), because the 3s line up on the right, but not with (2,), because 3 and 2 do not:

arr_keepdims

```
m = np.array([[1.0, 2.0, 3.0],
              [4.0, 5.0, 6.0]])
totals = m.sum(axis=1)
print(totals.shape)
try:
    m / totals
except ValueError as err:
    print(err)

totals = m.sum(axis=1, keepdims=True)
print(totals.shape)
print(m / totals)              # each row now adds up to 1
```

OUTPUT

```
(2,)
operands could not be broadcast together with shapes (2,3) (2,)
(2, 1)
[[0.167 0.333 0.5  ]
 [0.267 0.333 0.4  ]]
```

Summing each row gives one total per row, shape (2,), which does not line up with the rows of `m`. The fix is `keepdims=True`, which keeps the summed axis as a size-1 axis, shape (2, 1), so that each total stretches across its own row. Dividing each row by its total, as here, is exactly what we will do to turn scores into probabilities in Chapter 11.

6.11 Random numbers

Neural networks begin their lives as random numbers, so we need a way to make them. NumPy's random number generator can produce uniform values, integers, and samples from the bell-shaped normal distribution. Passing a seed makes the "random" numbers repeatable, which is why your output will match the output printed here.

arr_random

```
rng = np.random.default_rng(seed=0)
print(rng.random(5))                # uniform between 0 and 1
print(rng.integers(1, 7, size=10))  # dice: 1 to 6 (7 is excluded)
print(rng.normal(0, 1, size=5))     # bell curve: mean 0, spread 1
```

OUTPUT

```
[0.637 0.27  0.041 0.017 0.813]
[4 6 4 4 6 5 4 4 4 6]
[-0.623  0.041 -2.325 -0.219 -1.246]
```

A histogram shows the familiar bell shape of ten thousand normal samples (Figure 6.1).

arr_hist

```
import matplotlib.pyplot as plt

samples = rng.normal(0, 1, size=10_000)
plt.hist(samples, bins=60)
plt.xlabel('value')
plt.ylabel('count')
plt.show()
```

That is all the NumPy we need to begin. In the next chapter we will use these same operations, including indexing, slicing, reshaping and arithmetic, on pictures.

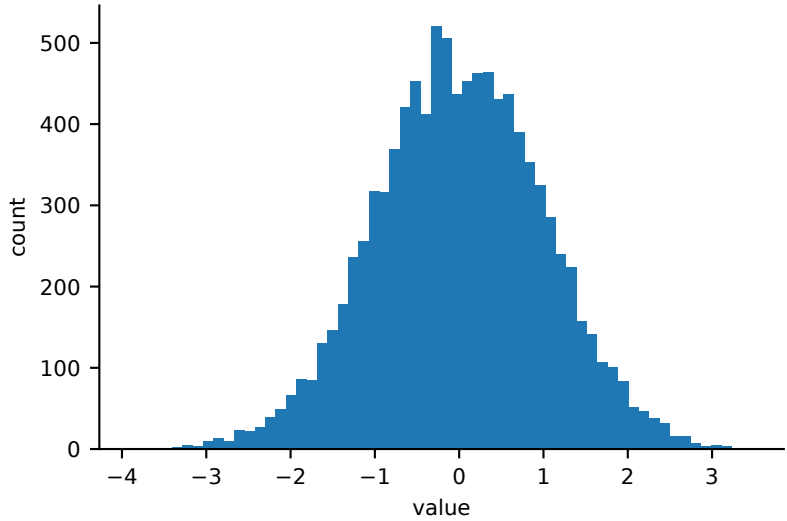


Figure 6.1: Ten thousand samples from the normal distribution.

Try it

1. Make a 3×4 array of random integers with `rng.integers` and find which row has the largest total, using `sum` with `axis=1` and then `argmax`.

2. Predict the shape of `np.ones((5, 1)) + np.ones((1, 4))` using the broadcasting rule, then check. Now try `np.ones(5) + np.ones(4)` and explain the error.

3. Compute the dot product of two arrays of a million random numbers, once with a Python `for` loop and once with `@`. Time both with the `time` module. How many times faster is `@`?

4. Draw 10,000 samples from the normal distribution. Use a mask to find what fraction of them lies between -1 and 1. (The answer should be close to 68 percent.)

Chapter 7

Images as Arrays

A digital camera does not store a picture. It stores a list of measurements, one for each tiny square of the sensor, saying how much light landed there. The "picture" only appears when a screen turns those numbers back into light. In this chapter we take that idea literally: we will build images out of arrays by hand, take real photographs apart into their arrays, and change them with nothing but the NumPy operations from Chapter 6.

7.1 A picture made of numbers

Here is an image typed in by hand. Each number is a *pixel* ("picture element"): 0 is black, 255 is white, and values in between are shades of gray.

img_smiley

```
import numpy as np
import matplotlib.pyplot as plt

face = np.array([
    [0,   0,   0,   0,   0,   0,   0,   0],
    [0,   0, 255,   0,   0, 255,   0,   0],
    [0,   0, 255,   0,   0, 255,   0,   0],
    [0,   0,   0,   0,   0,   0,   0,   0],
    [0, 255,   0,   0,   0,   0, 255,   0],
    [0,   0, 255, 255, 255, 255,   0,   0],
    [0,   0,   0,   0,   0,   0,   0,   0],
], dtype=np.uint8)

print(face.shape, face.dtype)
plt.figure(figsize=(2, 1.8))
plt.imshow(face, cmap='gray')
plt.show()
```

OUTPUT

```
(7, 8) uint8
```

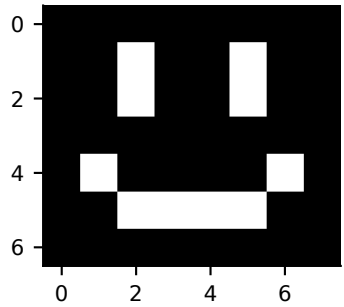


Figure 7.1: A 7×8 array of numbers, drawn as an image.

The values run from 0 to 255 because each pixel is stored as one byte, an unsigned 8-bit integer, or `uint8`. A byte can hold 256 different values, which turns out to be enough shades of gray that your eye can almost never see the steps between them.

Since we will draw a lot of images, let's wrap the drawing in a small helper. Its `size` is the width of the drawing in inches, and any extra settings are passed straight on to `imshow`:

img_plot

```
def plot(x, title=None, size=2.6, **kwargs):
    plt.figure(figsize=(size, size))
    plt.imshow(x, cmap='gray', **kwargs)
    plt.axis('off')
    if title:
        plt.title(title)
    plt.show()
```

Because the image is an array, changing the image means doing arithmetic. To make a negative, subtract every pixel from 255 (Figure 7.2).

img_invert

```
plot(255 - face, size=1.8)
```

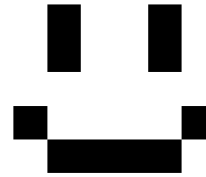


Figure 7.2: The negative image: `255 - face`.

7.2 Color

A color image needs three numbers per pixel: how much red, green and blue light to mix. NumPy stores this as a three-dimensional array with shape *(height, width, 3)*. The last axis holds the three *channels*.

img_rgb

```
pixels = np.array([
    # red          green          blue
    [[255, 0, 0],   [0, 255, 0],   [0, 0, 255]],
    # cyan         magenta        yellow
    [[0, 255, 255], [255, 0, 255], [255, 255, 0]],
], dtype=np.uint8)

print(pixels.shape)
plot(pixels, size=2)
```

OUTPUT

```
(2, 3, 3)
```

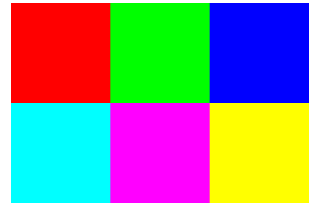


Figure 7.3: Six pixels. Full red plus full green makes yellow; green plus blue makes cyan.

Mixing light is not mixing paint. Red and green light together make yellow, and all three at full strength make white.

7.3 A real photograph

Real images come from files. The `imread` function from scikit-image reads a JPEG or PNG file, or a web URL, and hands back an ordinary NumPy array. To keep everything in this book reproducible, we will use the sample photographs that ship with scikit-image. Here we save one to disk and read it back exactly as you would read your own photo.

img_load

```
import os
from skimage import data
from skimage.io import imread, imsave

os.makedirs('data', exist_ok=True)      # a folder for our files
imsave('data/cat.png', data.chelsea())  # save a sample photo...
cat = imread('data/cat.png')            # ...and read it back

print(cat.shape, cat.dtype, cat.min(), cat.max())
plot(cat, size=3.4)
```

OUTPUT

```
(300, 451, 3) uint8 0 231
```



Figure 7.4: Chelsea the cat: an array of shape (300, 451, 3).

The photograph is 300 pixels tall and 451 wide, with three channels: $300 \times 451 \times 3 = 405{,}900$ numbers. A phone camera produces tens of millions. Indexing works exactly as it did for the small arrays in Chapter 6: `cat[row, column]` is one pixel, a list of three numbers, and a third index picks a single channel.

img_pixel

```
print(cat[100, 200])        # one pixel: [red, green, blue]
print(cat[100, 200, 0])     # just its red value
```

OUTPUT

```
[76 39 13]
76
```

7.4 Cropping

Cropping is slicing. To cut out a rectangle, give a range of rows and a range of columns:

img_crop

```
eye = cat[20:190, 110:290]
print(eye.shape)
plot(eye, size=2.2)
```

OUTPUT

```
(170, 180, 3)
```

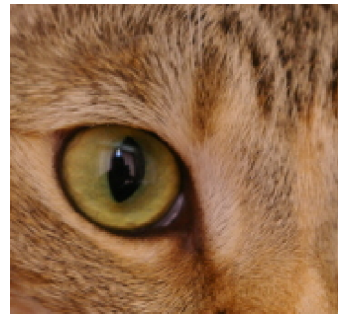


Figure 7.5: `cat[20:190, 110:290]`: a crop is just a slice.

Remember that ranges include the start and exclude the end, so `20:190` is 170 rows. The first index counts *down* from the top of the image, not up from the bottom as on a graph.

7.5 Taking the channels apart

Slicing along the last axis pulls out one channel at a time. Each is a two-dimensional grayscale image showing how much of that color is present (Figure 7.6).

img_channels

```
red, green, blue = cat[:, :, 0], cat[:, :, 1], cat[:, :, 2]

channels = [(red, 'red'), (green, 'green'), (blue, 'blue')]

plt.figure(figsize=(4.6, 1.4))
for i, (channel, name) in enumerate(channels):
    plt.subplot(1, 3, i + 1)
    plt.imshow(channel, cmap='gray', vmin=0, vmax=255)  # same scale
    plt.title(name)
    plt.axis('off')
plt.show()
```

Setting `vmin=0` and `vmax=255` draws all three channels on the same scale, so that their brightness can be compared. Left to itself, `imshow` stretches each image's own darkest and brightest values to black and white.



Figure 7.6: The red, green and blue channels, each drawn in gray. The orange fur is bright in red and dark in blue.

We can also edit channels. Setting the red and blue channels to zero leaves a green-only image:

img_only_green

```
only_green = cat.copy()
only_green[:, :, 0] = 0     # no red
only_green[:, :, 2] = 0     # no blue
plot(only_green)
```



Figure 7.7: Only the green channel remains.

7.6 A trap: overflow

Suppose we want a brighter image, so we multiply by two. Because the pixels are `uint8`, any result above 255 does not stop at 255. It wraps around and starts again from zero, like a car's odometer:

img_overflow

```
x = np.array([100, 200, 250], dtype=np.uint8)
print(x * 2)                # wraps around past 255!
print(x.astype(float) * 2)
```

OUTPUT

```
[200 144 244]
[200. 400. 500.]
```

$200 \times 2 = 400$, and $400 - 256 = 144$. On a photograph the result is psychedelic (Figure 7.8, left). The fix is to convert to floating point, do the arithmetic, *clip* the result back into the range 0 to 255, and only then convert back:

img_brighter

```
wrong = cat * 2
right = np.clip(cat.astype(float) * 2, 0, 255).astype(np.uint8)

plt.figure(figsize=(4.6, 1.8))
for i, (im, name) in enumerate([(wrong, 'cat * 2'), (right, 'clipped')]):
    plt.subplot(1, 2, i + 1)
    plt.imshow(im)
    plt.title(name)
    plt.axis('off')
plt.show()
```

This bug is common enough to be worth remembering. Whenever an image looks wildly, impossibly colorful after an edit, check the `dtype`.

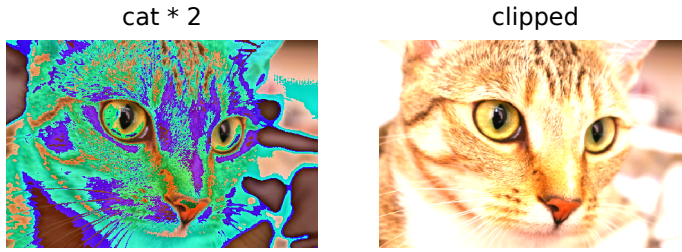


Figure 7.8: Left: `uint8` arithmetic wrapped around. Right: the same brightening done in floating point and clipped.

7.7 Gray, small, turned and flipped

Averaging the three channels gives a grayscale image. (Our eyes are most sensitive to green, so the standard conversion weights green most heavily, roughly 0.3 red, 0.6 green and 0.1 blue, but a plain average is good enough for us.) `mean` returns floating-point numbers, which is what we will want from now on anyway:

img_gray

```
gray = cat.mean(axis=2)     # average the three channels
print(gray.shape, gray.dtype)
```

OUTPUT

```
(300, 451) float64
```

scikit-image provides the standard geometric transformations. `resize` changes the number of pixels, which makes the image blurrier when shrinking, and `rotate` turns it. A mirror image needs no library at all: the slice `::-1` walks the columns backwards.

img_transforms

```
from skimage.transform import resize, rotate

small = resize(cat, (24, 36))     # now floats between 0 and 1
turned = rotate(cat, angle=30)
mirror = cat[:, ::-1]             # reverse the column order
print(small.shape, small.dtype, small.max())

versions = [(small, 'resized'), (turned, 'rotated'), (mirror, 'flipped')]

plt.figure(figsize=(4.6, 1.3))
for i, (im, name) in enumerate(versions):
    plt.subplot(1, 3, i + 1)
    plt.imshow(im)
    plt.title(name)
    plt.axis('off')
plt.show()
```

OUTPUT

```
(24, 36, 3) float64 0.8032994237411781
```

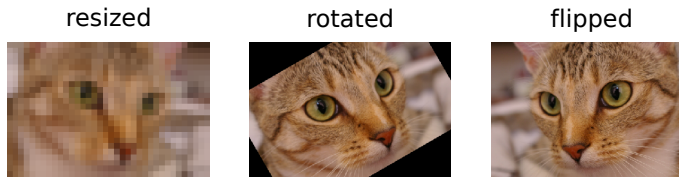


Figure 7.9: Resized to 24×36, rotated 30°, and mirrored.

Note that `resize` and `rotate` also convert the image to floats between 0 and 1. Deep learning libraries generally prefer that range to the 0–255 of image files, and we will follow them.

Flips, small rotations and crops are more than tricks. When we train networks, showing them flipped and shifted copies of each training image is one of the cheapest ways to help them generalize. A cat facing left is still a cat. We will use exactly that idea in Chapter 16.

7.8 How many shades?

We said that 256 shades are enough. How few would do? Each pixel of an 8-bit image is a choice among $2^8 = 256$ shades. With 4 bits there are 16 shades, with 2 bits four, and with 1 bit just black and white. Reducing the *bit depth* means rounding every pixel to the nearest of fewer shades (Figure 7.10):

img_bits

```
def reduce_bits(image, bits):
    levels = 2**bits                          # 8 bits: 256 shades
    q = np.round(image / 255 * (levels - 1)) # nearest shade, 0..levels-1
    return q / (levels - 1)                   # back to 0..1 for drawing

plt.figure(figsize=(4.6, 1.2))
for i, bits in enumerate([8, 4, 2, 1]):
    plt.subplot(1, 4, i + 1)
    plt.imshow(reduce_bits(gray, bits), cmap='gray', vmin=0, vmax=1)
    word = 'bit' if bits == 1 else 'bits'
    plt.title(f'{bits} {word}\n{2**bits} shades')
    plt.axis('off')
plt.show()
```

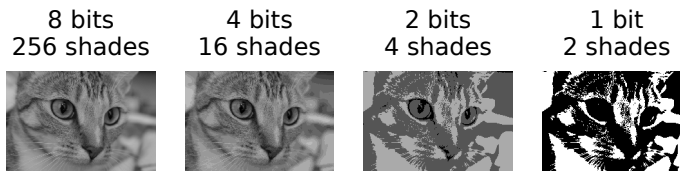


Figure 7.10: The gray cat at four bit depths.

Sixteen shades are hard to tell from 256. With four, smooth gradients break into flat bands, and with two the picture becomes a stark poster. Yet a newspaper prints photographs with only black ink on white paper, using a screen of dots that are larger where the picture is darker. A computer's version of the same idea is *dithering*: instead of rounding each pixel and forgetting the error, pass the error on to neighbors that have not been rounded yet. A pixel of 40 percent gray that is rounded down to black leaves a debt of 0.4, which makes its neighbors more likely to round up to white. Over any small patch, the fraction of white pixels then matches the original brightness. The Floyd–Steinberg method spreads each error to four neighbors in the proportions 7, 3, 5 and 1 sixteenths:

img_dither

```
def dither(image):
    """Floyd-Steinberg: black or white, passing each error on."""
    x = image / 255.0
    out = np.zeros_like(x)
    h, w = x.shape
    for r in range(h):
        for c in range(w):
            out[r, c] = 1.0 if x[r, c] >= 0.5 else 0.0
            err = x[r, c] - out[r, c]
            if c + 1 < w:
                x[r, c + 1] += err * 7 / 16
            if r + 1 < h:
                if c > 0:
                    x[r + 1, c - 1] += err * 3 / 16
                x[r + 1, c] += err * 5 / 16
                if c + 1 < w:
                    x[r + 1, c + 1] += err * 1 / 16
    return out

d = dither(gray)
print(np.unique(d), round(d.mean(), 3), round(gray.mean() / 255, 3))

plt.figure(figsize=(4.6, 1.75))
for i, (im, name) in enumerate([(reduce_bits(gray, 1), 'rounded'),
                                (d, 'dithered')]):
    plt.subplot(1, 2, i + 1)
    plt.imshow(im, cmap='gray', vmin=0, vmax=1)
    plt.title(name)
    plt.axis('off')
plt.show()
```

OUTPUT

```
[0. 1.] 0.452 0.452
```

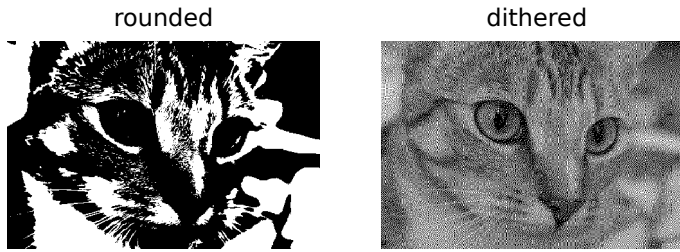


Figure 7.11: Two images with only black and white pixels. Left: each pixel rounded on its own. Right: dithered.

Every pixel of the dithered image is either 0 or 1, but its average brightness matches the original's to three decimal places, and from a little distance the eye averages the dots back into grays. Dithering is a small first example of an idea we will meet again: what a picture means lives in patterns of pixels, not in any single pixel.

7.9 How many images are there?

An image is a choice of shade for every pixel, so we can count the possible images. A 3×3 image in black and white has $2^9 = 512$ possibilities: small enough to print them all. Give each pixel 256 shades and the count is 256^9, a 22-digit number. Python's integers can hold numbers of any size, but we only need their length. That is what the base-10 logarithm counts: $\log_{10} 1000 = 3$, because $1000 = 10^3$, and a number has one more digit than its logarithm rounded down. The logarithm of a power is the power times the logarithm, so the logarithm of 256^9 is $9 \log_{10} 256$, and we never have to compute the number itself:

img count

```
from math import log10

for name, pixels, shades in [('3x3 black and white', 9, 2),
                             ('3x3 gray', 9, 256),
                             ('28x28 black and white', 784, 2),
                             ('28x28 gray', 784, 256),
                             ('the cat photo', 300 * 451 * 3, 256)]:
    digits = int(pixels * log10(shades)) + 1
    count = f'{shades}^{pixels}'
    print(f'{name:22s} {count:11s} has {digits:,} digits')
```

OUTPUT

```
3x3 black and white    2^9         has 3 digits
3x3 gray               256^9       has 22 digits
28x28 black and white  2^784       has 237 digits
28x28 gray             256^784     has 1,889 digits
the cat photo          256^405900  has 977,505 digits
```

The 28-pixel-square grayscale images of the next chapter could be any of a number with 1,889 digits. For comparison, the number of atoms in the observable universe is usually estimated to have about 80. Almost all of these images look like the ones in Figure 7.12:

img_noise

```
rng = np.random.default_rng(0)
noise = rng.integers(0, 256, size=(4, 28, 28))

plt.figure(figsize=(4.6, 1.2))
for i in range(4):
    plt.subplot(1, 4, i + 1)
    plt.imshow(noise[i], cmap='gray')
    plt.axis('off')
plt.show()
```

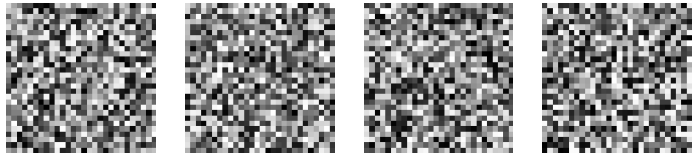


Figure 7.12: Four random 28×28 grayscale images, each pixel an independent random shade.

Pick an image at random and it is noise. Every handwritten digit, every face, every photograph ever taken lies in a vanishingly small corner of this space. That is good news. A classifier never has to deal with most possible images, only with the tiny, highly structured set that the world actually produces. Finding the structure of that set is what learning will mean.

7.10 Sound is an array too

Images are not the only data that are arrays. A microphone measures air pressure many thousands of times a second, and a recording is the list of measurements: a one-dimensional array. CD-quality audio takes 44,100 *samples* each second. A pure tone is a sine wave, and a chord is a sum of them:

img_sound

```
from scipy.io import wavfile

rate = 44_100                           # samples per second
t = np.arange(rate) / rate              # one second of time
a4 = np.sin(2 * np.pi * 440 * t)        # the A above middle C
print(t.shape, a4[:4])

chord = (np.sin(2 * np.pi * 440 * t) + np.sin(2 * np.pi * 554 * t)
         + np.sin(2 * np.pi * 659 * t)) / 3          # A major
wavfile.write('data/chord.wav', rate, (chord * 32767).astype(np.int16))

plt.figure(figsize=(4.6, 1.6))
plt.plot(t[:500] * 1000, a4[:500], label='A, 440 Hz')
plt.plot(t[:500] * 1000, chord[:500], label='A major chord')
plt.xlabel('milliseconds')
plt.legend(frameon=False, ncol=2, loc='lower center',
           bbox_to_anchor=(0.5, 1.0))
plt.show()
```

OUTPUT

```
(44100,) [0.         0.06264832 0.12505052 0.18696144]
```

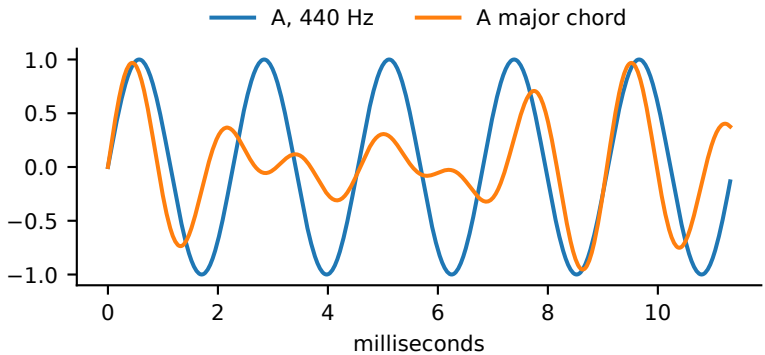


Figure 7.13: The first 11 milliseconds of a 440 Hz tone and of an A major chord, the sum of three tones.

A tone of 440 hertz (Hz) vibrates 440 times a second. Since a sine wave completes one cycle each time its input grows by 2π, the tone at time `t` seconds is `np.sin(2 * np.pi * 440 * t)`. `wavfile.write` saves the array as a sound file you can play. Sound files usually store each sample as a 16-bit whole number (`np.int16`), from −32,768 to 32,767, so we stretch the wave from the range −1 to 1 to fill it. The same array tools apply: adding arrays mixes sounds, multiplying by a number changes the volume, and a slice cuts out a moment, just as a slice cropped the cat.

So far we have been looking at one image at a time. In the next chapter we load sixty thousand of them, and let them become the data a machine learns from.

Try it

1. Type in an 8×8 image of your own initial and draw it with `plot`.
2. Read a photograph of your own with `imread`, print its shape, and crop out a face or an object.
3. Convert the cat to grayscale using the weights 0.3, 0.6 and 0.1 for red, green and blue, and plot the difference from the plain average. Where do the two disagree most?
4. `cat[:, :, ::-1]` reverses the order of the channels, swapping red and blue. Predict what the orange fur will look like, then check.
5. Compare `np.rot90(gray)` with `gray.T`. They are not the same picture. What is the difference?
6. Reduce each color channel of the cat to 2 bits with `reduce_bits` (64 colors in all) and draw the result. Then dither each channel separately; `np.stack([r, g, b], axis=2)` puts three channels back together. Which looks closer to the original?
7. How many different 28×28 images are there with exactly 100 white pixels and the rest black? (Python's `math.comb` counts the ways to choose which pixels are white.)
8. Make a one-second sound whose frequency rises from 220 Hz to 880 Hz. (Hint: make `f = np.linspace(220, 880, rate)`, one

frequency per sample. Each sample, the wave moves on by `f / rate` of a cycle, and `np.cumsum` adds up those steps, so `np.sin(2 * np.pi * np.cumsum(f) / rate)` is the sound.) Save it and listen.

Chapter 8

Digits as Vectors

Reading handwriting is one of the oldest problems in machine perception, and one of the most instructive. Every literate person does it without effort. Nobody can write down the rules they use, and the variety is enormous: no two people write a 7 the same way. In this chapter we meet the dataset on which a whole generation of researchers cut their teeth, MNIST, and we learn to see each handwritten digit the way a neural network will see it: as a single point in a space with 784 dimensions.

8.1 Sixty thousand digits

MNIST contains 70,000 small grayscale images of handwritten digits, collected from American high-school students and Census Bureau employees, each labeled with the digit it shows. The `torchvision` library, a companion to PyTorch, will download it for us.

dig_load

```
from torchvision.datasets import MNIST

train_set = MNIST('data', train=True, download=True)
test_set = MNIST('data', train=False, download=True)
print(len(train_set), len(test_set))
```

OUTPUT

```
60000 10000
```

The data is split in two. The 60,000 *training* images are what a model learns from. The 10,000 *test* images are kept aside and used only to measure how well it learned. A model is only useful if it does well on examples it has never seen; a student who memorizes the answer key has not learned arithmetic. We will be strict about this separation throughout the book.

The images are stored as one big array, which we convert to NumPy:

dig_arrays

```
X = train_set.data.numpy()
Y = train_set.targets.numpy()
X_test = test_set.data.numpy()
Y_test = test_set.targets.numpy()

print(X.shape, X.dtype)
print(Y.shape, Y[:10])
```

OUTPUT

```
(60000, 28, 28) uint8
(60000,) [5 0 4 1 9 2 1 3 1 4]
```

`X` is a stack of 60,000 images of 28×28 pixels, and `Y` holds the matching labels. We will use these letters throughout the book: `X` for inputs, `Y` for the answers, and `_test` for the held-out versions. Here is the first one:

dig_one

```
plot(X[0], title=f'label: {Y[0]}', size=2)
```

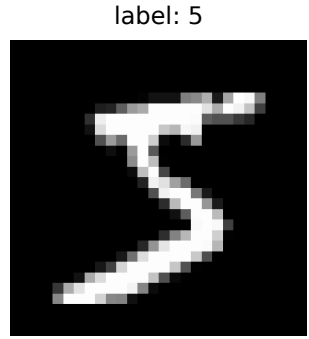


Figure 8.1: The first training image, labeled 5.

It is worth seeing the numbers behind the picture at least once. Dividing each pixel by 26 squashes the range 0–255 down to the single digits 0–9, small enough to print the whole image as text:

dig_ascii

```
for row in X[0] // 26:          # squash 0-255 down to the digits 0-9
    print(''.join(str(v) if v else '.' for v in row))
```

OUTPUT

```
..........................
..........................
..........................
..........................
..........................
.............45616994....
........1135699999869972....
.......1999999999933321.....
........8999997799..........
........364997..15..........
...........593..............
...........597..............
...........792..............
...........19864............
...........39994............
..........179951.........
.............3997.........
.............9992........
..........157997.........
...........15899997.........
...........48999973.........
.........28999973...........
.......68999973.............
....268999995...............
....5999855.................
..........................
..........................
..........................
```

8.2 Many at a glance

To look at many images at once we will use a *montage*, which tiles a stack of images into one big picture. Here is a helper we will use for the rest of the book. It separates the tiles with a one-pixel border, lets us choose the grid (`grid_shape`) and the width in inches, and has one option we will need for weights, which can be negative: `symmetric=True` draws zero as middle gray, positive values lighter and negative values darker.

dig_montage

```
from skimage.util import montage

def montage_plot(images, size=3.2, grid_shape=None, symmetric=False):
    images = np.asarray(images)
    lo, hi = images.min(), images.max()
    if symmetric:                       # zero is drawn as middle gray
        hi = np.abs(images).max()
        lo = -hi
    tiled = montage(images, grid_shape=grid_shape,
                    padding_width=1, fill=lo)
    plt.figure(figsize=(size, size * tiled.shape[0] / tiled.shape[1]))
    plt.imshow(tiled, cmap='gray', vmin=lo, vmax=hi)
    plt.axis('off')
    plt.show()

montage_plot(X[:36])
print(Y[:36].reshape(6, 6))
```

OUTPUT

```
[[5 0 4 1 9 2]
 [1 3 1 4 3 5]
 [3 6 1 7 2 8]
 [6 9 4 0 9 1]
 [1 2 4 3 2 7]
 [3 8 6 9 0 5]]
```

Look closely and you will see how hard the problem really is. Some 4s are closed at the top and some are open; 1s lean at different angles; the 2 at the end of the first row curls at its base, while those in the fifth row do not. The ten digits are not equally common, but they are close:

dig_counts

```
print(np.bincount(Y))
```

OUTPUT

```
[5923 6742 5958 6131 5842 5421 5918 6265 5851 5949]
```



Figure 8.2: The first 36 training digits. Their labels are printed above, in the same order.

8.3 From pictures to vectors

As in Chapter 7, we will work with pixel values between 0 and 1:

dig_scale

```
X = X / 255.0
X_test = X_test / 255.0
print(X.dtype, X.min(), X.max())
```

OUTPUT

```
float64 0.0 1.0
```

Now the key step. A 28×28 image has 784 pixels, and reshaping turns the grid into one long list, row after row. This is called *flattening*.

dig_vector

```
x = X[0].reshape(784)
print(x.shape)

plt.figure(figsize=(6, 1.6))
plt.plot(x)
plt.xlabel('pixel number')
plt.show()
```

OUTPUT

```
(784,)
```

Nothing is lost: `x.reshape(28, 28)` gives back the original image.

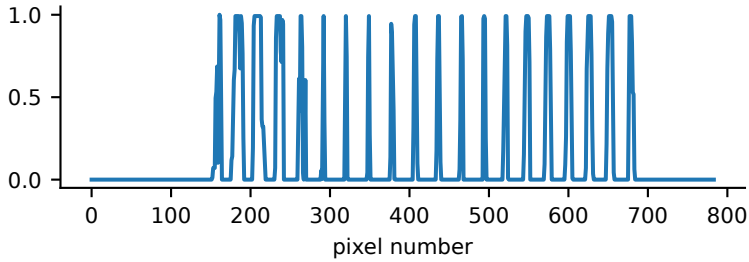


Figure 8.3: The same digit 5, flattened into a list of 784 numbers. The stroke shows up as bursts of bright pixels, one burst per image row.

Why throw away the grid? Because the weighted-sum machinery of Chapter 6 works on lists. Once each image is a row of 784 numbers, the whole training set is a single matrix, one image per row, ready to be multiplied:

dig_matrix

```
X = X.reshape(len(X), 784)
X_test = X_test.reshape(len(X_test), 784)
print(X.shape, X_test.shape)
```

OUTPUT

```
(60000, 784) (10000, 784)
```

A geometric picture helps here. A list of two numbers is a point on a plane; three numbers, a point in space. A list of 784 numbers is a point in a 784-dimensional space. Nobody can picture that space, but the mathematics does not care. Each handwritten digit is one point in it. Similar-looking images are nearby points, and the hope behind everything that follows is that the 5s cluster in one region, the 7s in another, and that a machine can learn where the boundaries between the regions lie.

8.4 A classifier with no learning

Before we let a machine learn anything, let's see how far simple averaging gets us. Average all the training images of each digit, pixel by pixel, and you get ten blurry "typical" digits (Figure 8.4).

dig_means

```
means = np.array([X[Y == k].mean(axis=0) for k in range(10)])
print(means.shape)
montage_plot(means.reshape(10, 28, 28), grid_shape=(2, 5))
```

OUTPUT

```
(10, 784)
```



Figure 8.4: The average image of each digit, 0 to 9.

These averages are *templates*. A simple way to classify a new image is to compare it with each template and pick the best match. The dot product from Chapter 6 measures how much two images overlap. Multiplying the whole test set by the matrix of templates gives a score for every image and every digit at once, and `argmax` picks the winner:

dig_template

```
scores = X_test @ means.T       # (10000, 784) @ (784, 10)
guesses = np.argmax(scores, axis=1)
print(scores.shape)
print(guesses[:10])
print(Y_test[:10])
print(np.mean(guesses == Y_test))
```

OUTPUT

```
(10000, 10)
[7 2 1 0 4 1 8 9 0 9]
[7 2 1 0 4 1 4 9 5 9]
0.631
```

Sixty-three percent correct, with no learning at all. Random guessing would get ten percent. But the dot product has a flaw: it rewards ink. A template with lots of bright pixels, like 0 or 8, overlaps

with almost everything. A fairer comparison is the *distance* between the image and each template, the sum of squared pixel differences. For each of the ten templates `m`, subtract it from every test image, square, and add up each row; `np.stack` then puts the ten columns of distances side by side, and the smallest distance in each row wins:

dig distance plain

```
dists = np.stack([((X_test - m) ** 2).sum(axis=1) for m in means],
                 axis=1)                    # one column per template
guesses = np.argmin(dists, axis=1)
print(np.mean(guesses == Y_test))
```

OUTPUT

```
0.8203
```

Eighty-two percent. If you like algebra, here is the same calculation done faster (it is safe to skip). Expanding $(x-m)^2 = x^2-2x\cdot m+m^2$ turns the middle part into the same matrix multiplication as the templates used, and gives the same answer:

dig_distance

```
dists = ((X_test ** 2).sum(axis=1, keepdims=True)    # |x|^2
         - 2 * X_test @ means.T                      # -2 x.m
         + (means ** 2).sum(axis=1))                 # |m|^2
guesses = np.argmin(dists, axis=1)
print(dists.shape)
print(np.mean(guesses == Y_test))
```

OUTPUT

```
(10000, 10)
0.8203
```

Eighty-two percent is a respectable baseline, and it took one line of averaging. Keep both numbers in mind. In Part III we will stop designing the templates ourselves and let the machine find its own. It will do much better.

8.5 A harder set: Fashion-MNIST

MNIST has become so easy for modern methods that researchers created a drop-in replacement with exactly the same format: 70,000 28×28 grayscale images in ten classes. These are photographs of clothing from the Zalando online catalog. This is Fashion-MNIST.

dig_fashion

```
from torchvision.datasets import FashionMNIST

fashion = FashionMNIST('data', train=True, download=True)
print(fashion.classes)
montage_plot(fashion.data[:36].numpy())
```

OUTPUT

```
['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt',
    ↪ 'Sneaker', 'Bag', 'Ankle boot']
```



Figure 8.5: The first 36 images of Fashion-MNIST.

Shirts, T-shirts, pullovers and coats look a lot alike at this resolution, which makes Fashion-MNIST a good deal harder. Because the format is identical, every piece of code in this book that works on one dataset works on the other by changing a single word. We will use digits when we want to see clearly what a model is doing, and clothing when we want a challenge.

Try it

1. Find the test images that the distance classifier gets wrong (a mask does it: `guesses != Y_test`) and look at the first 36 with `montage_plot`. Would you have got them right?

2. Which digit does the distance classifier get wrong most often? Use a mask for each digit in turn.

3. Build the templates from only the first 100 training images of each digit. How much does the accuracy drop?

4. Repeat the template and distance classifiers on Fashion-MNIST. Which kinds of clothing do the templates handle worst?

Part III

Learning from Data

Chapter 9

Chance

The last chapter made a promise: we will stop designing the templates ourselves and let the machine find its own. The machine will start where every learner starts, by guessing. Its first weights will be random numbers, the examples it studies will be shuffled into a random order, and in the next chapter its very first strategy will be nothing but random guessing. Before we hand the search over to chance, then, it is worth getting to know chance.

This is a short chapter about randomness with NumPy. We will make random numbers from scratch and see how they can fail, flip coins until a bell curve appears, and use random darts to compute π. Along the way we will meet the two facts about randomness that matter most for deep learning: sums of many random things are predictable, and their spread grows only as the square root of how many there are. Both will come back: in how a network's starting weights are chosen (Chapter 12), and in how much to trust a measured accuracy (Appendix A). A last, optional section lets simulation settle three puzzles that fool most people.

9.1 A recipe for chaos

A computer cannot flip a coin. Everything it does follows instructions, so its "random" numbers come from a recipe, a *pseudo-random number generator*, that turns one number into the next in a way that is hard to predict. Here is a toy recipe: take a number between 0 and 1, triple it, and keep only the part after the decimal point.

ch_recipe

```
x = 0.12345                # the seed
xs = []
for i in range(10):
    x = 3 * x % 1          # triple it, keep the part after the point
    xs.append(round(x, 4))
print(xs)
```

OUTPUT

```
[0.3704, 0.1111, 0.3332, 0.9995, 0.9984, 0.9951, 0.9852, 0.9555, 0.8664,
    ↪ 0.5991]
```

Each number is the *seed* of the next. Run it fifty thousand times and the values look scattered and cover the interval evenly (Figure 9.1).

ch_recipe_hist

```
x = 0.12345
xs = np.zeros(50_000)
for i in range(len(xs)):
    x = 3 * x % 1
    xs[i] = x

plt.figure(figsize=(4.6, 1.8))
plt.subplot(1, 2, 1)
plt.plot(xs[:200], '.', ms=2)
plt.title('the first 200')
plt.subplot(1, 2, 2)
plt.hist(xs, bins=50)
plt.title('all 50,000')
plt.show()
```

Even here, look closely at the printed list: 0.9995, 0.9984, 0.9951, 0.9852... When a value lands near a third, the next few numbers creep along near 1 in a pattern no coin would make. And there is a deeper problem. Each step triples any small error in the number, along with the number itself. The computer stores the seed with an error of about one part in 10^{16}, and 3^{34} is about 10^{16}, so after some 34 steps the digits we are looking at are nothing but magnified round-off: a

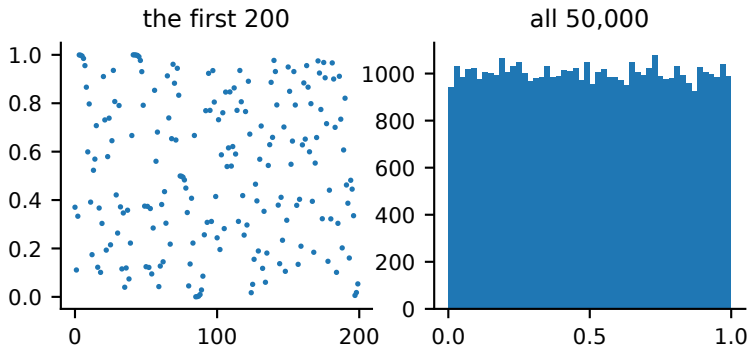


Figure 9.1: Fifty thousand numbers from the tripling recipe. Left: the first 200 in order. Right: how often each value occurs.

different computer, or a seed changed in its last digit, gives a completely different sequence. That extreme sensitivity to the starting point is what mathematicians call *chaos*, and it is exactly what makes the recipe look random.

A small change shows how badly a recipe can fail. Doubling instead of tripling looks just as reasonable on paper:

ch_doubling

```
x = 0.12345
for i in range(60):
    x = 2 * x % 1
    if i % 10 == 9:
        print(i + 1, x)
```

OUTPUT

```
10 0.4128000000000043
20 0.7072000000043772
30 0.1728000044822693
40 0.94720458984375
50 0.9375
60 0.0
```

After a few dozen steps it is stuck at exactly zero, forever. The computer stores a number in binary, as a string of 1s and 0s with about 53 significant binary digits, and doubling shifts those digits one place to the left. Once they have all been shifted out, nothing is left. Good generators are designed by specialists and tested for exactly these failures. NumPy's current one is called PCG64, and it is what `default_rng`, which we met in Chapter 6, creates. Its seed plays

the same role as our 0.12345: the same seed, the same sequence. In this chapter `rng = np.random.default_rng(seed=0)` makes the random numbers.

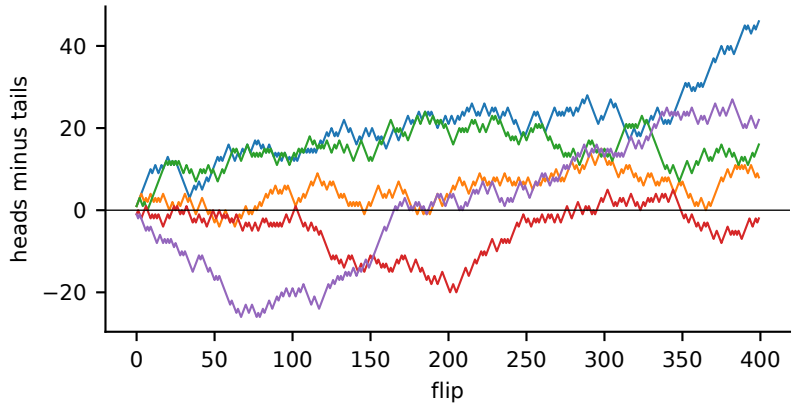


Figure 9.2: Five random walks of 400 coin flips each.

9.2 Flipping coins

A coin flip is a choice between two values. Counting heads as +1 and tails as −1, the sum of ten flips is "heads minus tails":

ch_coin

```
flips = rng.choice([-1, 1], size=10)
print(flips)
print(flips.sum())
```

OUTPUT

```
[ 1  1  1 -1 -1 -1 -1 -1 -1  1]
-2
```

Keep a running total, flip after flip, and we get a *random walk*: a path that drifts up and down with no plan at all (Figure 9.2). `cumsum` computes all the running totals in one call.

ch_walks

```
steps = rng.choice([-1, 1], size=(5, 400))
paths = steps.cumsum(axis=1)       # running total after each flip

plt.figure(figsize=(4.6, 2.2))
plt.plot(paths.T, lw=0.8)
plt.axhline(0, color='k', lw=0.5)
plt.xlabel('flip')
plt.ylabel('heads minus tails')
plt.show()
```

Each walk wanders off in its own direction, and some go a long

way. A walker who is 30 ahead after 400 flips may feel that luck is on their side, but every flip is still fifty-fifty. Now play the game a hundred thousand times, a hundred flips each, and keep only the final scores:

ch_sums

```
N = 100                                          # flips per game
games = rng.choice([-1, 1], size=(100_000, N))
totals = games.sum(axis=1)
print(totals[:10])
print(totals.mean(), totals.std())
```

OUTPUT

```
[ -4 -10  12  12   2  12 -12  14   6   4]
0.04416 9.984386305346964
```

Single games end all over the place, but the hundred thousand games together are very predictable. The average is almost exactly zero, and the typical distance from zero, the *standard deviation* that `std` computes, is almost exactly 10.

9.3 Pascal's triangle and the bell curve

Why do the scores pile up near zero? Count the ways to get there. Each row of *Pascal's triangle* is made by adding the two numbers above:

ch_pascal

```
rows = 7
P = np.zeros((rows, rows + 1), dtype=int)
P[0, 1] = 1
for i in range(1, rows):
    for j in range(1, rows + 1):
        P[i, j] = P[i - 1, j - 1] + P[i - 1, j]
print(P[:, 1:])
```

OUTPUT

```
[[ 1  0  0  0  0  0  0]
 [ 1  1  0  0  0  0  0]
 [ 1  2  1  0  0  0  0]
 [ 1  3  3  1  0  0  0]
 [ 1  4  6  4  1  0  0]
 [ 1  5 10 10  5  1  0]
 [ 1  6 15 20 15  6  1]]
```

Read row 6 as six flips. There is one way to get six heads, six ways to get five (the single tail can be any of the six flips), fifteen

ways to get four, and twenty ways to get three heads and three tails. Every sequence of flips is equally likely, so the outcome with the most sequences leading to it is the most likely outcome. Python's `comb` gives the entries of any row without building the triangle, so we can compute the exact chances for a hundred flips and compare them with our simulated games (Figure 9.3):

ch_binomial

```
from math import comb

heads = np.arange(N + 1)
paths_to = np.array([comb(N, k) for k in heads], dtype=float)
prob = paths_to / 2**N          # every path is equally likely
total = 2 * heads - N           # heads minus tails

plt.figure(figsize=(4.6, 2.2))
plt.hist(totals, bins=np.arange(-41, 42, 2), density=True,
         label='100,000 games')
plt.plot(total, prob / 2, 'k.', ms=3, label="Pascal's triangle")
plt.xlim(-40, 40)
plt.xlabel('heads minus tails after 100 flips')
plt.legend(frameon=False)
plt.show()
```

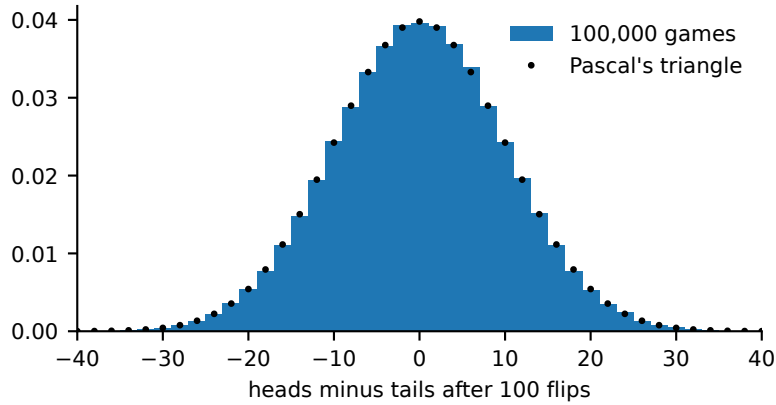


Figure 9.3: Bars: the scores of 100,000 simulated games of 100 flips. Dots: the exact chances, counted from row 100 of Pascal's triangle.

The dots sit on the bars. (A detail you can skip: with `density=True`, a bar's height is probability per unit of score. Each bar is two units wide and holds exactly one possible score, since a score is always even,

so a bar stands at half that score's probability, and the dots are halved to match.) Francis Galton built a machine that does this experiment with falling balls: each ball bounces left or right off row after row of pins, and the balls pile up into the same shape at the bottom. The web edition of this book has one you can run.

That shape is the *normal distribution*, or bell curve, which we sampled in Chapter 6. Its formula has a mean μ for the center and a standard deviation σ for the width; e^x is `np.exp(x)`, which we will meet properly in Chapter 11:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}}\, e^{-(x-\mu)^2/2\sigma^2}.$$

ch_normal

```
def normal_pdf(x, mu, sigma):
    return np.exp(-(x - mu)**2 / (2 * sigma**2)) / np.sqrt(2 * np.pi
                                                           * sigma**2)

z = rng.normal(loc=0, scale=1, size=100_000)
grid = np.linspace(-4, 4, 200)

plt.figure(figsize=(4.6, 2.0))
plt.hist(z, bins=80, density=True)
plt.plot(grid, normal_pdf(grid, 0, 1), 'k', lw=1)
plt.show()

print(np.mean(np.abs(z) < 1), np.mean(np.abs(z) < 2))
```

OUTPUT

```
0.68321 0.9551
```

About 68 percent of the numbers fall within one standard deviation of the mean, and about 95 percent within two. Those two percentages are worth remembering.

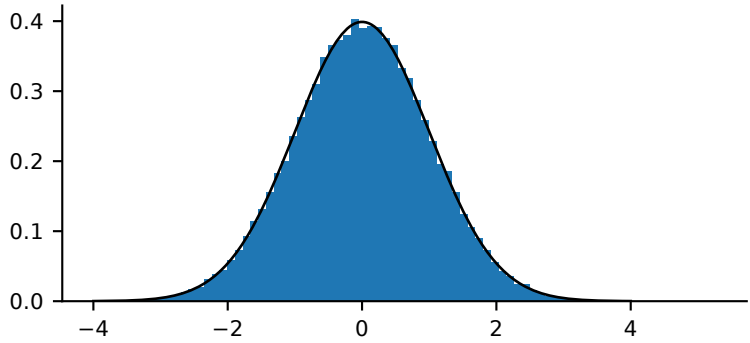


Figure 9.4: A histogram of 100,000 normal random numbers, with the bell curve's formula drawn over it.

9.4 The square-root law

How wide is the bell for a different number of flips? Let's measure:

ch_sqrt

```
for n in [1, 4, 16, 64, 256, 1024]:
    t = rng.choice([-1, 1], size=(20_000, n)).sum(axis=1)
    print(f'{n:5d} flips   spread {t.std():6.2f}   sqrt(N) {n**0.5:5.1f}')
```

OUTPUT

```
    1 flips   spread   1.00   sqrt(N)   1.0
    4 flips   spread   2.01   sqrt(N)   2.0
   16 flips   spread   4.02   sqrt(N)   4.0
   64 flips   spread   8.03   sqrt(N)   8.0
  256 flips   spread  15.92   sqrt(N)  16.0
 1024 flips   spread  31.97   sqrt(N)  32.0
```

The spread is the square root of the number of flips, almost exactly. Sixty-four times as many flips spread the scores only eight times as far. This *square-root law* is everywhere. It is why a poll of a thousand people is good to a few percent, and why a casino, which plays millions of games, can be sure of its profit while any single gambler can win.

Nor is there anything special about coins. Add up twelve uniform random numbers between 0 and 1, subtract 6, and the bell curve appears again (Figure 9.5):

ch_sums_of_anything

```
u = rng.random((100_000, 12))          # twelve uniform numbers per row
s = u.sum(axis=1) - 6                  # their sum, centered on zero
print(round(s.mean(), 3), round(s.std(), 3))

plt.figure(figsize=(4.6, 2.0))
plt.hist(s, bins=80, density=True)
plt.plot(grid, normal_pdf(grid, 0, 1), 'k', lw=1)
plt.show()
```

OUTPUT

```
-0.001 1.0
```

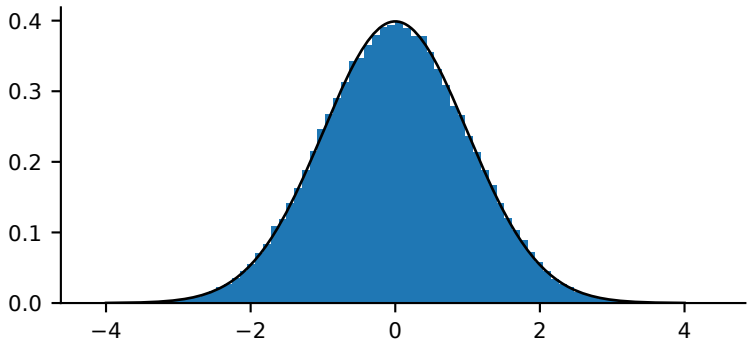


Figure 9.5: Each value is the sum of twelve uniform random numbers, minus 6. The curve is the standard bell curve from Figure 9.4.

This is the *central limit theorem*: add up many independent random numbers, of almost any kind, and the total follows a bell curve, with a spread that grows as the square root of how many were added. Twelve is a convenient number: one uniform number has a standard deviation of $\sqrt{1/12}$, so by the square-root law twelve of them have a standard deviation of exactly 1. For decades this was a standard way to make normal random numbers.

Here is why a book about neural networks cares. A neuron computes a dot product, a sum of hundreds of products, and when its weights are random each product is a random number. Give 10,000 random neurons the same 784-pixel input:

ch_neuron
```
x = rng.random(784)                          # an "image" of 784 pixels
W = rng.normal(size=(784, 10_000))           # 10,000 random neurons
scores = x @ W
print(round(scores.std(), 2), round(np.sqrt(np.sum(x**2)), 2))
```

OUTPUT
```
15.79 15.88
```

The scores spread out with a standard deviation equal to the *length* of the input: square each of the 784 pixels, add them up, and take the square root. In symbols that is $\sqrt{\sum x_i^2}$, where the Greek Σ (sigma) means "add up," and it is the straight-line size of the input treated as an arrow in 784 dimensions, just as the square-root law predicts. Stack layers of such neurons and this spread multiplies from layer to layer, which is why the starting size of the weights matters so much. We will meet the problem, and its square-root cure, in Chapter 12.

9.5 Throwing darts

Randomness can also compute things that have nothing random about them. Throw darts uniformly at a square two units wide, with a circle of radius 1 inside it. The circle covers $\pi/4$ of the square, so about $\pi/4$ of the darts land inside, and four times the fraction that do is an estimate of π (Figure 9.6).

ch_pi
```
n = 1_000_000
pts = rng.random((n, 2)) * 2 - 1             # darts in the square
inside = (pts**2).sum(axis=1) < 1            # ...that land in the circle
print(4 * inside.mean())

m = 2000
plt.figure(figsize=(2.4, 2.4))
plt.scatter(*pts[:m].T, s=1, c=np.where(inside[:m], '#f97306', '#888'))
plt.gca().set_aspect('equal')
plt.axis('off')
plt.show()
```

OUTPUT
```
3.14062
```

This is *Monte Carlo* estimation, named after the casino, and the same idea measures any area: throw darts at a box and count the

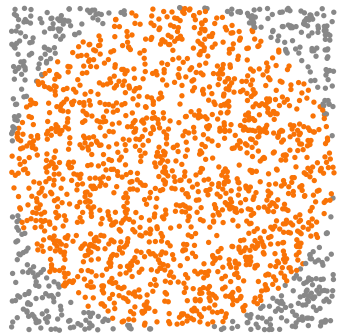


Figure 9.6: Two thousand of the million darts. Orange darts landed inside the circle.

fraction that land inside the shape. How accurate is it? A million darts gave π to two decimal places, not six. Repeating the experiment for different numbers of darts shows the trend (Figure 9.7). Both axes of the plot are *logarithmic*: each step along an axis multiplies by ten, so that a quantity that falls as $1/\sqrt{n}$ draws a straight line.

ch_error

```
true_pi = np.pi
sizes = np.logspace(1, 6, 26).astype(int)
errors = []
for n in sizes:
    trials = [4 * np.mean((rng.random((n, 2))**2).sum(axis=1) < 1)
              for _ in range(20)]
    errors.append(np.mean(np.abs(np.array(trials) - true_pi)))

plt.figure(figsize=(4.6, 2.2))
plt.loglog(sizes, errors, '.-', label='typical error')
plt.loglog(sizes, 1.3 / np.sqrt(sizes), 'k--', lw=0.8,
           label='1.3 / sqrt(n)')
plt.xlabel('darts')
plt.legend(frameon=False)
plt.show()
```

The square-root law again, now working against us: each extra decimal place of accuracy costs a hundred times as many darts. The same law governs every accuracy we will measure in this book. An accuracy measured on 50 images is an estimate from 50 darts, uncertain by several percentage points; on 10,000 images it is good to a fraction

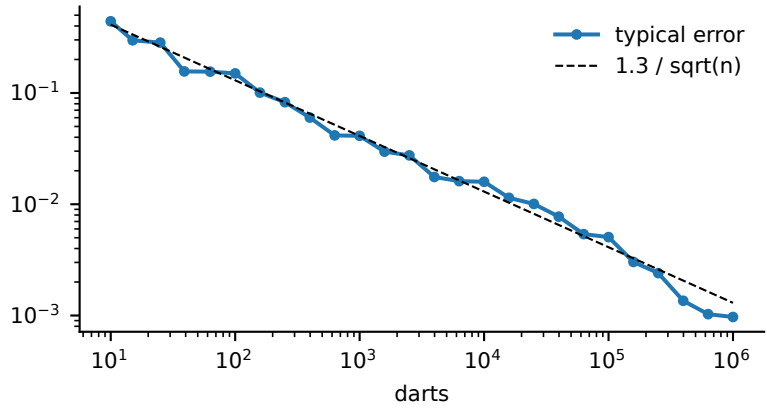


Figure 9.7: The average error in π against the number of darts, on logarithmic axes. The dashed line falls as $1/\sqrt{n}$.

of a point. We will need that when we compare results.

9.6 Excursion: when intuition fails

This last section is for fun; nothing later depends on it. Simulation is the quickest way to settle an argument, as long as it plays the game exactly as described. Three classic puzzles fool almost everyone.

How many people must be in a room before two of them probably share a birthday? Most people guess well over a hundred. We fill a room with random birthdays, sort them, and check for two equal neighbors:

ch_birthday

```
def shared_birthday(people, trials=20_000):
    days = rng.integers(0, 365, size=(trials, people))
    days.sort(axis=1)
    return np.mean((np.diff(days, axis=1) == 0).any(axis=1))

for people in [10, 20, 23, 30, 50, 70]:
    print(people, shared_birthday(people))
```

OUTPUT

```
10 0.1186
20 0.41085
23 0.50275
30 0.7063
50 0.97155
70 0.99935
```

Twenty-three people are enough for an even chance, and with seventy a shared birthday is all but certain. The intuition fails because it counts the wrong thing. Twenty-three people make 253 different pairs, and each pair has a small chance of matching.

You pick one of three doors. A car is behind one of them, goats behind the others. The host, who knows where the car is, opens a different door to show a goat and offers to let you switch. Should you? This is the *Monty Hall problem*, after the host of the television game show that used doors this way. Many people are certain it makes no difference. The simulation plays the game exactly as described, host and all. Numbering the doors 0, 1 and 2, the three numbers add up to 3, so once two doors are known, `3 - one - other` is the third:

ch_monty

```
trials = 100_000                       # doors are numbered 0, 1, 2
car = rng.integers(0, 3, trials)
pick = rng.integers(0, 3, trials)
other = (pick + rng.integers(1, 3, trials)) % 3   # a door not picked
host = np.where(pick == car, other,     # two goats left: either one
                3 - pick - car)         # otherwise: the one goat left
switch = 3 - pick - host                # the door neither of them chose
print('stay  ', np.mean(pick == car))
print('switch', np.mean(switch == car))
```

OUTPUT

```
stay   0.33473
switch 0.66527
```

Switching wins two times in three. The simulation shows why, too: when your first pick was right, the host can open either goat door; when it was wrong, which happens two times in three, the host has no choice, and the door left for you to switch to is the car.

A cereal brand puts one of 50 different cards in every box. How many boxes must you buy to collect them all? This is the *coupon collector's problem*:

ch_collector

```
def boxes_needed(n):
    seen, count = set(), 0
    while len(seen) < n:
        seen.add(int(rng.integers(n)))
        count += 1
    return count

n = 50
runs = [boxes_needed(n) for _ in range(5000)]
print(np.mean(runs), n * sum(1 / k for k in range(1, n + 1)))
```

OUTPUT

```
224.0996 224.96026691647114
```

About 225, four and a half times the number of cards, and the simulation agrees with the formula $n\,(1+\frac{1}{2}+\frac{1}{3}+\cdots+\frac{1}{n})$ printed beside it. The first cards come quickly; the last one takes on average 50 boxes by itself.

9.7 What chance teaches

Randomness will be everywhere from here on, so here is what to carry forward:

- Random numbers come from a recipe and a seed. Fix the seed and an experiment repeats exactly.
- A single random outcome tells you little. Many outcomes, added up or averaged, are predictable: they follow a bell curve.
- Spread grows as the square root of how many things are added. That is why averages over more data are more trustworthy, and why the starting size of a network's weights must shrink as layers get wider.
- When the mathematics is hard, simulate, but simulate the real situation. A few lines of NumPy can check a formula, or replace one.

In the next chapter we put chance to work. A machine that knows nothing about digits will guess its weights at random, keep the guesses that score better, and slowly learn to read.

Try it

1. Change the tripling recipe in `ch_recipe_hist` to multiply by 4 and then by 3.1. Which collapses to zero? (Multiplying by 4 shifts the binary digits two places at a time.)

2. Make the coins in `ch_sums` unfair: `rng.choice([-1, 1], size=..., p=[0.45, 0.55])`. Where is the bell's center now, and is its spread still close to 10?

3. The square-root law says that four times as many darts should halve the error. Check it with `ch_pi` at 10,000 and 40,000 darts, repeating each a few times.

4. Estimate the area of a quarter circle of radius 1 in a different way: throw darts `x` and `y` between 0 and 1 and count the fraction with `y < np.sqrt(1 - x**2)`. Is it closer to $\pi/4$ than `ch_pi`'s estimate with the same number of darts?

5. Using `shared_birthday`, find the smallest room in which the chance of a shared birthday passes 99 percent.

6. A harder one: rewrite `ch_monty` for 100 doors, where the host opens 98 goat doors and leaves one other door closed. What are the chances of winning by staying and by switching?

Chapter 10

Learning by Guessing

In Chapter 8 we built a classifier by hand. We averaged the images of each digit into a template, scored new images against the templates with one matrix multiplication, and picked the winner with `argmax`. The whole classifier was a 784×10 matrix: one column of 784 weights per digit.

Now we ask a different question. Forget averaging. Is there some other 784×10 matrix that classifies digits even better, and could a computer find it without being told what a "typical 5" looks like? This chapter answers that question in the bluntest way possible: by guessing. It is not how neural networks are usually trained. But it shows, with nothing hidden, what "learning" means for a machine. Learning is a search through a space of possible weights for the ones that work.

10.1 Enter PyTorch

From here on we will do our heavy arithmetic in PyTorch. A PyTorch *tensor* is an array, much like a NumPy array, with two extra powers: it can live on a graphics processor, and it can keep track of the calculations done with it so that derivatives come for free. We will need the second power in the next chapter. For now, tensors look and behave like the arrays we already know:

gs_tensor

```
import torch

t = torch.tensor([1.0, 2.0, 3.0])
print(t * 2)
print(t @ t)
print(t.shape, t.dtype)
```

OUTPUT

```
tensor([2., 4., 6.])
tensor(14.)
torch.Size([3]) torch.float32
```

Converting between the two libraries is easy:

gs_numpy

```
import numpy as np

a = np.arange(6).reshape(2, 3)
t = torch.tensor(a)
print(t)
print(t.numpy())
```

OUTPUT

```
tensor([[0, 1, 2],
        [3, 4, 5]])
[[0 1 2]
 [3 4 5]]
```

The two libraries agree on almost everything, but a few names differ, and they trip up everyone who moves between them:

NumPy	PyTorch
`a.sum(axis=1)`	`t.sum(dim=1)`
`keepdims=True`	`keepdim=True`
`a.astype(float)`	`t.float()`
`a.max(axis=1)`	`t.max(dim=1).values`
`a.argmax(axis=1)`	`t.max(dim=1).indices`

10.2 The graphics card

Graphics processors were built to compute the colors of millions of pixels at once for video games. That turns out to be exactly the kind of work neural networks need: the same simple arithmetic repeated over huge arrays. The cheap, massively parallel graphics card is one of the main reasons deep learning took off when it did. PyTorch can use an NVIDIA card (`'cuda'`), the graphics hardware built into Apple-silicon Macs (`'mps'`), or fall back to the ordinary processor (`'cpu'`). This snippet picks the best one available:

gs_device

```
if torch.cuda.is_available():
    device = 'cuda'                     # an NVIDIA graphics card
elif torch.backends.mps.is_available():
    device = 'mps'                      # an Apple-silicon Mac
else:
    device = 'cpu'
print(device)
```

OUTPUT

```
mps
```

The output in this book was produced on a Mac, so it says `mps`. Yours may say something else; everything still works, only the speed changes. Random numbers are generated differently on different hardware, so your exact numbers in this and later chapters will differ slightly from the ones printed here, though the story they tell should not.

We load MNIST exactly as in Chapter 8: each image flattened into a row of 784 numbers and scaled to 0–1.

gs_load

```
import numpy as np
from torchvision.datasets import MNIST

train_set = MNIST('data', train=True, download=True)
test_set = MNIST('data', train=False, download=True)
X = train_set.data.numpy().reshape(-1, 784) / 255.0
Y = train_set.targets.numpy()
X_test = test_set.data.numpy().reshape(-1, 784) / 255.0
Y_test = test_set.targets.numpy()
```

Then we turn the NumPy arrays into tensors on the device. Labels are whole numbers, so they keep an integer type.

gs_to_device
```
X = torch.tensor(X, dtype=torch.float32, device=device)
Y = torch.tensor(Y, device=device)
X_test = torch.tensor(X_test, dtype=torch.float32, device=device)
Y_test = torch.tensor(Y_test, device=device)
print(X.shape, X.device)
```

OUTPUT
```
torch.Size([60000, 784]) mps:0
```

As we saw in Chapter 9, a computer's random numbers come from a recipe that starts at a *seed*. PyTorch has its own generator, and fixing its seed makes every run of the code repeat exactly:

gs_seed
```
torch.manual_seed(0)    # the same "random" numbers on every run
```

10.3 One random guess

Let's make a completely random weight matrix and see how it does:

gs_one_guess
```
W = torch.randn(784, 10, device=device)

scores = X @ W              # (60000, 784) @ (784, 10)
guesses = scores.argmax(dim=1)
print(guesses[:10].cpu())
print(Y[:10].cpu())
print(round((guesses == Y).float().mean().item(), 4))
```

OUTPUT
```
tensor([4, 7, 9, 7, 2, 8, 0, 9, 7, 9])
tensor([5, 0, 4, 1, 9, 2, 1, 3, 1, 4])
0.0628
```

About six percent correct, which is no better than chance (ten percent with ten classes, give or take luck). We will be checking the accuracy of weight matrices a lot, so let's give it a name:

gs_accuracy
```
def matrix_accuracy(W, X, Y):
    guesses = (X @ W).argmax(dim=1)
    return (guesses == Y).float().mean().item()
```

10.4 Guess a thousand times

If one random matrix is useless, perhaps the best of a thousand is not. The loop below tries a thousand random matrices and prints a line each time it finds a new best one:

gs_search

```
best_acc = 0
for i in range(1000):
    W = torch.randn(784, 10, device=device)
    acc = matrix_accuracy(W, X, Y)
    if acc > best_acc:
        best_acc, W_best = acc, W
        print(i, round(acc, 4))
```

OUTPUT

```
0 0.062
1 0.1441
17 0.1503
53 0.1591
72 0.1611
134 0.1664
155 0.175
238 0.2394
```

Some random matrices are better than others. The best gets about 24 percent of the training images right, more than twice chance. But the improvements arrive more and more rarely, and that is the problem with pure guessing. A weight matrix has $784 \times 10 = 7{,}840$ numbers. The space of possible matrices is so vast that the chance of stumbling onto a good one at random is effectively zero.

10.5 Small steps

A better strategy is to stop starting from scratch. Take the best matrix so far, nudge every weight by a little random noise, and keep the result only if it scores better. Repeat. This is *hill climbing*: blindfolded on a hillside, you take a random step and keep it only if you went up.

To make each step quicker we score on the first 10,000 training images rather than all 60,000:

gs_climb

```
# a smaller set makes each guess quicker
X_small, Y_small = X[:10_000], Y[:10_000]

W = W_best.clone()
best_acc = matrix_accuracy(W, X_small, Y_small)
step = 0.1

for i in range(20_000):
    W_new = W + step * torch.randn(784, 10, device=device)
    acc = matrix_accuracy(W_new, X_small, Y_small)
    if acc > best_acc:
        W, best_acc = W_new, acc
    if i % 4000 == 0:
        print(i, round(best_acc, 4))
print('final', round(best_acc, 4))
```

OUTPUT

```
0 0.239
4000 0.6698
8000 0.6809
12000 0.6811
16000 0.6824
final 0.6829
```

Sixty-eight percent. Most of the gain came in the first few thousand steps, after which progress slowed to a crawl: the last 16,000 steps added only about one point. What really matters is how the matrix does on images it was never scored on:

gs_climb_test

```
print(round(matrix_accuracy(W, X_test, Y_test), 4))
```

OUTPUT

```
0.6712
```

Sixty-seven percent on the test set. The machine has found a matrix that does better than our hand-made dot-product templates from

Chapter 8 (63 percent), and nobody told it what any digit looks like. It knew only how many answers each guess got right.

10.6 Survival of the fittest

Hill climbing follows a single path, one guess at a time. Nature has another trick: keep a whole *population* of candidates, let the fittest survive, and fill the next generation with their mutated offspring. The graphics card lets us score a hundred candidate matrices at once. We stack them into one tensor `M` of shape (100, 784, 10) and score them all in a single operation. When one side of `@` has an extra axis in front, PyTorch repeats the multiplication for each entry along it, so `X_small @ M` is `X_small @ M[n]` for all hundred values of n at once, giving scores of shape (100, 10000, 10). After scoring, `parents.repeat(N // keep, 1, 1)` makes ten copies of the ten survivors, one after another, so the first ten children are the survivors themselves and only the other ninety are mutated.

gs_population

```
N, keep = 100, 10       # population size, number of survivors
M = torch.randn(N, 784, 10, device=device)    # N candidate matrices

for generation in range(1001):
    scores = X_small @ M                          # (N, 10000, 10)
    guesses = scores.argmax(dim=2)                  # (N, 10000)
    fitness = (guesses == Y_small).float().mean(dim=1)
    best = fitness.argsort(descending=True)[:keep]  # the survivors
    parents = M[best]
    children = parents.repeat(N // keep, 1, 1)   # 10 copies of the 10
    noise = torch.randn(N - keep, 784, 10, device=device)
    children[keep:] += 0.05 * noise       # mutate all but the survivors
    M = children
    if generation % 200 == 0:
        print(generation, round(fitness[best[0]].item(), 4))

W_evolved = M[0]
print('test', round(matrix_accuracy(W_evolved, X_test, Y_test), 4))
```

OUTPUT

```
0 0.1763
200 0.5456
400 0.6094
600 0.7438
800 0.7782
1000 0.7914
test 0.7687
```

Each generation, the ten best matrices survive unchanged and the

other ninety places go to noisy copies of them. After a thousand generations the best matrix gets 77 percent of the *test* images right. Simulated evolution, with no knowledge of what a digit is, has found a classifier nearly as good as our distance-based templates (82 percent).

Is evolution simply the smarter search? Not by this evidence. Count the guesses each method scored. Hill climbing scored 20,000 matrices. Evolution scored a hundred per generation for 1,001 generations: 100,100 matrices, five times as many. At generation 200, when it had scored about as many as the hill climber, its best was 55 percent, behind the hill climber's 68. What the population really bought us is the graphics card's parallelism: scoring a hundred matrices in one operation takes far less than a hundred times as long as scoring one, so we could afford many more guesses. (On the author's laptop the thousand generations took about as long as the 20,000 hill-climbing steps, roughly 25 seconds each.) The two searches also took different step sizes, 0.1 for the hill climber and 0.05 for the mutations, so even this comparison is not perfectly even. A fair comparison of search methods always counts the budget, changes one thing at a time, and does not trust just the final score.

Finally, let's look at what was found. Each column of the weight matrix holds 784 numbers, one per pixel, so we can reshape each column back into a 28×28 image (Figure 10.1).

gs_weights

```
columns = W_evolved.T.reshape(10, 28, 28).cpu().numpy()
montage_plot(columns, grid_shape=(2, 5), symmetric=True)
```

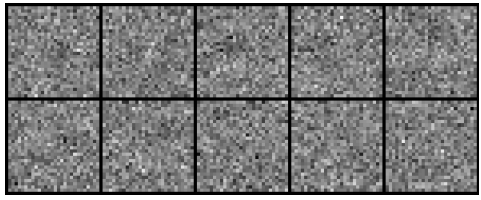


Figure 10.1: The ten columns of the evolved weight matrix, drawn as images. Bright pixels vote for a digit and dark pixels vote against it.

To the eye they are pure noise: no digit shapes at all. Yet this matrix gets 77 percent of the test images right, so the useful structure is there, spread thinly across thousands of weights and buried under the

randomness the search started from. Random search keeps whatever happens to work and never cleans up anything that does no harm.

10.7 What guessing teaches

It is easy to dismiss what we just did as a toy, but every idea in it is central to deep learning:

- A classifier is a *function with adjustable numbers*: here, one matrix multiplication with 7,840 weights.
- A *score* tells us how good a particular setting of the weights is.
- *Learning* is a search for weights with a better score, measured on training data and checked on test data.

What was wasteful was the search itself. Each random step was blind: it tried a direction, looked at the score, and, once the easy gains were gone, threw almost every step away. Imagine instead that, at every point on the hillside, you could feel which way was uphill. You would never waste a step. That feeling for the slope has a name, the *gradient*, and computing it is what PyTorch does best. It is the subject of the next chapter.

Try it

1. In `gs_climb`, change `step = 0.1` to `0.01` and then to `1.0`. Which climbs fastest at first, and which ends highest after 20,000 steps?
2. Give hill climbing the same budget as evolution: run `gs_climb` for 100,000 steps instead of 20,000 (it takes about five times as long). Does it catch up with the population's 77 percent?
3. In `gs_population`, try `keep = 1` (only the single best survives) and `keep = 50`, or change the mutation size `0.05` to `0.2`. Watch how the printed fitness changes.
4. Start the hill climber from the class-average templates of Chapter 8 instead of the best random matrix: in `gs_climb`, replace `W_best.clone()` with `torch.stack([X[Y == k].mean(dim=0) for k in range(10)]).T`. Can random steps improve on the templates' 63 percent?

Chapter 11

Following the Gradient

In the last chapter we found good weights by trial and error. Every guess was blind: we changed the weights at random and then checked whether things had improved. This chapter replaces blind guessing with something far better. For any setting of the weights, we will compute exactly which direction to move each of the 7,840 numbers to make the classifier a little less wrong. That direction is called the *gradient*. Repeatedly stepping along it, *gradient descent*, is the engine inside essentially every neural network trained today.

We will need MNIST again later in the chapter, so let's load it now. This is the same data as in Chapters 8 and 10, flattened to 784 numbers per image and scaled to 0–1; this time we let PyTorch do the reshaping and wrap it in a small function, since we load two halves. The seed makes the random numbers repeatable. The `device` is picked exactly as in Chapter 10; every later chapter does it the same way, so we print those lines again only in the three chapters at the end of the book that start from a blank page, where each loader is shown complete.

gr_load

```
import torch
from torchvision.datasets import MNIST

torch.manual_seed(0)

def load(train):
    ds = MNIST('data', train=train, download=True)
    X = ds.data.reshape(-1, 784).float() / 255.0
    return X.to(device), ds.targets.to(device)

X, Y = load(train=True)
X_test, Y_test = load(train=False)
print(X.shape, X_test.shape, device)
```

OUTPUT

```
torch.Size([60000, 784]) torch.Size([10000, 784]) mps
```

11.1 Slopes for free

How steep is the curve $y = x^2$ at $x = 3$? Nudge x up by a small amount h, see how much y changes, and divide by h to get the change per unit of nudge:

gr_nudge

```
def f(x):
    return x**2

for h in [0.1, 0.01, 0.001]:              # smaller and smaller nudges
    print(h, round((f(3 + h) - f(3)) / h, 6))
```

OUTPUT

```
0.1 6.1
0.01 6.01
0.001 6.001
```

The smaller the nudge, the closer the ratio comes to 6. That number is the *slope*, or *derivative*, of x^2 at 3: nudge x up a little and y goes up about six times as much. Its sign says which way is uphill. Try other points and a pattern appears: the slope of x^2 at any x is $2x$. If you met derivatives at school, that is the rule you learned; if not, the nudge is all the definition you will need.

Working out derivatives by hand gets tedious quickly, and a neural network has thousands or millions of weights. PyTorch does it for us. Mark a tensor with `requires_grad=True` and PyTorch records every calculation done with it. Calling `backward()` on the result then runs the calculation in reverse and fills in the derivative. This is *automatic differentiation*, or *autograd*.

gr_autograd

```
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
y.backward()          # work out dy/dx
print(x.grad)         # 2 * x = 6
```

OUTPUT

```
tensor(6.)
```

11.2 The chain rule, backwards

How does PyTorch know the derivative? It does not know any formula for the whole calculation. Instead it breaks the calculation into tiny steps, each of which it does know how to differentiate, and combines them with the *chain rule*: if a affects b and b affects c, then the effect of a on c is the effect of a on b times the effect of b on c.

Here is a one-weight "model" small enough to do by hand. It predicts $p = wx$ for the input $x = 2$; we wanted the answer $y = 10$, and the loss is the squared error $(wx - y)^2$:

gr_chain

```
w = torch.tensor(3.0, requires_grad=True)
x, y = 2.0, 10.0      # one input and the answer we want for it

p = w * x             # prediction: 6
e = p - y             # error: -4
loss = e ** 2         # 16
loss.backward()

by_hand = (2 * e) * 1 * x     # dloss/de * de/dp * dp/dw
print(loss.item(), by_hand.item(), w.grad.item())
```

OUTPUT

```
16.0 -16.0 -16.0
```

With $w = 3$ the prediction is 6, the error e is -4 and the loss is 16. Now walk backwards. The loss is e^2, so it changes $2e = -8$ times as fast as e. The error $e = p - y$ changes exactly as fast as p (a factor of 1). And $p = wx$ changes $x = 2$ times as fast as w. Multiply the three factors: $-8 \times 1 \times 2 = -16$. That is what `backward()` found too. The minus sign says that increasing w lowers the loss, which is right: w should grow toward 5.

That is all `backward()` does, however big the calculation. As the forward calculation runs, PyTorch records each step and what it was applied to. `backward()` then walks that record in reverse, from the loss back to the weights, multiplying the local slopes together along the way. Running the chain rule backwards like this is called *backpropagation*, and it costs only a small multiple of the forward calculation, typically two or three times as much, even with millions of weights.

11.3 A backward() of your own

This section is optional: it uses a little more Python than the rest of the book, and nothing later depends on it. There is nothing hidden in `backward()`, and you can build it yourself in a dozen lines. The Python you need: a `class` bundles some data with the functions that work on it; `__init__` runs whenever a new one is made; and Python calls a class's `__mul__` when it sees `a * b` and its `__add__` when it sees `a + b`. Finally, `backward` calls itself on each of the numbers it was made from, which is how it walks all the way back. A `Value` below is a number that remembers which numbers it was made from, and the local slope with respect to each. For a sum both slopes are 1; for a product $a \times b$ the slope with respect to a is b, and with respect to b is a. `backward` adds the incoming slope to the number's own `grad` and passes it on to its parents, multiplied by the local slope: the chain rule, one step at a time.

gr value

```
class Value:
    """A number that remembers how it was made, for backward()."""
    def __init__(self, data, parents=(), slopes=()):
        self.data, self.grad = data, 0.0
        self.parents, self.slopes = parents, slopes  # local derivatives

    def __add__(self, other):
        return Value(self.data + other.data, (self, other), (1.0, 1.0))

    def __mul__(self, other):
        return Value(self.data * other.data, (self, other),
                     (other.data, self.data))

    def backward(self, grad=1.0):
        self.grad += grad
        for parent, slope in zip(self.parents, self.slopes):
            parent.backward(grad * slope)        # the chain rule
```

Now the one-weight model again, built from `Value`s (subtracting 10 is written as adding −10, to keep the class small):

gr_value_use

```
w, x, minus_y = Value(3.0), Value(2.0), Value(-10.0)
e = w * x + minus_y          # error: 6 - 10 = -4
loss = e * e                 # 16
loss.backward()
print(loss.data, w.grad)
```

OUTPUT

```
16.0 -16.0
```

The same −16. Because `e` is used twice in `e * e`, its `backward` runs twice and the two contributions add up, which is how the chain rule handles a number that feeds into a calculation along more than one path. PyTorch's autograd is this idea, written to work on whole arrays at once, to visit each step only once, and to know the slopes of hundreds of operations instead of two.

11.4 Rolling downhill

Here is gradient descent in its simplest form. We want the value of x that makes $(x-2)^2$ as small as possible. We already know the answer is 2, but let's pretend we don't. Starting from $x = -3$, we repeatedly compute the slope and take a small step *against* it, which is downhill. The loop prints where x is and how big the loss is there, before taking the step:

gr_descent

```
x = torch.tensor(-3.0, requires_grad=True)
lr = 0.1                          # learning rate: how big a step

for step in range(31):
    loss = (x - 2) ** 2           # smallest when x = 2
    loss.backward()               # slope is now in x.grad
    if step % 5 == 0:
        print(f'step {step:2}   x = {x.item():7.4f}'
              f'   loss = {loss.item():7.4f}')
    with torch.no_grad():
        x -= lr * x.grad          # step downhill
    x.grad.zero_()                # forget this gradient
```

OUTPUT

```
step  0   x = -3.0000   loss = 25.0000
step  5   x =  0.3616   loss =  2.6844
step 10   x =  1.4631   loss =  0.2882
step 15   x =  1.8241   loss =  0.0309
step 20   x =  1.9424   loss =  0.0033
step 25   x =  1.9811   loss =  0.0004
step 30   x =  1.9938   loss =  0.0000
```

Each step moves x by the slope times a small number called the *learning rate*. Far from the bottom the slope is steep and the steps are large; near the bottom the slope flattens and the steps shrink, so x settles smoothly at 2.

The learning rate needs some care. To see why, here is the same loop packed into a function that records where x has been, run with two learning rates:

gr_descend_fn

```
def descend(lr, steps, start=-3.0):
    x = torch.tensor(start, requires_grad=True)
    path = []
    for step in range(steps):
        path.append(x.item())
        loss = (x - 2) ** 2
        loss.backward()
        with torch.no_grad():
            x -= lr * x.grad
        x.grad.zero_()
    return path

print([round(v, 2) for v in descend(lr=0.1, steps=8)])
print([round(v, 2) for v in descend(lr=1.1, steps=8)])
```

OUTPUT

```
[-3.0, -2.0, -1.2, -0.56, -0.05, 0.36, 0.69, 0.95]
[-3.0, 8.0, -5.2, 10.64, -8.37, 14.44, -12.93, 19.92]
```

gr_descent_plot

```
fig, axes = plt.subplots(1, 2, figsize=(4.6, 2.0))
for ax, lr in zip(axes, [0.1, 1.1]):
    path = np.array(descend(lr, steps=8))
    z = np.linspace(-16, 20, 200) if lr > 1 else np.linspace(-4, 8, 200)
    ax.plot(z, (z - 2) ** 2, color='gray', linewidth=0.8)
    ax.plot(path, (path - 2) ** 2, 'o-', markersize=3)
    ax.set_title(f'lr = {lr}')
    ax.set_xlabel('x')
axes[0].set_ylabel('loss')
plt.tight_layout()
plt.show()
```

With `lr=0.1` each step covers a fifth of the remaining distance to 2. With `lr=1.1` the step is so big that x jumps clean over the bottom and lands higher up the other side, and each jump is bigger than the last: the loss grows instead of shrinking (Figure 11.1). Too small a learning rate wastes time; too large a one makes training blow up. Picking it is a matter of trying a few values.

Three details of the code will appear in every training loop in this book:

- the update happens inside `torch.no_grad()`, because the step itself is not part of the calculation we want derivatives of;
- the gradient is set back to zero after each step, since PyTorch

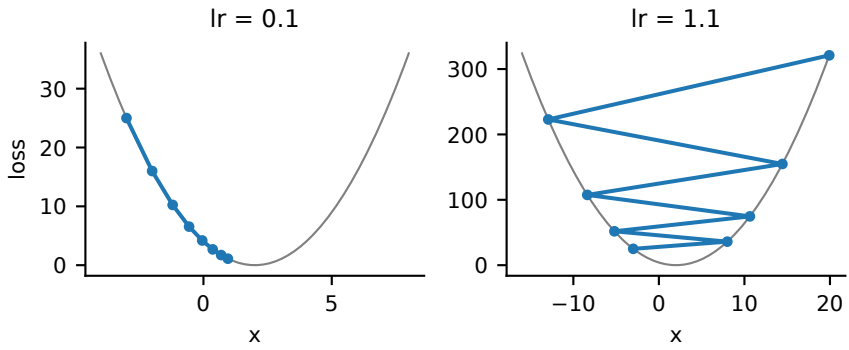


Figure 11.1: Eight steps of gradient descent on $(x-2)^2$, starting at $x=-3$. Left: a small learning rate rolls steadily downhill. Right: a learning rate of 1.1 overshoots the bottom every time, by more and more.

would otherwise *add* each new gradient to the old one;

- the quantity being minimized is called the *loss*.

To train a classifier we need the same three ingredients: weights with `requires_grad`, a loss that is small when the classifier is right, and the downhill step. The only new piece is the loss.

11.5 From scores to probabilities

Accuracy, the fraction of right answers, is what we care about, but it is a terrible thing to descend. Change one weight by a tiny amount and almost no answer flips, so the accuracy does not change at all and its slope is zero almost everywhere. We need a smooth measure of *how* right or wrong the classifier is.

Two functions from mathematics do the work, and it is worth meeting them first. The *exponential* e^x, `exp(x)`, raises the number $e = 2.718\ldots$ to the power x. It is always positive, it is 1 at $x = 0$, and it grows very fast: each step of 1 in x multiplies it by about 2.7. The *logarithm* `log` undoes it: $\log(e^x) = x$. For numbers between 0 and 1 the logarithm is negative, so $-\log p$ is positive, and it is 0 for $p = 1$ and grows without limit as p approaches 0:

gr_explog

```
x = torch.tensor([-2.0, 0.0, 1.0, 2.0, 5.0])
print(torch.exp(x))                    # always positive, grows fast
p = torch.tensor([1.0, 0.5, 0.1, 0.01, 0.0001])
print(-torch.log(p))                   # 0 when sure, large when not
```

OUTPUT

```
tensor([   0.135,    1.000,    2.718,    7.389,  148.413])
tensor([-0.000, 0.693, 2.303, 4.605, 9.210])
```

(The `-0.000` is simply zero: $-\log 1 = 0$, printed with the minus sign of the calculation that produced it.)

The first step is to turn the ten raw scores for each image into ten probabilities. The *softmax* function does this: exponentiate each score, so that every value is positive and bigger scores pull further ahead, then divide by the total so they add up to one.

gr_softmax
```
def softmax(scores):
    e = torch.exp(scores - scores.max(dim=1, keepdim=True).values)
    return e / e.sum(dim=1, keepdim=True)

scores = torch.tensor([[2.0, 1.0, 0.1],
                       [0.0, 0.0, 5.0]])
p = softmax(scores)
print(p)
print(p.sum(dim=1))
```

OUTPUT
```
tensor([[0.659, 0.242, 0.099],
        [0.007, 0.007, 0.987]])
tensor([1.000, 1.000])
```

In the second row the third score is far ahead of the others, so softmax gives it almost all of the probability. Subtracting each row's maximum score before exponentiating changes nothing mathematically, but it keeps `exp` from overflowing when scores are large.

11.6 Cross-entropy

Now the loss. For each image, look up the probability the classifier gave to the *correct* answer and take minus its logarithm. If the classifier was sure and right, the probability is near 1 and the loss is near 0. If it gave the right answer a probability near 0, the loss is huge. Averaged over a batch of images, this is the *cross-entropy* loss.

gr_xent
```
def cross_entropy(scores, labels):
    # the logarithm of the softmax, computed safely
    log_p = scores - scores.logsumexp(dim=1, keepdim=True)
    rows = torch.arange(len(labels), device=labels.device)
    return -log_p[rows, labels].mean()   # one entry from each row

print(cross_entropy(scores, torch.tensor([0, 2])))   # both right
print(cross_entropy(scores, torch.tensor([2, 0])))   # both wrong
```

OUTPUT
```
tensor(0.215)
tensor(3.665)
```

The expression `log_p[rows, labels]` is the paired indexing of Chapter 6: `rows` is $0, 1, 2, \ldots$ and `labels` gives a column for each row, so it picks out the correct class's entry from every row at once.

Why minus the logarithm, rather than, say, one minus the probability? Because it punishes confident mistakes without limit. Giving the right answer a probability of 0.01 costs 4.6; giving it 0.0001 costs 9.2. A classifier cannot get away with being sure and wrong. (There is a deeper reason, from information theory, that gives cross-entropy its name, but we will not need it.)

The next paragraph is a numerical-safety detail, safe to skip. We compute the logarithm of the softmax directly, as each score minus the log of the sum of the exponentials (`logsumexp`). This is mathematically the same as `softmax(scores).log()` but never takes the logarithm of a number that has rounded down to zero. PyTorch has its own version, and ours agrees with it:

gr_xent_check

```
import torch.nn.functional as F

labels = torch.tensor([0, 2])
print(cross_entropy(scores, labels))
print(F.cross_entropy(scores, labels))
```

OUTPUT

```
tensor(0.215)
tensor(0.215)
```

Note what both functions take as input: the *raw scores*, not probabilities. Cross-entropy applies the softmax itself. Passing scores that have already been through a softmax would apply it twice. Training still more or less works if you do that by mistake, which is exactly why the bug is so common and so easy to miss.

11.7 The slope of cross-entropy, by hand

Autograd will give us the gradient of the cross-entropy loss, but it is worth knowing what it is, because the answer is beautifully simple. For one image with pixels x, scores $z = xW$ and softmax probabilities p, the slope of the loss with respect to each score, that is, how fast the loss changes if only score k is nudged, is

$$\frac{\partial \,\text{loss}}{\partial z_k} = p_k - y_k.$$

The curly ∂ is the usual way to write "the slope with respect to one number while the others stay put." Here y is not the label itself but its

one-hot form, ten numbers with a 1 for the correct class and 0 for the others (`F.one_hot` makes it). In words: push down each wrong class's score by the probability it was given, and push up the correct class's score by the probability it was *not* given. (Getting there takes a few lines of the chain rule through the logarithm and the exponentials; the minus-log and the exponential undo each other almost perfectly, which is one reason this pair of functions is used together.) Since $z = xW$, the slope for the weight joining pixel i to class k is $x_i\,(p_k - y_k)$, averaged over the batch. We check the formula two ways: against autograd, and against the oldest test of a slope there is, nudging one weight and watching the loss move.

gr_by_hand

```
x5, y5 = X[:5], Y[:5]                       # five images
W = 0.01 * torch.arange(7840.0, device=device).reshape(784, 10).sin()
# (any small, varied weights will do; these are simply repeatable)
W.requires_grad_()

loss = cross_entropy(x5 @ W, y5)
loss.backward()                             # autograd's answer

p = (x5 @ W).softmax(dim=1)                 # probabilities
onehot = F.one_hot(y5, 10).float()
by_hand = x5.T @ (p - onehot) / 5           # the formula: x (p - y)
print((W.grad - by_hand).abs().max().item())

with torch.no_grad():                       # nudge one weight a little
    W2 = W.clone()
    W2[300, 3] += 1e-3
    change = cross_entropy(x5 @ W2, y5) - cross_entropy(x5 @ W, y5)
print(change.item() / 1e-3, by_hand[300, 3].item())
```

OUTPUT

```
2.9802322387695312e-08
0.04124641418457031 0.041017886251211166
```

The formula is the line `x5.T @ (p - onehot) / 5`. Row i of `x5.T` is pixel i in each of the five images, so the matrix product multiplies x_i by $p_k - y_k$ for every image and adds up over the five, giving one slope for every pixel and class: a 784×10 array, the shape of W. The formula agrees with autograd to the last digit 32-bit numbers can hold. Nudging weight $(300, 3)$ by a thousandth changes the loss by 0.0412 per unit, against the formula's 0.0410: as close as a finite nudge, and 32-bit arithmetic, allow. Notice also what the formula says about learning. A

pixel that is 0 in an image ($x_i = 0$) gets no update at all from that image, and an image the network already classifies confidently and correctly ($p \approx y$) hardly moves the weights. Learning is driven by mistakes, in proportion to their size.

11.8 Training a classifier

We train on small random batches of images rather than all 60,000 at once. The gradient from 128 images is a noisy but perfectly usable estimate of the gradient from all of them, at a tiny fraction of the cost. This is *stochastic gradient descent* (SGD).

Before training, we set aside the last 10,000 training images as a *validation set*. We never train on them; we use them to watch progress while we experiment. The test set stays locked away until the very end, so that the final test number is an honest estimate of how the classifier does on images that played no part in any of our decisions.

gr_split

```
X_val, Y_val = X[50_000:], Y[50_000:]   # held back to check progress
X, Y = X[:50_000], Y[:50_000]           # used for training
print(X.shape, X_val.shape)
```

OUTPUT

```
torch.Size([50000, 784]) torch.Size([10000, 784])
```

`get_batch` draws a random batch of training images, and we also need an accuracy function that works on raw scores.

gr_batch

```
def get_batch(X, Y, batch_size):
    i = torch.randint(len(X), (batch_size,), device=X.device)
    return X[i], Y[i]

def accuracy(scores, Y):
    return (scores.argmax(dim=1) == Y).float().mean().item()
```

The model is the same matrix multiplication as in Chapter 10, with one addition: a *bias* for each class, a number added to its score whatever the input. It is the b in $y = mx + b$. We keep the weights in a list `w` so that the training loop can update all of them the same way.

gr_model

```
def model(x, w):
    return x @ w[0] + w[1]

w = [(0.01 * torch.randn(784, 10, device=device)).requires_grad_(),
     torch.zeros(10, device=device, requires_grad=True)]
```

And here is the whole training loop. Every 500 steps it prints the loss on the current batch and the accuracy on the validation set, computed inside `torch.no_grad()` since checking needs no gradients:

gr_train

```
lr = 0.5
losses = []

for step in range(3001):
    x, y = get_batch(X, Y, 128)
    loss = cross_entropy(model(x, w), y)
    loss.backward()
    with torch.no_grad():
        for p in w:
            p -= lr * p.grad
            p.grad.zero_()
    losses.append(loss.item())
    if step % 500 == 0:
        with torch.no_grad():
            acc = accuracy(model(X_val, w), Y_val)
        print(step, round(loss.item(), 3), round(acc, 4))
```

OUTPUT

```
0 2.322 0.4427
500 0.289 0.9167
1000 0.344 0.9209
1500 0.227 0.9204
2000 0.293 0.9223
2500 0.327 0.924
3000 0.298 0.9265
```

After the first 500 steps the classifier is already right on 92 percent of the validation images, and it ends a little higher, at 92.7 percent: better than the hand-made templates of Chapter 8 (82 percent) or evolution in Chapter 10 (77 percent), in a fraction of the time. One complete pass through the training set is called an *epoch*. Three thousand batches of 128 images is 384,000 images, which is about eight epochs of our 50,000 training images. Only about, because `get_batch` draws images at random, so some images turn up more often than others and a few

may not turn up at all. On average it comes to the same thing.

The loss on each batch jumps around because every batch is different, but the trend is unmistakable (Figure 11.2).

gr_curve

```
plt.figure(figsize=(4.6, 2.4))
plt.plot(losses, linewidth=0.5)
plt.xlabel('step')
plt.ylabel('loss')
plt.show()
```

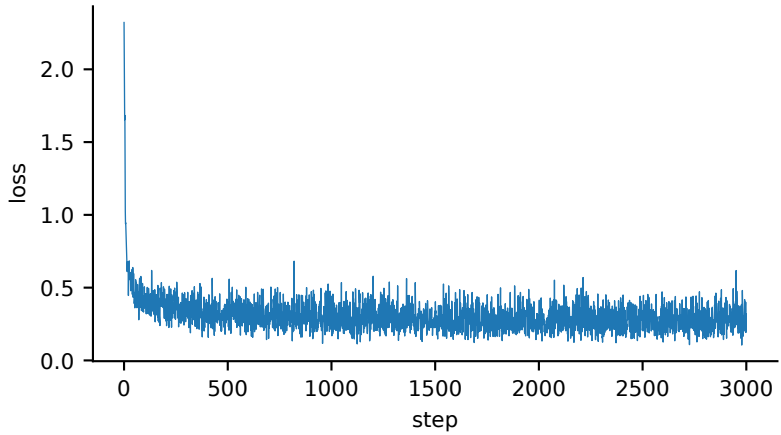


Figure 11.2: Training loss for each batch.

11.9 What did it learn?

As in the last chapter, each column of the weight matrix can be drawn as an image (Figure 11.3). The contrast with the noisy evolved weights of Figure 10.1 is striking.

gr_weights

```
columns = w[0].T.reshape(10, 28, 28).detach().cpu().numpy()
montage_plot(columns, grid_shape=(2, 5), symmetric=True)
```

These are templates again, but sharper than averages. The 0 is a bright ring with a dark center, because ink in the middle is evidence *against* a zero. The weights for each digit emphasize whatever distinguishes it from the others. Nobody designed them; they are what rolling downhill on the cross-entropy produced.

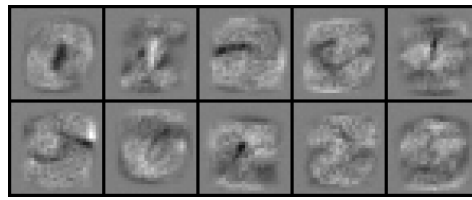


Figure 11.3: The learned weights for each digit. Bright pixels vote for the digit and dark pixels vote against it.

Looking at the mistakes is just as instructive:

gr_mistakes

```
with torch.no_grad():
    guesses = model(X_val, w).argmax(dim=1)
wrong = (guesses != Y_val).nonzero().flatten()[:16]
print((guesses != Y_val).sum().item(), 'mistakes out of', len(Y_val))
print('guessed:', guesses[wrong].tolist())
print('actual: ', Y_val[wrong].tolist())
images = X_val[wrong].reshape(-1, 28, 28).cpu().numpy()
montage_plot(images, size=4.6, grid_shape=(2, 8))
```

OUTPUT

```
735 mistakes out of 10000
guessed: [5, 6, 5, 5, 6, 8, 4, 8, 1, 1, 4, 4, 6, 3, 7, 5]
actual:  [3, 3, 3, 0, 2, 9, 9, 3, 9, 4, 1, 9, 4, 5, 9, 8]
```



Figure 11.4: The first 16 validation digits the linear classifier got wrong. The listing above prints its guesses and the true labels.

Some of these would trouble a person too. Others are clear to us but fool a classifier that can only add up pixels independently. A digit drawn a little off center, or with an unusual slant, puts its ink on pixels that the weights for its class do not expect. Overcoming that limitation is what the rest of this book is about.

11.10 Letting PyTorch take the step

The update step we wrote by hand is so common that PyTorch packages it as an *optimizer*. `torch.optim.SGD` does exactly what our loop did:

```
w = [(0.01 * torch.randn(784, 10, device=device)).requires_grad_(),
     torch.zeros(10, device=device, requires_grad=True)]
optimizer = torch.optim.SGD(w, lr=0.5)

for step in range(3001):
    x, y = get_batch(X, Y, 128)
    loss = cross_entropy(model(x, w), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
with torch.no_grad():
    print(round(accuracy(model(X_val, w), Y_val), 4))
```

OUTPUT

```
0.923
```

The pattern of `zero_grad()`, `backward()`, `step()` is the heartbeat of PyTorch training, and you will see it in every remaining chapter. It reaches 92.3 percent on the validation set, the same as our hand-written loop to within the noise of the random batches.

The advantage of the optimizer object is that a cleverer update rule is one line away. *Adam* keeps a running average of past gradients for each weight and adapts each weight's step size; it is the most widely used optimizer in deep learning.

A simpler improvement, which SGD offers as an option, is *momentum*: `torch.optim.SGD(w, lr=0.05, momentum=0.9)` makes each step a running blend of the current gradient and the previous steps, like a ball that keeps rolling in the direction it was already going. Noise from individual batches partly cancels out, and the steady downhill direction builds up speed. Adam includes the same idea.

gr_adam

```
w = [(0.01 * torch.randn(784, 10, device=device)).requires_grad_(),
     torch.zeros(10, device=device, requires_grad=True)]
optimizer = torch.optim.Adam(w, lr=0.01)

for step in range(3001):
    x, y = get_batch(X, Y, 128)
    loss = cross_entropy(model(x, w), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
with torch.no_grad():
    print(round(accuracy(model(X_val, w), Y_val), 4))
```

OUTPUT

```
0.9234
```

On a problem this simple, Adam and plain SGD end up in about the same place, 92.3 percent on the validation set. We will reach for Adam whenever the problems get harder.

Our experiments are finished, so now, and only now, we look at the test set. The model in `w` is the last one we trained, with Adam:

gr_test

```
with torch.no_grad():
    print('test', round(accuracy(model(X_test, w), Y_test), 4))
```

OUTPUT

```
test 0.9204
```

A linear classifier, a single matrix multiplication trained by gradient descent, gets 92 percent of handwritten digits it has never seen right. To do better we need to stop treating each pixel as an independent vote. We need *layers*.

Try it

1. In `gr_descend_fn`, try `lr=0.5`, `0.9` and `1.0`. One of them lands on 2 in a single step, and one bounces between the same two points forever. Can you see why from the slope $2(x - 2)$?

2. In `gr_train`, change `lr = 0.5` to `0.05` and then to `5.0`. How does the validation accuracy after 500 steps change, and what happens to the printed loss?

3. In `gr_optim`, use `torch.optim.SGD(w, lr=0.05, momentum=0.9)`. Does momentum with a tenth of the learning rate keep up with plain SGD?

4. Make the classic mistake on purpose. Change `gr_train` so that it gives `cross_entropy` the probabilities `softmax(model(x, w))` instead of the scores. Training still runs. How much worse is the validation accuracy after 3,000 steps?

5. Change the batch size in `gr_train` from 128 to 8 and to 1,024. Compare the loss curve's noise (`gr_curve`) and the final validation accuracy.

Chapter 12

Going Deeper

The classifier of the last chapter treats each pixel as an independent vote. Pixel 300 being bright adds a fixed amount to the score for 7, regardless of anything else in the image. That is why it cannot get much past 92 percent: whether a stroke means "seven" depends on the strokes around it, and a single weighted sum cannot express "this pixel matters only if that one is also on."

The fix is to compute features of features. A first layer of weighted sums looks at the raw pixels and detects simple patterns; a second layer looks at *those* detections and combines them into decisions. Stack enough of these layers and you have a *deep* network. This chapter builds one, and finds that there is a catch: stacking layers does nothing at all unless we add one small, crucial ingredient between them.

12.1 Two layers that are really one

Let's set things up as in Chapter 11: a seed, the `device` chosen as in Chapter 10, and a loader, which this time takes the dataset as an argument because we will use two of them. As before, the last 10,000 training images become the validation set and the test set waits until the end. Three helpers carry over unchanged and are not printed again: `montage_plot` from Chapter 8, and `get_batch` and `accuracy` from Chapter 11.

lv_load

```
import torch
import torch.nn.functional as F
from torchvision.datasets import MNIST, FashionMNIST

torch.manual_seed(0)

def load(dataset, train):
    ds = dataset('data', train=train, download=True)
    X = ds.data.reshape(-1, 784).float() / 255.0
    return X.to(device), ds.targets.to(device)

X, Y = load(MNIST, train=True)
X_test, Y_test = load(MNIST, train=False)
X_val, Y_val = X[50_000:], Y[50_000:]      # validation split
X, Y = X[:50_000], Y[:50_000]
print(len(X), len(X_val), len(X_test))
```

OUTPUT

```
50000 10000 10000
```

Now the catch. Multiply the input by one matrix and then by another, and the result is the same as multiplying by the single matrix `W1 @ W2`:

ly_collapse

```
x = torch.randn(5, 784)
W1 = 0.05 * torch.randn(784, 100)
W2 = 0.1 * torch.randn(100, 10)

two_layers = (x @ W1) @ W2
one_layer = x @ (W1 @ W2)
diff = (two_layers - one_layer).abs().max().item()
print(f'largest difference: {diff:.1g}')
print(torch.allclose(two_layers, one_layer, atol=1e-5))
```

OUTPUT

```
largest difference: 2e-06
True
```

The two answers differ by about two millionths, which is floating-point rounding: the computer rounds at every step, and the two orders of operation round differently. So we ask `allclose` whether they agree to within 10^{-5} (`atol`, the absolute tolerance), and they do. However many matrices we chain together, the result is just one matrix in disguise, a linear classifier again. Depth buys nothing.

12.2 The bend

What breaks the collapse is a *nonlinearity*, a function applied to every number between the layers that is not a straight line. The simplest that works well, and the most widely used, is the *rectified linear unit*, or ReLU: leave positive numbers alone, replace negative numbers with zero.

lv_relu

```
def relu(x):
    return x.clamp(min=0)     # negatives become 0

z = torch.linspace(-3, 3, 7)
print(z)
print(relu(z))
```

OUTPUT

```
tensor([-3., -2., -1.,  0.,  1.,  2.,  3.])
tensor([0., 0., 0., 0., 1., 2., 3.])
```

lv_relu_plot

```
z = torch.linspace(-3, 3, 100)
plt.figure(figsize=(2.4, 1.8))
plt.plot(z, relu(z))
plt.axhline(0, color='gray', linewidth=0.5)
plt.axvline(0, color='gray', linewidth=0.5)
plt.title('relu(x)')
plt.show()
```

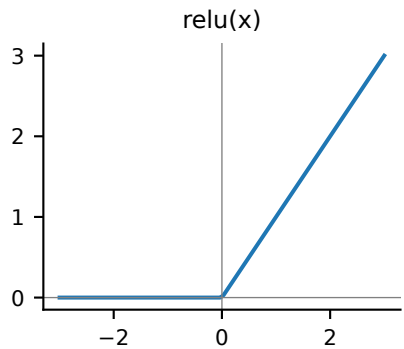


Figure 12.1: The ReLU function. It is straight on both sides, but the bend at zero is enough.

It looks almost too simple to matter. But think of each hidden number as a *detector*. It computes a weighted sum of the pixels, and the

ReLU makes it fire in proportion to how strongly its pattern is present, and stay silent otherwise. A silent detector contributes nothing to the next layer, so which detectors fire changes the computation the next layer performs. That input-dependent switching is exactly the "this matters only if that is present" logic that a single layer lacked. In fact a network with one hidden layer of ReLUs can approximate essentially any reasonable function, given enough hidden units. You can see why in one dimension. Each ReLU, `relu(x - k)`, is a hinge: flat until $x = k$, then a straight line. A weighted sum of hinges is a chain of straight segments, bending wherever one of them switches on, and with enough bends a chain of segments can follow any smooth curve:

ly_hinges

```
def hinge_fit(f, n):
    """Copy f on [0, 6] with n ReLU hinges: a bend at each knot."""
    knots = torch.linspace(0, 6, n + 1)
    slopes = (f(knots[1:]) - f(knots[:-1])) / (knots[1:] - knots[:-1])
    bends = torch.cat([slopes[:1], slopes[1:] - slopes[:-1]])
    return lambda x: f(knots[0]) + sum(
        b * relu(x - k) for b, k in zip(bends, knots[:-1]))

x = torch.linspace(0, 6, 300)
plt.figure(figsize=(4.6, 1.6))
for i, n in enumerate([3, 12]):
    plt.subplot(1, 2, i + 1)
    plt.plot(x, torch.sin(x), color='gray', lw=3, alpha=0.4)
    plt.plot(x, hinge_fit(torch.sin, n)(x))
    plt.title(f'{n} ReLUs')
plt.show()
```

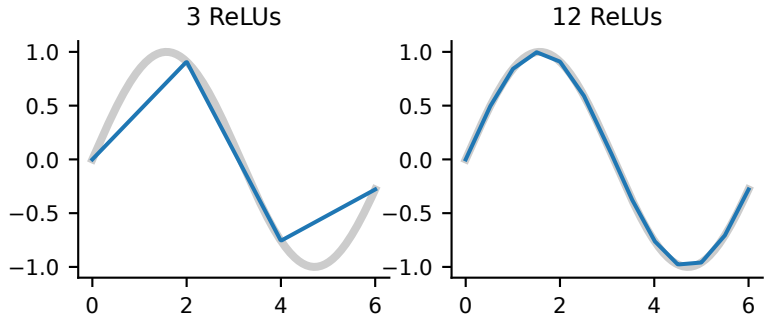


Figure 12.2: A sine curve (thick gray) copied by a sum of 3 and of 12 ReLU hinges. Each hinge adds one bend.

Here we chose the hinges by hand, one bend at each of n evenly spaced points, with the change of slope there as its weight. A hidden layer learns where to put its bends and how sharp to make them, in 784 dimensions instead of one.

12.3 Starting weights

Each layer needs a weight matrix and a bias vector, and before training they need starting values. How we choose the random starting weights matters more than you might expect. The experiment below pushes a batch of random inputs through ten layers of 256 units, each a random matrix followed by a ReLU, and prints how spread out the numbers are (their standard deviation) after every second layer. It tries three sizes of random weights: plain `randn`, the same scaled down to 0.01, and `randn` scaled by $\sqrt{2/n_{\text{in}}}$, where $n_{\text{in}} = 256$ is the number of inputs to each layer.

lv_init

```
x = torch.randn(1000, 256)          # a batch of 1000 random inputs

for name, scale in [('randn', 1.0), ('small', 0.01),
                    ('He', (2 / 256) ** 0.5)]:
    h = x
    spread = []
    for layer_number in range(1, 11):
        h = relu(h @ (scale * torch.randn(256, 256)))
        if layer_number % 2 == 0:       # print every other layer
            spread.append(f'{h.std().item():8.2g}')
    print(f'{name:6}', ' '.join(spread))
```

OUTPUT

```
randn   1.1e+02  1.4e+04  1.8e+06  2.3e+08  2.9e+10
small     0.011  0.00013  1.8e-06  2.3e-08  3.3e-10
He         0.85     0.93        1      1.1        1
```

With plain `randn` the numbers grow about tenfold at every layer; after ten layers they are in the tens of billions, and a deeper network would overflow. With weights of 0.01 they shrink about tenfold per layer instead, until the signal is less than a billionth of its starting size and the last layers see essentially nothing. The scale $\sqrt{2/n_{\text{in}}}$ keeps the numbers near 1 however deep we go. This is the square-root law of Chapter 9 at work: a neuron adds up n_{in} random products, so its

output spreads out as $\sqrt{n_{\text{in}}}$ times the size of one product, and dividing the weights by $\sqrt{n_{\text{in}}}$ cancels that growth. The 2 makes up for the ReLU, which zeroes about half of its inputs. This is *He initialization*, after Kaiming He, who proposed it, and our `layer` function uses it:

lv_layer

```
def layer(n_in, n_out):
    W = torch.randn(n_in, n_out, device=device) * (2 / n_in) ** 0.5
    b = torch.zeros(n_out, device=device)
    return [W.requires_grad_(), b.requires_grad_()]

w = layer(784, 500) + layer(500, 10)
print([tuple(p.shape) for p in w])
```

OUTPUT

```
[(784, 500), (500,), (500, 10), (10,)]
```

12.4 A network with a hidden layer

The network has 784 inputs, a *hidden layer* of 500 detectors, and 10 outputs:

lv_model

```
def model(x, w):
    x = relu(x @ w[0] + w[1])     # hidden layer
    return x @ w[2] + w[3]        # output layer: raw scores, no relu
```

Note that there is no ReLU after the output layer. The ten outputs are scores for the cross-entropy loss, and a score must be allowed to be negative; clipping them at zero would throw away information about how strongly the network rejects each class.

From now on we will use PyTorch's built-in `F.cross_entropy` (we checked in Chapter 11 that it matches ours) and wrap the training loop in a function, since we will be calling it several times. The function takes the data to check progress on as arguments, so we can hand it the validation set of whichever dataset we are using. `evaluate` does the checking inside `torch.no_grad()`.

We also switch from SGD to Adam. With several layers, the gradients of different weight matrices can differ a lot in size, and a single learning rate that suits one layer may be too large or too small for another. Adam adapts each weight's step size by itself, so it is much less fussy about the learning rate, and `lr=0.001` is the usual starting point.

ly_train_fn
```
def evaluate(model, w, X, Y):
    with torch.no_grad():         # no gradients needed to check
        return round(accuracy(model(X, w), Y), 4)

def train(model, w, X, Y, X_val, Y_val,
          steps=5000, lr=0.001, batch_size=128):
    optimizer = torch.optim.Adam(w, lr=lr)
    for step in range(steps + 1):
        x, y = get_batch(X, Y, batch_size)
        loss = F.cross_entropy(model(x, w), y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        if step % 1000 == 0:
            acc = evaluate(model, w, X_val, Y_val)
            print(step, round(loss.item(), 3), acc)
```

ly_train
```
train(model, w, X, Y, X_val, Y_val)
```

OUTPUT
```
0 2.371 0.2977
1000 0.081 0.9714
2000 0.031 0.9765
3000 0.061 0.9784
4000 0.007 0.9806
5000 0.008 0.9807
```

From 92 percent to 98 on the validation set, with one hidden layer. That is a big jump: the error rate has fallen from about 7 percent to about 2, by roughly three quarters. The accuracy wobbles a little from one checkpoint to the next because each step uses a different random batch; the trend matters, not the last digit.

12.5 What the hidden layer sees

Each hidden unit has 784 incoming weights, one per pixel, so as before we can draw them as images (Figure 12.3).

ly_hidden
```
first = w[0].T[:36].reshape(36, 28, 28).detach().cpu().numpy()
montage_plot(first, size=4.6, grid_shape=(4, 9), symmetric=True)
```

These are not digit templates anymore. Much of each image is still the random noise it started as. Pixels near the border are blank

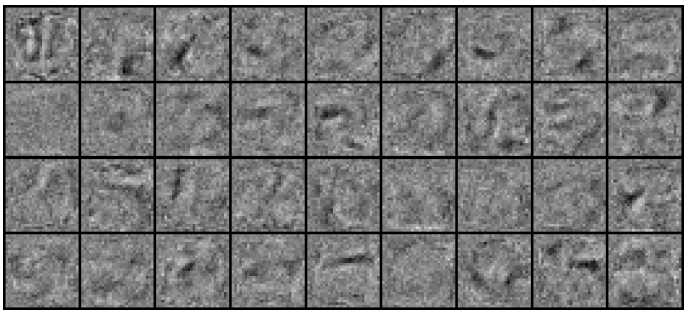


Figure 12.3: Incoming weights of the first 36 of the 500 hidden units.

in almost every digit, so their weights get no gradient and never change. But in the middle you can make out fragments: strokes and pieces of curves, often a dark stroke beside a light one. No single unit decides anything. Each detects some partial pattern, and the output layer learns which combinations of fragments mean which digit. This division of labor, simple detectors feeding more abstract ones, is the central idea of deep learning, and we will see it again, much more clearly, inside the networks of Part IV.

12.6 Any number of layers

Writing out every layer by hand gets tedious. Since `w` is a list of alternating weights and biases, a loop can apply any number of layers, using a ReLU after every layer except the last:

lv_model_deep

```
def model(x, w):
    n_layers = len(w) // 2              # w holds W, b, W, b, ...
    for i in range(n_layers):
        W, b = w[2 * i], w[2 * i + 1]
        x = x @ W + b
        if i < n_layers - 1:            # relu after all but the last
            x = relu(x)
    return x
```

And a helper builds the weights for any list of layer sizes. Here are two hidden layers of 256 units each:

lv_deeper

```
def network(sizes):
    w = []
    for n_in, n_out in zip(sizes[:-1], sizes[1:]):
        w += layer(n_in, n_out)
    return w

w = network([784, 256, 256, 10])
train(model, w, X, Y, X_val, Y_val)
```

OUTPUT

```
0 2.557 0.1697
1000 0.037 0.9718
2000 0.065 0.9761
3000 0.012 0.975
4000 0.002 0.9763
5000 0.001 0.9806
```

On MNIST this does about as well as the single wide hidden layer. MNIST is close to solved by now; a deeper network has little left to find.

12.7 Memorizing versus learning

Compare how the network does on the images it trained on with the validation images it has never trained on:

lv overfit

```
print('train     ', evaluate(model, w, X, Y))
print('validation', evaluate(model, w, X_val, Y_val))
```

OUTPUT

```
train      0.9977
validation 0.9806
```

On the training images it is almost perfect. On the validation images it is about 1.7 points worse. This gap is *overfitting*: some of what the network learned is specific to the particular training images, quirks that do not carry over to new ones. A network with hundreds of thousands of weights has room to memorize a lot. A small gap like this one is normal and harmless. When the gap grows large, the usual remedies are more data, a smaller network, stopping training earlier, or showing the network altered copies of its training images, which we will do in Chapter 16.

We are done experimenting on MNIST, so it is time to look at the test set, once, with the deeper network:

lv_test

```
print('test', evaluate(model, w, X_test, Y_test))
```

OUTPUT

```
test 0.9808
```

98.1 percent, the same as on the validation set. Our choices were not tuned to the validation images so closely that they failed to carry over.

12.8 A harder problem

MNIST is nearly solved, so let's try the same networks on the clothing images of Fashion-MNIST from Chapter 8. We load it in place of MNIST, with the same split. Because `network` and `model` accept any list of sizes, `[784, 10]`, with no hidden layer at all, gives us the linear classifier for comparison:

lv_fashion

```
X, Y = load(FashionMNIST, train=True)
X_test, Y_test = load(FashionMNIST, train=False)
X_val, Y_val = X[50_000:], Y[50_000:]
X, Y = X[:50_000], Y[:50_000]

w_linear = network([784, 10])           # no hidden layer at all
train(model, w_linear, X, Y, X_val, Y_val)
```

OUTPUT

```
0 2.722 0.0993
1000 0.396 0.8321
2000 0.471 0.8465
3000 0.54 0.8535
4000 0.557 0.8495
5000 0.458 0.8531
```

The linear classifier levels off at about 85 percent on the validation set. Now the network with two hidden layers:

lv_fashion_deep

```
w_deep = network([784, 256, 256, 10])
train(model, w_deep, X, Y, X_val, Y_val)
```

OUTPUT

```
0 2.523 0.1923
1000 0.373 0.8625
2000 0.307 0.8808
3000 0.267 0.8884
4000 0.176 0.8889
5000 0.207 0.8899
```

It reaches 89 percent. Clothing is harder than digits for both models, and the hidden layers help less here: they gain about four points over the linear classifier, where on MNIST they gained more than five and removed three quarters of the errors. Flattened pixels can only take us so far on clothing. The deep network also overfits more on clothing:

ly_fashion_gap
```
print('train     ', evaluate(model, w_deep, X, Y))
print('validation', evaluate(model, w_deep, X_val, Y_val))
```

OUTPUT
```
train      0.9307
validation 0.8899
```

The gap between training and validation accuracy is now about 4 points, against 1.7 on MNIST.

12.9 The same network, the PyTorch way

Our list of weights and our `model` loop work, and you now know exactly what is inside a network. PyTorch packages the same pieces as *modules*. `nn.Linear(784, 256)` is a layer: it creates and holds its own weight matrix and bias. `nn.ReLU()` is the bend. `nn.Sequential` chains modules together, feeding each one's output to the next, which is what our loop in `model` did:

ly_nn
```
import torch.nn as nn

net = nn.Sequential(
    nn.Linear(784, 256), nn.ReLU(),
    nn.Linear(256, 256), nn.ReLU(),
    nn.Linear(256, 10),
).to(device)
print(net)
print(sum(p.numel() for p in net.parameters()),   # weights in net
      sum(p.numel() for p in w_deep))             # and in w_deep
```

OUTPUT
```
Sequential(
  (0): Linear(in_features=784, out_features=256, bias=True)
  (1): ReLU()
  (2): Linear(in_features=256, out_features=256, bias=True)
  (3): ReLU()
  (4): Linear(in_features=256, out_features=10, bias=True)
)
269322 269322
```

It has exactly as many weights as `w_deep`. (`nn.Linear` stores its matrix the other way round, 256 by 784, and computes `x @ W.T + b`, but that is only bookkeeping.) `net.parameters()` hands the optimizer

every weight inside the network, and calling `net(x)` runs the whole stack. The training loop is the one inside our `train` function:

lv_nn_train

```
optimizer = torch.optim.Adam(net.parameters(), lr=0.001)

for step in range(5001):
    x, y = get_batch(X, Y, 128)
    loss = F.cross_entropy(net(x), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

net.eval()                    # switch to evaluation mode
with torch.no_grad():
    print('validation', round(accuracy(net(X_val), Y_val), 4))
    print('test      ', round(accuracy(net(X_test), Y_test), 4))
```

OUTPUT

```
validation 0.8909
test       0.8829
```

`net.eval()` switches the network into evaluation mode. It changes nothing for this network, but some kinds of layers you will meet later behave differently during training and checking, so it is a good habit to switch before evaluating. The modular network reaches 89 percent on the validation set, the same as `w_deep`, even though `nn.Linear` starts from PyTorch's own default initialization, which is about two and a half times smaller than He's (it was designed with a different rule in mind). With Fashion-MNIST experiments finished, we look at its test set once: 88.3 percent. Starting with AlexNet in Chapter 15, our networks will be written this way.

Deep as it is, this network still has a basic handicap. Like the linear model, it starts by flattening the image into a list, so it has no idea which pixels are next to which. Shift a sneaker three pixels to the right and, as far as the network can tell, it is a completely different list of numbers. Images have structure that these networks ignore. The next part of the book is about networks that use it.

Try it

1. In `ly_init`, change the 2 in the He scale to 1. How fast does the spread shrink now, and why does the ReLU make the 2 necessary?
2. Vary the width of the hidden layer in `ly_layer` and `ly_train`: try 20, 100 and 2,000 units instead of 500. How much width does MNIST actually need?
3. Remove the `relu` from `ly_model` and retrain. Since the two layers now collapse into one, the validation accuracy should fall back to about that of the linear classifier. Does it?
4. Train `w_deep` on Fashion-MNIST for 20,000 steps instead of 5,000 (`steps=20_000`) and run `ly_fashion_gap` again. Does the validation accuracy keep improving while the gap grows?
5. In `ly_nn_train`, replace `torch.optim.Adam` with `torch.optim.SGD`, keeping `net.parameters()` but with `lr=0.1`. How close does it get to Adam's validation accuracy in the same 5,000 steps?

Part IV

Seeing with Convolutions

Chapter 13

Convolution

The networks of the last chapter begin by flattening the image, and in doing so they throw away the most obvious fact about pictures: that nearby pixels belong together. A stroke, an edge, the rim of a coin are local patterns, and the same pattern can appear anywhere in the frame. In this chapter we meet the operation built to exploit that fact. *Convolution* slides one small pattern across an image and asks, at every position, "how much does this patch look like my pattern?" It is one of the oldest tools of image processing, and it is the operation that gives convolutional neural networks their name.

13.1 Finding coins

Let's start with a concrete task. Here is a photograph of coins, another sample image from scikit-image, drawn with the `plot` helper of Chapter 7:

cv_coins

```
from skimage import data

coins = data.coins() / 255.0
print(coins.shape)
plot(coins, size=3.2)
```

OUTPUT

```
(303, 384)
```

We cut one coin out to use as a pattern to search for. A small pattern used this way is called a *template*, a *filter* or a *kernel*; the three words mean the same thing here.

cv_coin

```
coin = coins[170:220, 75:130]
print(coin.shape)
plot(coin, size=1.2)
```

OUTPUT

```
(50, 55)
```



Figure 13.1: The coins image: an array of shape (303, 384).



Figure 13.2: The kernel: one coin, 50×55 pixels.

How do we measure whether some patch of the image looks like the coin? We already know how: the dot product. Multiply the patch by the kernel pixel by pixel and add everything up. First we subtract the average brightness from both, so that dark pixels become negative and count *against* a match. Otherwise any bright patch would score well.

cv_one_spot

```
image = coins - coins.mean()        # center both on zero, so dark
kernel = coin - coin.mean()         # pixels count against a match

def match_at(r, c):
    h, w = kernel.shape
    patch = image[r:r + h, c:c + w]
    return np.sum(patch * kernel)

print(round(match_at(170, 75), 1))    # where the coin came from
print(round(match_at(100, 200), 1))   # somewhere else
```

OUTPUT

```
188.2
6.8
```

The score where the coin actually is dwarfs the score at an arbitrary spot. Now do this at every position: lay the kernel over the top-left corner of the image, compute the score, slide one pixel to the right, compute again, and so on across every row. SciPy does exactly this with `correlate2d`:

cv_correlate

```
from scipy.signal import correlate2d

z = correlate2d(image, kernel, mode='valid')
print(z.shape)
row, col = np.unravel_index(z.argmax(), z.shape)
print(int(row), int(col))
plot(z, size=3.2)
```

OUTPUT

```
(254, 330)
170 75
```

The output is itself an image, a *map* of where the pattern appears, and its brightest point is exactly where we cut the coin out. But look at Figure 13.3: every coin produces a bright spot. The kernel has found the other coins too, although it has never seen them.

The raw score is still biased by brightness, since a brighter coin produces bigger products. Normalizing each patch before comparing removes that bias. scikit-image packages this as `match_template`, and `peak_local_max` finds the separate bright spots:

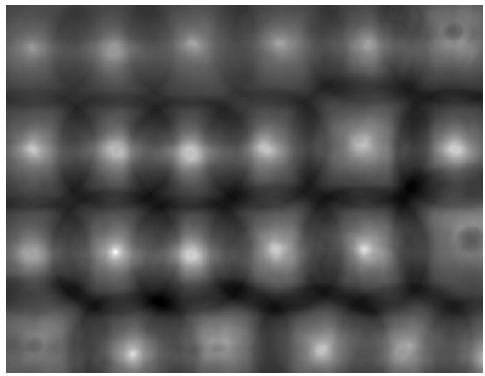


Figure 13.3: The match score at every position. Bright spots are where the image looks most like the kernel.

cv_ncc

```
from skimage.feature import match_template, peak_local_max

score = match_template(coins, coin)
peaks = peak_local_max(score, min_distance=20, threshold_abs=0.5)
print(len(peaks), 'matches')

plt.figure(figsize=(3.2, 2.6))
plt.imshow(coins, cmap='gray')
h, w = coin.shape                       # mark each coin's center
plt.scatter(peaks[:, 1] + w // 2, peaks[:, 0] + h // 2, s=40,
            facecolors='none', edgecolors='#f97306', linewidths=1.5)
plt.axis('off')
plt.show()
```

OUTPUT

```
13 matches
```

Thirteen matches. The coins that were missed are a different size or show a different design, so they really do look less like this one coin. A single kernel detects a single pattern. To find all the coins we would need several kernels, and that is exactly what a convolutional network will have.

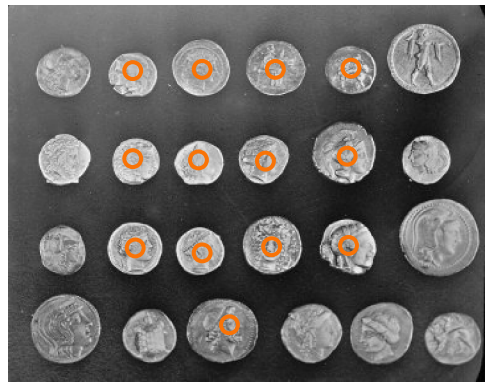


Figure 13.4: Circles mark the 13 strongest matches for the coin kernel.

13.2 Correlation and convolution

Strictly speaking, what we just did is called *cross-correlation*. *Convolution* is the same sliding sum with the kernel flipped upside down and left to right first; the flip comes from its origins in signal processing, where it makes the mathematics tidier. For a symmetric kernel the two are identical, and for a kernel that is going to be *learned* it makes no difference at all, since a network would simply learn the flipped kernel. Deep learning libraries, PyTorch included, actually compute cross-correlation and call it convolution. We will do the same, and use `correlate2d` throughout. (SciPy also has `convolve2d`, which does flip the kernel. For the edge detectors below that would turn every response upside down, bright where it should be dark.)

13.3 Kernels as tools

Small kernels do many of the classic operations of image processing. A 5×5 kernel of equal weights averages each pixel with its neighbors, which blurs the image. The *Sobel* kernel has positive weights on one side and negative on the other, so it responds where brightness changes, at an edge, and transposing it turns a horizontal-edge detector into a vertical one.

cv_kernels

```
camera = data.camera() / 255.0

blur = np.ones((5, 5)) / 25
sobel = np.array([[ 1,  2,  1],
                  [ 0,  0,  0],
                  [-1, -2, -1]])

def filt(image, k):            # mirror the border instead of zeros
    return correlate2d(image, k, mode='same', boundary='symm')

results = [(camera, 'original'),
           (filt(camera, blur), 'blur'),
           (filt(camera, sobel), 'horizontal edges'),
           (filt(camera, sobel.T), 'vertical edges')]

plt.figure(figsize=(3.6, 3.9))
for i, (im, name) in enumerate(results):
    plt.subplot(2, 2, i + 1)
    plt.imshow(im, cmap='gray')
    plt.title(name)
    plt.axis('off')
plt.show()
```

The small `filt` helper asks for `mode='same'`, which pads the border so that the output is the same size as the input, and `boundary='symm'`, which fills that padding with a mirror image of the edge pixels. Padding with zeros would put an artificial edge all the way around the frame, and the edge detectors would dutifully find it.

What do *random* 3×3 kernels do? Each one picks out some mixture of brightness and edges in different directions (Figure 13.6).

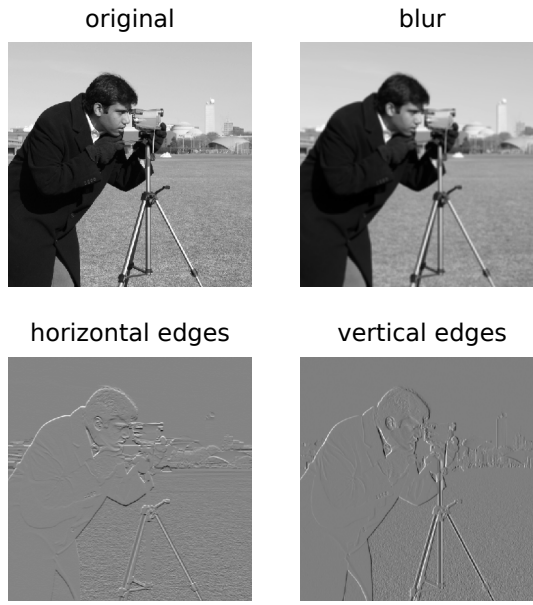


Figure 13.5: One image, three kernels. The edge detectors respond only where brightness changes, and only in one direction.

cv_random

```
rng = np.random.default_rng(0)

plt.figure(figsize=(3.6, 3.6))
for i in range(9):
    k = rng.uniform(-1, 1, size=(3, 3))
    plt.subplot(3, 3, i + 1)
    plt.imshow(filt(camera, k), cmap='gray')
    plt.axis('off')
plt.show()
```

Some of them are useful, most are not, and none was designed. Hold on to that thought: a convolutional network starts with random kernels like these and gradually adjusts them, by gradient descent, until they detect whatever the task needs.

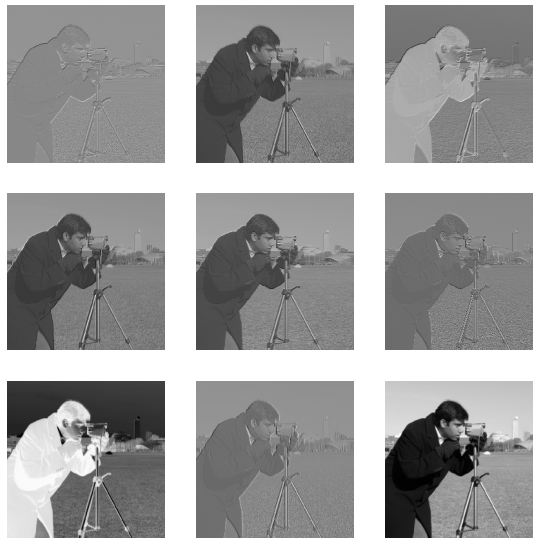


Figure 13.6: Nine random 3×3 kernels applied to the same image.

13.4 Convolution by hand

There is no magic in `correlate2d`. Two loops over the output positions and the dot product at each one are all it takes:

cv_loop

```
def conv(image, kernel):
    kh, kw = kernel.shape
    out = np.zeros((image.shape[0] - kh + 1, image.shape[1] - kw + 1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i, j] = np.sum(image[i:i + kh, j:j + kw] * kernel)
    return out

z_loop = conv(camera, sobel)
z_scipy = correlate2d(camera, sobel, mode='valid')
print(np.allclose(z_loop, z_scipy))
```

OUTPUT

```
True
```

Our loop gives the same answer as SciPy. The *valid* output, which never lets the kernel hang off the edge of the image, is two pixels smaller in each direction for a 3×3 kernel.

13.5 Convolution in PyTorch

PyTorch's `F.conv2d` computes many convolutions at once. It expects images as a four-dimensional tensor of shape (*batch*, *channels*, *height*, *width*), and kernels as (*output channels*, *input channels*, *height*, *width*). For a single grayscale image and a single kernel, most of those sizes are 1, which we add with `[None, None]`:

cv_torch

```
import torch
import torch.nn.functional as F

x = torch.tensor(camera, dtype=torch.float32)[None, None]  # (1, 1, H, W)
k = torch.tensor(sobel, dtype=torch.float32)[None, None]   # (1, 1, 3, 3)
z_torch = F.conv2d(x, k)
print(x.shape, k.shape, z_torch.shape)
print(np.allclose(z_torch[0, 0].numpy(), z_scipy, atol=1e-5))
```

OUTPUT

```
torch.Size([1, 1, 512, 512]) torch.Size([1, 1, 3, 3]) torch.Size([1, 1,
    ↪ 510, 510])
True
```

Same answer again. Those extra dimensions are where the power is. With 64 kernels stacked along the first axis, a single call to `F.conv2d` produces 64 feature maps, and on a graphics card it computes them all in parallel. (Sixty-four is the number of kernels in the first layer of the AlexNet we will open up in Chapter 15, though its kernels are 11×11 and in color.) Here is a rough comparison of speed:

cv_timing

```
import time

def sync():
    if device == 'cuda':
        torch.cuda.synchronize()
    elif device == 'mps':
        torch.mps.synchronize()

kernels = rng.uniform(-1, 1, size=(64, 3, 3))

t = time.time()
conv(camera, kernels[0])
print(f'python loop, 1 kernel:   {time.time() - t:.3f} s')

t = time.time()
for k in kernels:
    correlate2d(camera, k, mode='valid')
print(f'scipy, 64 kernels:       {time.time() - t:.3f} s')

xg = x.to(device)
kg = torch.tensor(kernels, dtype=torch.float32, device=device)[:, None]
F.conv2d(xg, kg)                                    # warm up
sync()
t = time.time()
for _ in range(10):
    F.conv2d(xg, kg)
sync()                                              # wait for the GPU
print(f'pytorch {device}, 64 kernels: {(time.time() - t) / 10:.4f} s')
```

OUTPUT

```
python loop, 1 kernel:   0.606 s
scipy, 64 kernels:       0.308 s
pytorch mps, 64 kernels: 0.0018 s
```

Notice the calls to `sync()`. A graphics card works *asynchronously*: `F.conv2d` returns as soon as the work is queued, not when it is finished. Without waiting for the card, the timer would measure only how long it takes to hand over the job, and the GPU would look impossibly fast. Even measured honestly, it computed 64 feature maps more than a hundred times faster than SciPy did, and much faster than our Python loop computed a single one. Your timings will differ with your hardware, but not by enough to change that picture.

13.6 From filters to networks

A convolution kernel is a neuron in disguise. Like the neurons of Chapter 12, it computes a weighted sum of its inputs. There are two differences. It looks only at a small neighborhood of pixels, rather than the whole image; and the *same* weights are used at every position, rather than separate weights for each location. Those two constraints are exactly what images call for. Local patterns are built from nearby pixels, and a coin, an edge or the loop of a 9 means the same thing wherever it appears.

Put a layer of learnable kernels where the matrix multiplication used to be, follow it with a ReLU, stack a few of these layers, and you have a *convolutional neural network*. That is the next chapter. First, a short detour that shows how much a single small kernel can do when it is applied over and over.

13.7 Coda: convolution as a rule of nature

This coda is for fun; the next chapter does not depend on it. So far a convolution has been something we apply to an image once. Apply it again and again, feeding each output back in as the next input, and it becomes something else: a law of motion. Many systems in nature are governed by local rules. What happens next at each point depends only on what is happening at that point and its neighbors, and "look at the neighbors and add them up" is exactly what a kernel computes.

The best-known example is John Conway's *Game of Life*, from 1970. Each cell of a grid is alive (1) or dead (0). At each step, count each cell's eight living neighbors. A dead cell with exactly three comes to life; a living cell with two or three survives; every other cell is dead at the next step. Counting neighbors is a correlation with a kernel of ones that has a zero in the middle, and the rule is two comparisons. `boundary='wrap'` joins the grid's left edge to its right and its top to its bottom, so that nothing falls off.

cv_life

```
neighbors = np.array([[1, 1, 1],
                      [1, 0, 1],
                      [1, 1, 1]])

def life_step(world):
    n = correlate2d(world, neighbors, mode='same', boundary='wrap')
    return ((n == 3) | ((world == 1) & (n == 2))).astype(int)

world = np.zeros((8, 8), dtype=int)
world[1, 2] = world[2, 3] = world[3, 1] = world[3, 2] = world[3, 3] = 1

plt.figure(figsize=(4.6, 1.0))
for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(world, cmap='gray_r', vmin=0, vmax=1)
    plt.title(f'step {i}')
    plt.xticks([])
    plt.yticks([])
    plt.gca().spines[:].set_visible(True)
    world = life_step(world)
plt.show()
```

Five living cells, given this rule, make a shape that walks across the grid forever (Figure 13.7). Nobody programmed walking; it emerges from the rule. Start from a random soup instead, and the soup boils

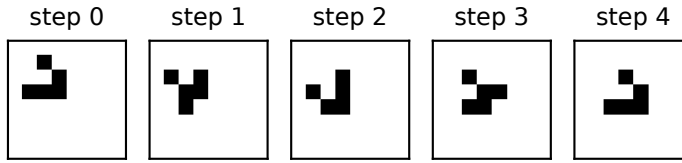


Figure 13.7: A *glider*. After four steps it has the same shape, one cell further down and to the right.

down to a scattering of stable blocks, blinking bars and the occasional glider (Figure 13.8).

cv_soup

```
world = (rng.random((100, 100)) < 0.3).astype(int)
plt.figure(figsize=(4.6, 1.6))
for i in range(301):
    if i in (0, 30, 300):
        plt.subplot(1, 3, [0, 30, 300].index(i) + 1)
        plt.imshow(world, cmap='gray_r')
        plt.title(f'step {i}\n{world.sum()} alive')
        plt.axis('off')
    world = life_step(world)
plt.show()
```

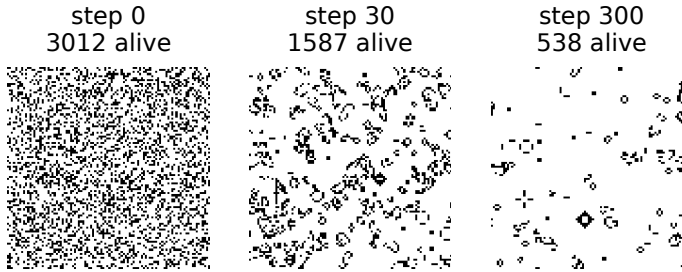


Figure 13.8: The Game of Life from a random start.

Physics works the same way, with real numbers instead of alive and dead. The kernel below, the *Laplacian*, compares each point with the average of its neighbors. Its weights add up to zero, so on a flat surface it gives zero everywhere: only differences between a point and its neighbors count. Applied to a single bump it is negative at the bump and positive around it:

cv_laplace

```
laplace = torch.tensor([[0.5,  1.0, 0.5],
                        [1.0, -6.0, 1.0],
                        [0.5,  1.0, 0.5]]) / 2   # neighbors minus center

def lap(u):                                       # u: (1, 1, H, W)
    u = F.pad(u, (1, 1, 1, 1), mode='circular')   # wrap around the edges
    return F.conv2d(u, laplace[None, None])

bump = torch.zeros(1, 1, 5, 5)
bump[0, 0, 2, 2] = 1.0
print(lap(bump)[0, 0])
```

OUTPUT

```
tensor([[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
        [ 0.0000,  0.2500,  0.5000,  0.2500,  0.0000],
        [ 0.0000,  0.5000, -3.0000,  0.5000,  0.0000],
        [ 0.0000,  0.2500,  0.5000,  0.2500,  0.0000],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000]])
```

Think of a drumhead, a stretched elastic sheet. Each point of it is pulled toward the level of the points around it: a high point is pulled down, a low point up, and the Laplacian measures that pull. Keep two arrays, the height of the sheet and how fast each point is moving. Each step, the pull changes the speed and the speed changes the height:

cv_waves

```
N = 200
rows, cols = torch.meshgrid(torch.arange(N), torch.arange(N),
                            indexing='ij')
u = torch.zeros(1, 1, N, N)               # height of the sheet
v = torch.zeros(1, 1, N, N)               # how fast it is moving
for r, c in [(60, 60), (130, 80), (90, 150)]:     # three taps
    u[0, 0] += torch.exp(-((rows - r)**2 + (cols - c)**2) / 8.0)

frames = []
for t in range(161):
    v += 0.25 * lap(u)                    # pulled toward the neighbors
    u += v
    if t in (20, 60, 160):
        frames.append(u[0, 0].clone())

plt.figure(figsize=(4.6, 1.6))
for i, (f, t) in enumerate(zip(frames, (20, 60, 160))):
    plt.subplot(1, 3, i + 1)
    plt.imshow(f, cmap='gray', vmin=-0.1, vmax=0.1)
    plt.title(f'step {t}')
    plt.axis('off')
plt.show()
```

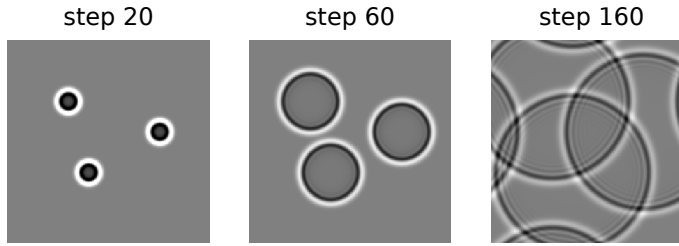


Figure 13.9: Three taps on a drumhead, simulated by repeated convolution. The rings spread, pass through one another, and wrap around the edges.

That loop is the *wave equation*, the simplest physics of vibrating sheets, sound and light, and the only spatial operation in it is a 3×3 convolution (Figure 13.9). (Real ripples on water are more complicated: their speed depends on their wavelength.) The faint wake inside each ring is not a bug: in a flat, two-dimensional world a sharp tap always leaves one. We used `F.conv2d` this time, with `F.pad` in `'circular'` mode to wrap the edges, so the same code would run on a graphics card once its tensors are moved there with `.to(device)`.

Chemistry, too. In the *Gray–Scott* model two chemicals spread through a dish at different speeds while one feeds on the other. Each step is two Laplacians and a little arithmetic. The numbers 0.16 and 0.08 are how fast each chemical spreads; `feed` is how fast fresh U flows in and `kill` how fast V is removed, and `U * V * V` is the reaction, in which two units of V turn one unit of U into more V:

cv_gray_scott

```
N = 128
U = torch.ones(1, 1, N, N)                # chemical U fills the dish
V = torch.zeros(1, 1, N, N)
U[..., 54:74, 54:74] = 0.5                # a square of chemical V
V[..., 54:74, 54:74] = 0.25
torch.manual_seed(0)
V += 0.02 * torch.rand(1, 1, N, N)

feed, kill = 0.037, 0.06
for t in range(10_000):
    uvv = U * V * V                       # the reaction: U + 2V -> 3V
    U += 0.16 * lap(U) - uvv + feed * (1 - U)
    V += 0.08 * lap(V) + uvv - (feed + kill) * V

plt.figure(figsize=(2.4, 2.4))
plt.imshow(V[0, 0], cmap='magma')
plt.axis('off')
plt.show()
```

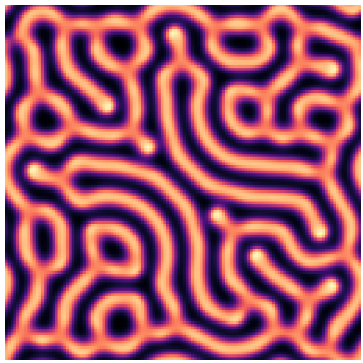


Figure 13.10: Chemical V in the Gray–Scott model after 10,000 steps, grown from a single square.

From a planted square, a labyrinth of stripes grows to fill the dish (Figure 13.10), like the patterns on a fish or a seashell. Alan Turing

proposed in 1952 that some animal markings form by *reaction–diffusion*, though in his version the pattern arises from an even mixture all by itself rather than from a planted seed. Change `feed` and `kill` slightly and you get spots, or worms, or patterns that divide like cells.

These simulations are not neural networks, but they show what repeated local rules can do: a 3×3 kernel, applied over and over, is enough to produce motion, waves and growth. Researchers have even trained small networks to *learn* such rules from examples, and Appendix A does it for the Game of Life. A deep convolutional network has the same structure, a stack of local operations applied one after another, except that its kernels are not given by physics. They are learned, and in the next chapter we learn them.

Try it

1. Cut a different coin out of `coins`, one that Figure 13.4 missed, and run `match_template` with it. Which coins does it find now? How many matches do you get if you lower `threshold_abs` to 0.4?

2. Replace `correlate2d` with `convolve2d` (also in `scipy.signal`) in the `filt` helper. The blur is unchanged; what happens to the two edge images, and why?

3. Write a kernel that detects *diagonal* edges, running from bottom left to top right, and apply it to `camera`. (Hint: rotate the Sobel kernel by 45 degrees in your head.)

4. Change the blur kernel to 15×15. How does the blur change? Then try `blur = np.zeros((15, 15)); blur[7, :] = 1 / 15`, a single row of weights. What kind of blur is that?

5. Change the Game of Life rule in `life_step` so that a dead cell comes to life with exactly three *or six* neighbors (a rule called HighLife). Start from a random soup. Does it look different?

6. In `cv_waves`, multiply `v` by 0.99 after each step. What does that do to the waves, and what does it correspond to in a real drum?

7. Run `cv_gray_scott` with `feed, kill = 0.03, 0.062`, and then with `0.025, 0.06`. Describe the patterns you get.

8. In `cv_timing`, change the image to `np.tile(camera, (4, 4))`, sixteen times as many pixels, and run the timings again. Which of the three methods slows down the most?

Chapter 14

A Convolutional Network

We now have every ingredient. From Chapter 12 we have layers, ReLUs and training by gradient descent; from Chapter 13, the idea of a small kernel slid across the image. A *convolutional neural network* (CNN) puts them together: its first layers are banks of kernels, and instead of designing the kernels as we did for coins and edges, we let gradient descent learn them. In this chapter we build one from scratch, in the same style as before, with nothing but a list of weight tensors, and train it on Fashion-MNIST.

14.1 Images with channels

A convolution layer takes a stack of images, called *channels*, and produces a new stack. The input to the first layer is a grayscale image, so one channel, and PyTorch wants that channel as an explicit axis. This time we do not flatten. (The `device`, `montage_plot` and `get_batch` carry over from earlier chapters, as before.)

cnn_load

```
import torch
import torch.nn.functional as F
from torchvision.datasets import FashionMNIST
torch.manual_seed(0)

def load(train):
    ds = FashionMNIST('data', train=train, download=True)
    X = torch.tensor(ds.data.numpy()[:, None] / 255.0,  # channel axis
                     dtype=torch.float32)
    return X.to(device), ds.targets.to(device), ds.classes

X, Y, classes = load(train=True)
X_test, Y_test, _ = load(train=False)

order = torch.randperm(len(X)).to(device)     # shuffle, then set aside
X_val, Y_val = X[order[:5000]], Y[order[:5000]]   # 5,000 for validation
X, Y = X[order[5000:]], Y[order[5000:]]
print(X.shape, X_val.shape, X_test.shape)
```

OUTPUT

```
torch.Size([55000, 1, 28, 28]) torch.Size([5000, 1, 28, 28])
    ↪ torch.Size([10000, 1, 28, 28])
```

The validation set is 5,000 images this time, picked at random rather than taken from the end. Any set the network does not train on will do; 5,000 is enough to measure accuracy to about half a percentage point, by the square-root law of Chapter 9, and leaves more for training.

Each image now has shape (1, 28, 28): one channel, 28 rows, 28 columns. A color image would have three channels, and the outputs of a convolution layer with 32 kernels have 32, one feature map per kernel. A kernel in the second layer looks at all 32 maps of the first at once, so its weights have shape (32, 3, 3), and the second layer combines the first layer's patterns into larger ones.

14.2 Max pooling

Between convolution layers we will shrink the feature maps with *max pooling*: cut each map into 2×2 blocks and keep only the largest value in each.

cnn_pool

```
x = torch.tensor([[1., 5., 0., 2.],
                  [3., 2., 1., 1.],
                  [0., 0., 4., 0.],
                  [7., 1., 0., 3.]])
print(F.max_pool2d(x[None, None], 2)[0, 0])
```

OUTPUT

```
tensor([[5., 2.],
        [7., 4.]])
```

Pooling halves the width and height of every map, which does three useful things. It cuts the amount of computation for the layers that follow. It lets each kernel in the next layer cover twice as much of the original image. And it builds in a little tolerance to position: if a pattern moves by a pixel inside a pooling block, the maximum stays the same. "There is an edge somewhere around here" is often all the next layer needs to know.

14.3 The network

Our network has two convolution layers of 32 and 64 kernels, each followed by a ReLU and max pooling, and then the familiar fully connected layers of Chapter 12 to turn the final maps into ten scores. The weights are created with the same He scaling as before; for a convolution, the number of inputs to each output is the number of input channels times the kernel area.

cnn_weights

```
def conv_layer(n_in, n_out, k=3):
    std = (2 / (n_in * k * k)) ** 0.5
    W = torch.randn(n_out, n_in, k, k, device=device) * std
    b = torch.zeros(n_out, device=device)
    return [W.requires_grad_(), b.requires_grad_()]

def layer(n_in, n_out):
    std = (2 / n_in) ** 0.5
    W = torch.randn(n_out, n_in, device=device) * std
    b = torch.zeros(n_out, device=device)
    return [W.requires_grad_(), b.requires_grad_()]

w = (conv_layer(1, 32) + conv_layer(32, 64)
     + layer(64 * 7 * 7, 128) + layer(128, 10))
w0 = w[0].detach().clone()      # keep a copy of the starting kernels
```

`F.conv2d` and `F.linear` take the weights and biases directly. Because `F.linear` multiplies by the *transpose* of its weight matrix, the fully connected weights are stored as (*outputs*, *inputs*) this time. The `verbose` flag lets us watch the shapes as a batch of five images flows through:

cnn_model

```
def model(x, w, verbose=False):
    x = F.relu(F.conv2d(x, w[0], w[1], padding=1))   # 32 maps, 28x28
    x = F.max_pool2d(x, 2)                           # 32 maps, 14x14
    if verbose: print(x.shape)
    x = F.relu(F.conv2d(x, w[2], w[3], padding=1))   # 64 maps, 14x14
    x = F.max_pool2d(x, 2)                           # 64 maps, 7x7
    if verbose: print(x.shape)
    x = x.flatten(start_dim=1)                       # 3136 numbers
    x = F.relu(F.linear(x, w[4], w[5]))              # 128
    if verbose: print(x.shape)
    return F.linear(x, w[6], w[7])                   # 10 scores

print(model(X[:5], w, verbose=True).shape)
```

OUTPUT

```
torch.Size([5, 32, 14, 14])
torch.Size([5, 64, 7, 7])
torch.Size([5, 128])
torch.Size([5, 10])
```

The `padding=1` in each `F.conv2d` adds a one-pixel border of zeros around every map before the 3×3 kernel slides over it, so the output keeps the size of the input. Without it, as we saw in Chapter 13, each convolution would shrink the map by two pixels.

Read it as a funnel. A 28×28 image becomes 32 maps of 14×14, then 64 maps of 7×7, then one list of $64 \times 7 \times 7 = 3{,}136$ numbers, then 128, then 10. The spatial size shrinks while the number of channels grows: the network trades *where* for *what*.

cnn_count

```
print('convolutions:', sum(p.numel() for p in w[:4]))
print('total:       ', sum(p.numel() for p in w))
```

OUTPUT

```
convolutions: 18816
total:        421642
```

Notice how few weights the convolutions need: under 19,000 for both layers, because each small kernel is reused at every position. Almost all of the network's 420,000 weights are in the first fully connected layer.

14.4 Training

To measure accuracy we run the network over a whole set of images, a thousand at a time so as not to run out of memory, and without recording gradients. The helper is called `accuracy_on` because it takes images and labels, not scores like the `accuracy` of Chapter 11.

cnn_predict

```
def predict(X, batch_size=1000):      # in batches, to save memory
    with torch.no_grad():
        return torch.cat([model(X[i:i + batch_size], w).argmax(dim=1)
                          for i in range(0, len(X), batch_size)])

def accuracy_on(X, Y):
    return (predict(X) == Y).float().mean().item()
```

The training loop is unchanged; only the model is different.

cnn_train

```
optimizer = torch.optim.Adam(w, lr=0.001)

for step in range(6001):
    x, y = get_batch(X, Y, 128)
    loss = F.cross_entropy(model(x, w), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if step % 1000 == 0:
        val_acc = accuracy_on(X_val, Y_val)
        print(step, round(loss.item(), 3), round(val_acc, 4))
```

OUTPUT

```
0 3.76 0.238
1000 0.255 0.9014
2000 0.169 0.9126
3000 0.154 0.9156
4000 0.12 0.9142
5000 0.114 0.918
6000 0.033 0.9072
```

The last column is accuracy on the 5,000 validation images, which training never sees. It climbs past 90 percent within the first thousand steps, peaks near 92 percent, and has slipped back a little by the end. One dip could be the ordinary wobble from one checkpoint to the next, or the start of memorizing; the clearer sign comes below, in the gap between training and validation accuracy. Once training is finished

we measure all three sets, and look at the test set for the first and only time:

cnn test

```
print('training:  ', round(accuracy_on(X, Y), 4))
print('validation:', round(accuracy_on(X_val, Y_val), 4))
print('test:      ', round(accuracy_on(X_test, Y_test), 4))
```

OUTPUT

```
training:   0.9735
validation: 0.9072
test:       0.909
```

On the test set the CNN scores 90.9 percent, against 88.3 for the deep fully connected network of Chapter 12. (The validation numbers, 90.7 here against 89, come from different validation sets, which is one more reason to compare on the shared test set.) That gain is bigger than it looks: the error rate falls from 11.7 percent to 9.1, so the CNN makes about a fifth fewer mistakes. And it got there by knowing something about images that the fully connected network did not, namely that nearby pixels go together and that a pattern means the same thing wherever it appears.

Notice the first line, though. On the images it trained on, the network is right 97.4 percent of the time, six and a half points better than on images it has never seen. With 420,000 weights and only 55,000 training images, it has room to memorize, and the gap is larger than the 4 points we saw for the fully connected network on clothing.

14.5 Which mistakes?

A single accuracy number hides a lot. A *confusion matrix* counts, for every true class (row), how often the network guessed each class (column). Correct answers lie on the diagonal.

cnn_confusion

```
guesses = predict(X_test)
confusion = torch.zeros(10, 10, dtype=torch.long)
for actual, guess in zip(Y_test.tolist(), guesses.tolist()):
    confusion[actual, guess] += 1
print(confusion)
```

OUTPUT

```
tensor([[870,   1,  15,   5,   7,   1,  95,   0,   6,   0],
        [  3, 984,   0,   8,   1,   0,   3,   0,   1,   0],
        [ 16,   1, 823,   4,  65,   0,  91,   0,   0,   0],
        [ 26,   5,  12, 859,  42,   0,  56,   0,   0,   0],
        [  0,   1,  25,   8, 917,   0,  49,   0,   0,   0],
        [  0,   0,   0,   0,   0, 958,   0,  24,   2,  16],
        [ 94,   2,  37,  12,  71,   0, 778,   0,   6,   0],
        [  0,   0,   0,   0,   0,   3,   0, 934,   1,  62],
        [  4,   1,   2,   2,   2,   2,   5,   2, 980,   0],
        [  0,   0,   0,   0,   0,   3,   1,   9,   0, 987]])
```

Trousers, bags, ankle boots and sandals are nearly always right. The trouble is concentrated in one corner, among the tops:

cnn_worst

```
mistakes = confusion.clone()
mistakes.fill_diagonal_(0)
for i in mistakes.flatten().argsort(descending=True)[:4].tolist():
    actual, guess = divmod(i, 10)
    print(f'{classes[actual]:>12} taken for {classes[guess]:<12}'
          f'{mistakes[actual, guess].item()} times')
```

OUTPUT

```
 T-shirt/top taken for Shirt       95 times
       Shirt taken for T-shirt/top 94 times
    Pullover taken for Shirt       91 times
       Shirt taken for Coat        71 times
```

At 28×28 pixels in grayscale, the difference between a shirt and a T-shirt is a collar or a button placket a few pixels wide. Many people would struggle with some of these images too.

14.6 What the network learned

The 32 kernels of the first layer are tiny, only 3×3, but we can still draw them. We kept a copy of them before training, `w0`, so we can put the two side by side, with zero as middle gray in both panels (Figure 14.1).

cnn_kernels

```
before = w0[:, 0].cpu().numpy()                    # (32, 3, 3)
after = w[0][:, 0].detach().cpu().numpy()
hi = max(np.abs(before).max(), np.abs(after).max())  # zero: middle gray

plt.figure(figsize=(4.6, 1.5))
for i, (k, name) in enumerate([(before, 'before training'),
                               (after, 'after training')]):
    plt.subplot(1, 2, i + 1)                        # the first 8 of 32
    tiled = montage(k[:8], grid_shape=(2, 4), padding_width=1, fill=-hi)
    plt.imshow(tiled, cmap='gray', vmin=-hi, vmax=hi)
    plt.title(name)
    plt.axis('off')
plt.show()
```

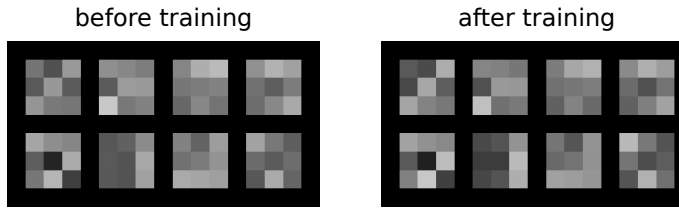


Figure 14.1: Eight of the 32 first-layer kernels, before training (left) and after (right).

You might expect training to have turned random kernels into tidy edge detectors like the Sobel kernel of Chapter 13. It did not. Each kernel after training looks very much like the same kernel before. We can measure how much: the *cosine similarity* of two kernels is their dot product divided by the product of their lengths. It is 1 when the two have exactly the same pattern, whatever their size, and 0 when they have nothing in common.

cnn_similarity
```
def similarity(a, b):          # 1 when two kernels share the same pattern
    a, b = a.flatten(start_dim=1), b.flatten(start_dim=1)
    return (a * b).sum(dim=1) / (a.norm(dim=1) * b.norm(dim=1))

s = similarity(w0, w[0].detach())
print(round(s.mean().item(), 3), round(s.min().item(), 3))
```

OUTPUT
```
0.974 0.898
```

On average each kernel is still 97 percent the same pattern it started with, and even the most changed one is at 0.90. Gradient descent nudged the kernels; it did not redesign them. Before reading too much into that, we should check how much it depends on where they started. Here is the same network trained again, with first-layer kernels started ten times smaller:

cnn control
```
w_trained = w                          # keep the network we trained
w = (conv_layer(1, 32) + conv_layer(32, 64)
     + layer(64 * 7 * 7, 128) + layer(128, 10))
with torch.no_grad():
    w[0] *= 0.1                        # first kernels ten times smaller
start = w[0].detach().clone()
optimizer = torch.optim.Adam(w, lr=0.001)
for step in range(6001):
    x, y = get_batch(X, Y, 128)
    loss = F.cross_entropy(model(x, w), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
s = similarity(start, w[0].detach())
print(round(s.mean().item(), 3), round(s.min().item(), 3))
print('validation', round(accuracy_on(X_val, Y_val), 4))
w = w_trained
```

OUTPUT
```
0.814 0.541
validation 0.9184
```

Now the kernels move a good deal more, to an average similarity of 0.81 and as low as 0.54, and the network does at least as well on the validation images (91.8 percent against 90.7; with one run of each, a point is within the noise). So part of the reason the kernels barely moved is simply that they started large: the same-sized steps change a

large kernel's pattern less. The other part is real. Random 3×3 kernels are already a reasonable set of detectors, as Figure 13.6 suggested: each responds to some mixture of brightness and edges in some direction, and the network does about as well whether it keeps them or reshapes them. With 32 of them, the later layers have plenty to work with, and the rest of the network learns how to use what they report. In big networks trained on millions of color photographs, the first-layer kernels end up looking quite different, as we will see in the next chapter.

What the kernels do is easier to see in the feature maps they produce for an actual image (Figure 14.2).

cnn_maps

```
image = X_test[:1]
with torch.no_grad():
    maps = F.relu(F.conv2d(image, w[0], w[1], padding=1))
print(classes[Y_test[0]], maps.shape)
maps = maps[0].cpu().numpy()                    # (32, 28, 28)
montage_plot(maps, size=4.6, grid_shape=(4, 8))
```

OUTPUT

```
Ankle boot torch.Size([1, 32, 28, 28])
```

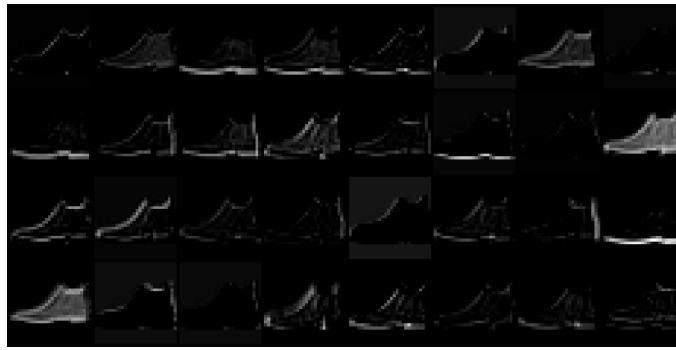


Figure 14.2: The first layer's 32 feature maps for one test image, an ankle boot.

Each map is the boot seen through one kernel. Some respond to the whole boot, some mostly to the sole, some to its outline along the top, and a few are nearly blank for this image. The second layer combines these maps, and its kernels can respond to arrangements of them: corners, curves, a sole meeting an upper. That progression,

from simple responses to parts to objects, becomes much more striking in a much bigger network. In the next chapter we open one up.

Try it

1. Change the training loop in `cnn_train` to run 3,000 steps instead of 6,000, then run `cnn_test` again. Does the gap between training and test accuracy shrink? Does test accuracy suffer?

2. Freeze the first layer at its random start: right after `cnn_weights`, run `w[0].requires_grad_(False)`, then retrain. How much validation accuracy is lost when the first-layer kernels never learn at all? (Chapter 17 takes this idea much further.)

3. Remove the first `F.max_pool2d` from `model`. What size is the input to the first fully connected layer now, and what does `cnn_count` report? (You will need to change `layer(64 * 7 * 7, 128)` to match.)

4. Train the same network on MNIST digits by replacing `FashionMNIST` with `MNIST` in `cnn_load`. How does its test accuracy compare with the 98.1 percent of the fully connected network in Chapter 12?

5. Use `cnn_maps` on an image the network gets wrong: find one with `(predict(X_test) != Y_test).nonzero()`. Do the feature maps look any different from those of the boot?

6. A challenge: teach a network the Game of Life from Chapter 13, using pairs of random worlds and the worlds one step later as training data. Try it yourself first; Appendix A works through one solution, in which a 3×3 convolution with 8 channels learns the rule exactly.

Chapter 15

Inside AlexNet

In 2012 a convolutional network called AlexNet, built by Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, won the ImageNet competition. The task was to sort photographs into a thousand categories, and AlexNet's error rate was far below that of every competing entry. That result is the turning point described in Part I: within a few years, almost every serious approach to computer vision was a deep neural network.

Architecturally, AlexNet is the network of the last chapter scaled up: more layers, many more kernels, color images of 224×224 pixels, and 1.2 million training images. Training it ourselves would take a long time, but we do not need to. The `torchvision` library ships with AlexNet's trained weights.[1] In this chapter we use it to classify photographs, look inside it layer by layer, and then rebuild its forward pass by hand, in the style of the previous chapters, to prove that there is nothing in it we do not understand.

15.1 Loading a trained network

ax_load

```
from torchvision.models import alexnet, AlexNet_Weights

weights = AlexNet_Weights.DEFAULT
net = alexnet(weights=weights).to(device)
net.eval()
print(net)
```

OUTPUT

```
AlexNet(
  (features): Sequential(
```

[1] Strictly, `torchvision`'s AlexNet is a slightly later, simplified version that Krizhevsky described in 2014 ("One weird trick for parallelizing convolutional neural networks"), designed to run on a single graphics card. The 2012 network was split across two cards and had 96 and 256 kernels in its first two layers where this one has 64 and 192, and it included a step called local response normalization that was later dropped. The two are the same design and perform about equally well.

```
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1,
    ↪ ceil_mode=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1,
    ↪ ceil_mode=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU(inplace=True)
    (10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
    ↪ 1))
    (11): ReLU(inplace=True)
    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1,
    ↪ ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=9216, out_features=4096, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=4096, out_features=4096, bias=True)
    (5): ReLU(inplace=True)
    (6): Linear(in_features=4096, out_features=1000, bias=True)
  )
)
```

This time the network is a PyTorch *module* object rather than our list of weights, but the printout should look familiar. The `features` part is five convolution layers with ReLUs and three max pools; the `classifier` part is three fully connected layers. Three things are new. The max pools take 3×3 windows two pixels apart, so neighboring windows overlap. `AdaptiveAvgPool2d((6, 6))` averages each feature map down to 6×6 whatever size it arrives at; for the 224×224 images AlexNet expects, the maps are already 6×6 and it does nothing. And there is *dropout*, which we will come back to shortly. The first convolution uses 64 kernels of 11×11 pixels with a *stride* of 4, meaning the kernel jumps four pixels at a time, which shrinks the large input image quickly.

The call to `net.eval()` matters, and we will see why below. The thousand category names come with the weights:

ax_labels

```
labels = weights.meta['categories']
print(len(labels))
print(labels[:5])
print(labels[281:286])
```

OUTPUT

```
1000
['tench', 'goldfish', 'great white shark', 'tiger shark', 'hammerhead']
['tabby', 'tiger cat', 'Persian cat', 'Siamese cat', 'Egyptian cat']
```

15.2 Preparing an image

AlexNet was trained on images prepared in a particular way, and a new image must be prepared in exactly the same way:

ax_preprocess

```
from torchvision import transforms

preprocess = transforms.Compose([
    transforms.Resize(256),           # shorter side to 256 pixels
    transforms.CenterCrop(224),       # the central 224 x 224 square
    transforms.ToTensor(),            # to a (3, 224, 224) tensor, 0-1
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])
```

The final step shifts and scales each color channel by the average and spread of that channel over the ImageNet training images, so that the inputs have roughly zero mean and unit spread, as they did during training. Feed a network differently prepared data from what it was trained on and it will still produce confident-looking answers. They will just be worse, and nothing will warn you.

15.3 What is in this picture?

Let's show AlexNet the cat from Chapter 7:

ax_classify

```
from PIL import Image
from skimage import data

cat = Image.fromarray(data.chelsea())
x = preprocess(cat)[None].to(device)       # a batch of one
print(x.shape)

with torch.no_grad():
    scores = net(x)
probs = scores.softmax(dim=1)
top = probs.topk(5)
for p, i in zip(top.values[0], top.indices[0]):
    print(f'{p.item():6.1%}  {labels[i]}')
```

OUTPUT

```
torch.Size([1, 3, 224, 224])
 78.6%  Egyptian cat
 13.7%  tiger cat
  7.1%  tabby
  0.5%  lynx
  0.0%  Persian cat
```

The extra `[None]` adds a batch axis: networks always process batches, even a batch of one. Softmax turns the thousand scores into probabilities, and `topk` picks out the most likely five. All five guesses are cats, and the top three are all reasonable descriptions of Chelsea.

Here is a small function to try more photographs:

ax_classify_fn

```
def classify(image, k=3):
    x = preprocess(Image.fromarray(image))[None].to(device)
    with torch.no_grad():
        probs = net(x).softmax(dim=1)[0]
    top = probs.topk(k)
    return [(labels[i], round(p.item(), 3))
            for p, i in zip(top.values, top.indices)]

photos = {'astronaut': data.astronaut(), 'coffee': data.coffee(),
          'rocket': data.rocket()}
for name, image in photos.items():
    print(name, classify(image))
```

OUTPUT

```
astronaut [('dumbbell', 0.268), ('puck', 0.062), ('neck brace', 0.052)]
coffee [('espresso', 0.973), ('consomme', 0.017), ('cup', 0.004)]
rocket [('mosque', 0.623), ('drilling platform', 0.171), ('obelisk', 0.074)]
```

ax_photos

```
plt.figure(figsize=(4.6, 1.5))
for i, (name, image) in enumerate(photos.items()):
    plt.subplot(1, 3, i + 1)
    plt.imshow(image)
    plt.title(name)
    plt.axis('off')
plt.show()
```



Figure 15.1: Three more sample photographs from scikit-image.

The coffee is easy: espresso, 97 percent. The other two are instructive failures. "Astronaut" is not one of the thousand categories, so AlexNet cannot give that answer; it settles on "dumbbell," perhaps drawn by the round shapes in the center crop, but only with 27 percent, and the probability is spread thinly over other guesses. That is a hesitant mistake, and the low number tells us so. The rocket is different: AlexNet calls it a mosque, an echo of tall white structures against the

sky, with 62 percent. That is a *confident* mistake, and nothing in the output distinguishes it from a confident right answer. A probability is how strongly the network prefers one of its answers, not how likely it is to be right. And a classifier can only choose among the answers it was trained on; it will always choose one, however poorly any of them fits. That is worth remembering whenever you read about a system that classifies people, documents or medical images.

15.4 Why `eval()` matters

During training, *dropout* randomly sets half of the numbers flowing between the fully connected layers to zero, a different half each time, and doubles the half that survive. (In general, with a fraction p dropped, the survivors are multiplied by $1/(1-p)$.) The doubling keeps the expected size of every number the same as with nothing dropped, so the next layer sees inputs of the usual size either way. Dropout sounds destructive, but it stops the network from relying on any single unit, which reduces overfitting. When we *use* the network, though, we want all of it working, and we want the same answer every time. Because of the doubling during training, simply switching dropout off leaves the outputs at the right scale. The `train()` and `eval()` methods switch dropout (and a few other layers that behave differently during training) on and off:

ax_eval

```
net.train()                             # training mode: dropout is on
with torch.no_grad():
    a, b = net(x), net(x)
print(torch.allclose(a, b))

net.eval()                              # evaluation mode: dropout is off
with torch.no_grad():
    a, b = net(x), net(x)
print(torch.allclose(a, b))
```

OUTPUT

```
False
True
```

In training mode, the same image gives different scores on two runs. If you forget `eval()`, your predictions will flicker from run to run. If you forget to switch `train()` back on before training, the network

will train without dropout. Both mistakes are silent; we will be careful to avoid them in the next chapter.

15.5 The first layer

The 64 kernels of the first layer each have shape $3 \times 11 \times 11$: one 11×11 grid of weights for each color channel. That means we can draw them as small color images (Figure 15.2).

ax_filters

```
w0 = net.features[0].weight.detach().cpu()        # (64, 3, 11, 11)
print(w0.shape)
f = w0.permute(0, 2, 3, 1).numpy()                # (64, 11, 11, 3)
f = (f - f.min()) / (f.max() - f.min())           # rescale to 0-1
tiled = montage(f, grid_shape=(8, 8), padding_width=1, fill=(1, 1, 1),
                channel_axis=-1)
plt.figure(figsize=(3.2, 3.2))
plt.imshow(tiled)
plt.axis('off')
plt.show()
```

OUTPUT

```
torch.Size([64, 3, 11, 11])
```

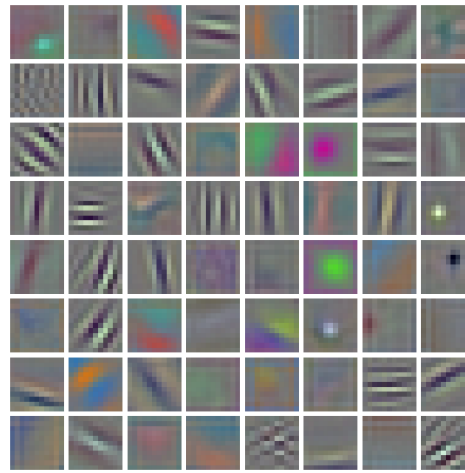


Figure 15.2: AlexNet's 64 first-layer kernels, learned from ImageNet.

Compare these with Figure 14.1. The kernels of our small network were still mostly the random patterns they started as; these are not.

There are edge detectors at every angle, some fine and some coarse, along with stripes, spots, and patches that respond to particular colors. Neuroscientists will recognize a striking resemblance to the receptive fields of neurons in the primary visual cortex. Nobody put them there. They are what a thousand categories of photographs, plus gradient descent, produced.

Applied to the cat, each kernel gives one 55×55 feature map. Rather than squeeze all 64 onto the page, we show eight of them next to the 224×224 crop the network actually sees (Figure 15.3):

ax_maps

```
with torch.no_grad():
    maps = net.features[:2](x)                    # first conv + ReLU
print(maps.shape)

def brighten(maps):                   # scale each map to a maximum of 1
    return maps / (maps.amax(dim=(1, 2), keepdim=True) + 1e-6)

crop = transforms.CenterCrop(224)(transforms.Resize(256)(cat))
picks = [1, 5, 13, 21, 37, 47, 52, 58]           # eight of the 64
shown = brighten(maps[0, picks]).cpu().numpy()

plt.figure(figsize=(4.0, 4.4))
plt.subplot(3, 3, 1)
plt.imshow(crop)
plt.title('input')
plt.axis('off')
for i, k in enumerate(picks):
    plt.subplot(3, 3, i + 2)
    plt.imshow(shown[i], cmap='gray')
    plt.title(f'map {k}')
    plt.axis('off')
plt.show()
```

OUTPUT

```
torch.Size([1, 64, 55, 55])
```

Each map is scaled to its own maximum before drawing, so that weak responses are visible too. Some maps trace the outlines of the eyes and the head, some respond to the fine stripes of the fur, and some to broad areas of brightness or color. Changing the numbers in `picks` shows the others.

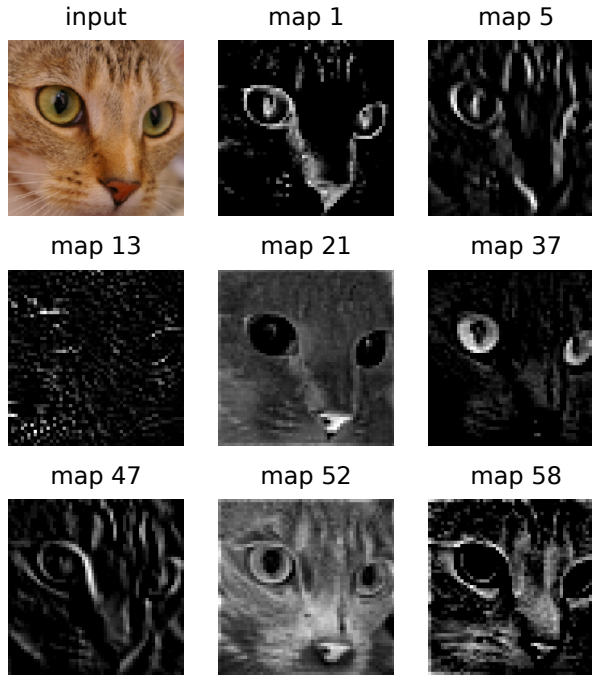


Figure 15.3: The input crop and eight of the 64 first-layer feature maps for the cat.

15.6 Deeper layers

After all five convolution layers, the image has been reduced to 256 maps of only 6×6:

ax deep maps

```
with torch.no_grad():
    deep = net.features(x)                          # all conv layers
print(deep.shape)
deep_maps = brighten(deep[0, :64]).cpu().numpy()  # (64, 6, 6)
tiled = montage(deep_maps, grid_shape=(8, 8), padding_width=1, fill=0.5)
plt.figure(figsize=(3.2, 3.2))
plt.imshow(tiled, cmap='gray', vmin=0, vmax=1)
plt.axis('off')
plt.show()
```

OUTPUT

```
torch.Size([1, 256, 6, 6])
```

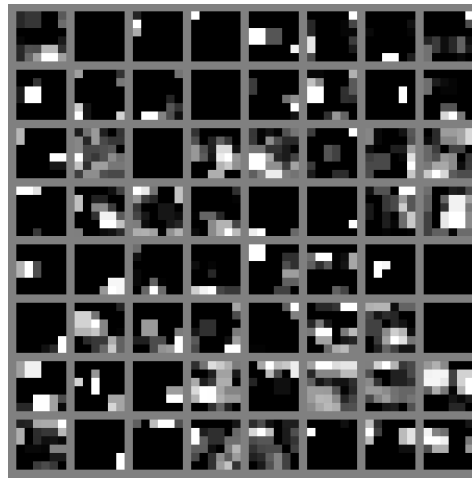


Figure 15.4: The first 64 of the 256 deepest feature maps for the cat, each 6×6, separated by gray lines.

These maps no longer look like the image at all. Each one is a detector for some complex combination of shapes, colors and textures, roughly located on a coarse 6×6 grid. One might fire for "furry texture here," another for "pointed ear shape there." The fully connected layers then combine these 9,216 numbers into the thousand class scores. This is the progression from pixels to perception that gives

this book its subtitle. At the bottom are measurements of light; at the top, descriptions of objects; and in between, layer after layer of learned features, each built from the ones below.

15.7 AlexNet by hand

How much of this machinery is actually new? Almost none. Here is AlexNet's entire forward pass, written with the same functions we have been using, reading the weights *and biases* directly out of the trained modules:

ax_manual

```
import torch.nn.functional as F

c = [net.features[i] for i in (0, 3, 6, 8, 10)]  # the conv layers
fc = [net.classifier[i] for i in (1, 4, 6)]      # the linear layers

def alexnet_by_hand(x, show=False):
    x = F.relu(F.conv2d(x, c[0].weight, c[0].bias, stride=4, padding=2))
    x = F.max_pool2d(x, kernel_size=3, stride=2)
    if show: print(x.shape)
    x = F.relu(F.conv2d(x, c[1].weight, c[1].bias, padding=2))
    x = F.max_pool2d(x, kernel_size=3, stride=2)
    if show: print(x.shape)
    x = F.relu(F.conv2d(x, c[2].weight, c[2].bias, padding=1))
    x = F.relu(F.conv2d(x, c[3].weight, c[3].bias, padding=1))
    x = F.relu(F.conv2d(x, c[4].weight, c[4].bias, padding=1))
    x = F.max_pool2d(x, kernel_size=3, stride=2)
    if show: print(x.shape)
    x = F.adaptive_avg_pool2d(x, 6).flatten(start_dim=1)
    if show: print(x.shape)
    x = F.relu(F.linear(x, fc[0].weight, fc[0].bias))
    x = F.relu(F.linear(x, fc[1].weight, fc[1].bias))
    return F.linear(x, fc[2].weight, fc[2].bias)

with torch.no_grad():
    mine = alexnet_by_hand(x, show=True)
    theirs = net(x)
print(torch.allclose(mine, theirs, atol=1e-4))
print(labels[mine.argmax()])
```

OUTPUT

```
torch.Size([1, 64, 27, 27])
torch.Size([1, 192, 13, 13])
torch.Size([1, 256, 6, 6])
torch.Size([1, 9216])
True
Egyptian cat
```

Our hand-built version produces the same scores as the real thing. (Dropout does not appear, because in evaluation mode it does nothing.)

Five convolutions, seven ReLUs, three max pools, an average pool and three matrix multiplications: that is all AlexNet is. What makes it work is not the code but the weights. There are a lot of them:

ax_count

```
print(sum(p.numel() for p in net.parameters()))
```

OUTPUT

```
61100840
```

Sixty-one million numbers, found by gradient descent on 1.2 million images.

15.8 Saving and loading weights

Since the weights are everything, saving a network means saving its weights. A module's `state_dict()` is a dictionary from layer names to weight tensors, and `torch.save` writes it to a file. To load it, create a network of the same shape, whose weights start out random, and pour the saved weights in:

ax_save

```
torch.save(net.state_dict(), 'data/alexnet.pt')

copy = alexnet()                                  # random weights
copy.load_state_dict(torch.load('data/alexnet.pt', map_location=device))
copy = copy.to(device).eval()
with torch.no_grad():
    print(torch.allclose(copy(x), net(x)))
```

OUTPUT

```
True
```

That is how trained networks are shared: `alexnet(weights=...)` did exactly this with a file downloaded from the internet. In the next chapter we will save weights of our own, after teaching AlexNet a new task.

Try it

1. Classify a photograph of your own. With `f` the name of the file, run `im = Image.open(f).convert('RGB')` and then `classify(np.array(im))`. Try something that is among the thousand categories and something that is not. Can you tell the two cases apart from the probabilities alone?

2. Watch dropout at work on its own: `d = torch.nn.Dropout(0.5); d.train(); print(d(torch.ones(12)))`. Which values come out, and why? Run it again, then try `d.eval()`.

3. Remove the `Normalize` step from `preprocess` and classify the cat and the coffee again. Does AlexNet warn you that something is wrong?

4. Mirror the cat with `data.chelsea()[:, ::-1]`, or cut out just its face with `data.chelsea()[:200, 100:350]`, and classify it. Which changes affect the answer, and by how much?

5. Change `picks` in `ax_maps` to look at other first-layer maps, and compare each one with its kernel in Figure 15.2, where the kernels are numbered row by row from 0 at the top left. Can you predict from a kernel what its map will show?

Chapter 16

Teaching an Old Network New Tricks

AlexNet knows a thousand categories. Suppose you need a category it does not know: a particular plant disease, defects on a production line, or your own dog versus the neighbor's. Training a network of 61 million weights from scratch would need far more examples than you have. But you do not have to start from scratch. The early layers of AlexNet detect edges, textures and shapes that are useful for almost *any* photograph; the layers grow more specific to the thousand ImageNet categories as they go, and the last layer is nothing but a score for each of them. Replace that last layer with one for your own categories, and train briefly on a modest number of examples, either the new layer alone or, gently, the whole network. This is *transfer learning*, and it is probably the most practically useful technique in this book. We will try both versions, and, to see what the borrowed weights are worth, compare them with the same network trained from random weights.

16.1 A small dataset

We will teach AlexNet to tell ants from bees, using a small dataset from the PyTorch tutorials: a few hundred photographs, sorted into one folder per class. The code downloads and unpacks it into the `data` folder the first time it runs, then counts the photographs:

tf_folders

```
import os
from torchvision.datasets.utils import download_and_extract_archive

data_dir = 'data/hymenoptera_data'
if not os.path.exists(data_dir):        # 47 MB, downloaded only once
    url = 'https://download.pytorch.org/tutorial/hymenoptera_data.zip'
    download_and_extract_archive(url, 'data')
for split in ['train', 'val']:
    for name in sorted(os.listdir(f'{data_dir}/{split}')):
        n = len(os.listdir(f'{data_dir}/{split}/{name}'))
        print(split, name, n)
```

OUTPUT

```
train ants 124
train bees 121
val ants 70
val bees 83
```

This one-folder-per-class layout is the standard way to organize image data, and it is all you need to build a dataset of your own. Take a few dozen photographs of each thing you want to recognize, with a phone or from an image search, and sort them into `train` and `val` folders with one subfolder per class. The code in this chapter will work unchanged.

16.2 More data for free

With only about 120 training photographs per class, we will stretch them with *data augmentation*. Every time a training image is used, we take a randomly sized and positioned crop of it and mirror it half of the time. An ant is still an ant when seen slightly closer or the other way around, so each epoch the network sees new variations of the same photographs. Validation images get the plain, repeatable preparation from Chapter 15.

tf_transforms

```
from torchvision import transforms

normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                 std=[0.229, 0.224, 0.225])
train_tf = transforms.Compose([
    transforms.RandomResizedCrop(224),     # a random part of the photo
    transforms.RandomHorizontalFlip(),     # mirrored half of the time
    transforms.ToTensor(),
    normalize,
])
val_tf = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    normalize,
])
```

PyTorch's `ImageFolder` reads the folder layout, using the subfolder names as labels, and a `DataLoader` serves the images in shuffled batches. The dataset comes with only two folders, so we treat its `val` folder as our test set, to be looked at once at the end, and carve a validation set of 50 photographs out of the training folder, as we did for Fashion-MNIST. `Subset` picks images from a dataset by position. We open the training folder twice, once with augmentation and once without, so that the validation photographs get the plain preparation:

tf_loaders

```
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader, Subset
torch.manual_seed(0)

train_all = ImageFolder(f'{data_dir}/train', train_tf)
plain_all = ImageFolder(f'{data_dir}/train', val_tf)  # same, unaugmented
order = torch.randperm(len(train_all)).tolist()
train_data = Subset(train_all, order[50:])
val_data = Subset(plain_all, order[:50])          # 50 for validation
test_data = ImageFolder(f'{data_dir}/val', val_tf)  # kept for the end

train_loader = DataLoader(train_data, batch_size=16, shuffle=True)
val_loader = DataLoader(val_data, batch_size=64)
test_loader = DataLoader(test_data, batch_size=64)

classes = train_all.classes
print(classes, len(train_data), len(val_data), len(test_data))
```

OUTPUT

```
['ants', 'bees'] 194 50 153
```

That leaves 194 photographs to learn from. The loader found 244 images in the training folder where our count found 245. The odd one out is a file called `imageNotFound.gif`, a placeholder that came along when the dataset was collected from the web, and which `ImageFolder` skips because GIF is not among the image formats it accepts. Real datasets are always a little dirty, so it pays to look.

Here is one batch of training images, as the network will see them (Figure 16.1):

tf_show

```
def show(t):                          # undo the normalization, for display
    mean = torch.tensor([0.485, 0.456, 0.406])[:, None, None]
    std = torch.tensor([0.229, 0.224, 0.225])[:, None, None]
    return (t * std + mean).clamp(0, 1).permute(1, 2, 0).numpy()

images, labels = next(iter(train_loader))
plt.figure(figsize=(4.6, 2.6))
for i in range(8):
    plt.subplot(2, 4, i + 1)
    plt.imshow(show(images[i]))
    plt.title(classes[labels[i]])
    plt.axis('off')
plt.show()
```

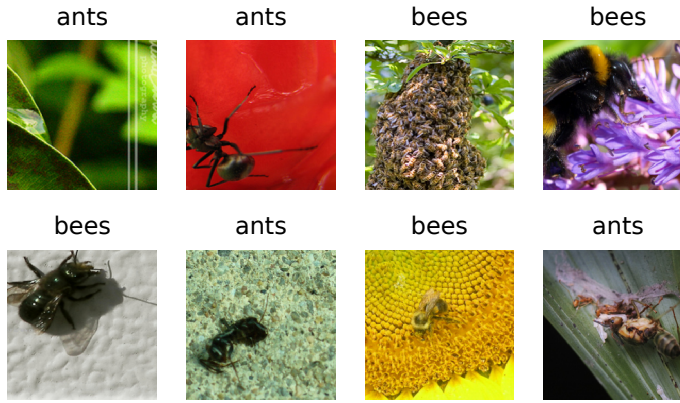


Figure 16.1: A batch of augmented training images: random crops, some mirrored.

16.3 A new head

Since Chapter 15 our network has been a PyTorch *module* rather than a list of weights. A module (technically, an object of a class derived from `nn.Module`) is a piece of network that holds its own weights and can be called like a function. Modules contain other modules: `net` contains `net.classifier`, an `nn.Sequential` like the one at the end of Chapter 12, which contains `net.classifier[6]`, the final `nn.Linear` layer. `nn.Linear` is the module version of the `F.linear` layers we have been writing by hand: it holds a weight matrix and a bias. `net.parameters()` lists the weights of every module inside `net`, and we can replace any of those modules by assigning a new one.

That is what we do here. We load the pretrained AlexNet and replace its last layer, the $4096 \to 1000$ matrix multiplication that produces ImageNet scores, with a new, randomly initialized $4096 \to 2$ layer. Before adding the new layer we *freeze* all the others, by switching off their `requires_grad`, so that gradient descent will leave ImageNet's weights exactly as they are:

tf_model

```
import torch.nn as nn
from torchvision.models import alexnet, AlexNet_Weights

net = alexnet(weights=AlexNet_Weights.DEFAULT)
print(net.classifier[6])
net.requires_grad_(False)                    # freeze every weight...
net.classifier[6] = nn.Linear(4096, len(classes))   # ...add a new layer
net = net.to(device)
print(net.classifier[6])
print([p.shape for p in net.parameters() if p.requires_grad])
```

OUTPUT

```
Linear(in_features=4096, out_features=1000, bias=True)
Linear(in_features=4096, out_features=2, bias=True)
[torch.Size([2, 4096]), torch.Size([2])]
```

The last line lists the weights that can still learn: the new layer's 2×4096 matrix and its two biases, about eight thousand numbers out of 57 million (AlexNet's 61 million, less the four million of the ImageNet layer we just removed). Everything before that layer still carries what AlexNet learned from ImageNet. Before training, the new layer is random, so the network should do no better than a coin toss:

tf_evaluate

```
def evaluate(net, loader):
    net.eval()
    correct = 0
    with torch.no_grad():
        for x, y in loader:
            guesses = net(x.to(device)).argmax(dim=1)
            correct += (guesses == y.to(device)).sum().item()
    return correct / len(loader.dataset)

print(round(evaluate(net, val_loader), 3))
```

OUTPUT

```
0.52
```

With 50 validation photographs, 0.52 is indeed a coin toss. This `evaluate` is new: the one of Chapter 12 took a list of weights and whole tensors, while this one takes a module and a data loader. Note its structure: it switches the network to `eval()` mode and turns off gradient tracking, two separate things that are both needed when we only want answers.

16.4 Training the new layer

Now we train. The loop is the one we have used throughout, organized by epochs over the data loader. The optimizer is SGD with `momentum=0.9`, which lets each step carry along nine tenths of the previous step, smoothing out the noise of small batches, as in Chapter 11. There are three details that matter:

- we hand the optimizer only the parameters it should change, here `net.classifier[6].parameters()`;
- `net.train()` at the start of every epoch, because `evaluate` switched the network to evaluation mode at the end of the previous one;
- a small learning rate, so that pretrained weights, when we do let them change, are adjusted rather than overwritten.

We keep the weights from the *last* epoch. It is tempting to keep the best-scoring epoch instead, but with a validation set this small the best of ten noisy scores flatters the network, and we would be choosing the weights by the very numbers we report.

tf_train

```
def train(net, params, epochs, lr=0.001):
    optimizer = torch.optim.SGD(params, lr=lr, momentum=0.9)
    history = []
    for epoch in range(epochs):
        net.train()                                  # dropout on
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            loss = nn.functional.cross_entropy(net(x), y)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        acc = evaluate(net, val_loader)              # switches to eval()
        history.append(round(acc, 3))
    print('validation accuracy by epoch:')
    print(history)
    return history

frozen = train(net, net.classifier[6].parameters(), epochs=10)
```

OUTPUT

```
validation accuracy by epoch:
[0.84, 0.82, 0.92, 0.84, 0.94, 0.84, 0.9, 0.86, 0.9, 0.86]
```

The printout is the validation accuracy after each epoch. After the first pass through the photographs the new layer is already right 84 percent of the time, it touches 94 percent at epoch 5, and it ends at 86. With only 50 validation images, each one is worth 2 points, so the ups and downs from epoch to epoch, between 82 and 94, are mostly noise. On our Mac each epoch took less than a second: only 8,000 weights are learning, and the gradient never has to travel back through the frozen layers.

16.5 Fine-tuning everything

The frozen layers were trained to tell apart a thousand ImageNet categories, not ants from bees. We can let them adjust too. This is called *fine-tuning*: start again from the pretrained weights, replace the last layer, and this time let every weight change. Because training starts from good weights and the learning rate is small, a few epochs are enough.

tf_finetune

```
tuned_net = alexnet(weights=AlexNet_Weights.DEFAULT)
tuned_net.classifier[6] = nn.Linear(4096, len(classes))
tuned_net = tuned_net.to(device)
tuned = train(tuned_net, tuned_net.parameters(), epochs=10)
```

OUTPUT

```
validation accuracy by epoch:
[0.72, 0.8, 0.84, 0.78, 0.84, 0.84, 0.84, 0.86, 0.82, 0.86]
```

Fine-tuning starts lower, at 72 percent, and also ends at 86 percent. On this small dataset, letting 57 million weights change made no clear difference: with a validation set of 50 photographs, the two runs are within noise of each other, and there is no sign that 194 photographs can improve much on what ImageNet taught the lower layers. With a larger dataset, or one that looks less like ImageNet (X-rays, say, or microscope slides), fine-tuning often wins.

16.6 Without the head start

What are the pretrained weights actually worth? The same architecture, the same data and the same training loop, but starting from random weights, as `alexnet()` without the `weights` argument gives us:

tf_scratch

```
scratch_net = alexnet()                          # random weights
scratch_net.classifier[6] = nn.Linear(4096, len(classes))
scratch_net = scratch_net.to(device)
scratch = train(scratch_net, scratch_net.parameters(), epochs=10)
```

OUTPUT

```
validation accuracy by epoch:
[0.46, 0.48, 0.46, 0.46, 0.46, 0.46, 0.46, 0.46, 0.46, 0.46]
```

This network learns nothing. Its validation accuracy sits at 0.46 in almost every epoch, which is what you get by calling every photograph an ant. At these settings (a learning rate chosen for fine-tuning, ten passes over 194 photographs) training 57 million weights from random starting values simply fails. That is a failure of training, not proof that random weights carry no information; the next chapter measures what they do carry.

Now that all three are trained, we look at the test set, once, for each:

tf_compare

```
results = {'last layer only': (net, frozen),
           'fine-tune all': (tuned_net, tuned),
           'from scratch': (scratch_net, scratch)}
for name, (model, history) in results.items():
    test_acc = evaluate(model, test_loader)
    print(f'{name:16} validation {history[-1]:.3f}  test {test_acc:.3f}')
```

OUTPUT

```
last layer only  validation 0.860  test 0.869
fine-tune all    validation 0.860  test 0.882
from scratch     validation 0.460  test 0.458
```

The test set tells the same story as validation. Training only the new last layer gives 86.9 percent on photographs the network has never seen; fine-tuning everything gives 88.2; starting from scratch gives 45.8, the fraction of ants in the test set, because it calls everything an ant. (The test set has 153 photographs, so the gap between the first two, two photographs, is well within noise.) What separates a useful classifier from a useless one here is the head start: features AlexNet

learned from 1.2 million ImageNet photographs. That is why transfer learning is the first thing to try whenever you have a small dataset.

16.7 Did it already know?

Before we celebrate, one more control, and it is the one most often forgotten. We said we were teaching AlexNet something new. Is that true? Its thousand categories are listed in its weights' description, and we can simply look for ours. If they are there, we can build a two-class network with no training at all: keep AlexNet exactly as it is, but keep only the two rows of its last layer that score those two categories.

tf_zeroshot

```
names = AlexNet_Weights.DEFAULT.meta['categories']
ant, bee = names.index('ant'), names.index('bee')
print(ant, names[ant], bee, names[bee])

zero_shot = alexnet(weights=AlexNet_Weights.DEFAULT)
old = zero_shot.classifier[6]                 # the 1,000 ImageNet scores
zero_shot.classifier[6] = nn.Linear(4096, 2)  # keep only two of them
with torch.no_grad():
    zero_shot.classifier[6].weight.copy_(old.weight[[ant, bee]])
    zero_shot.classifier[6].bias.copy_(old.bias[[ant, bee]])
zero_shot = zero_shot.to(device)
print('validation', evaluate(zero_shot, val_loader),
      ' test', round(evaluate(zero_shot, test_loader), 3))
```

OUTPUT

```
310 ant 309 bee
validation 0.98  test 0.974
```

They are there: category 310 is *ant* and 309 is *bee*. And AlexNet, untouched, tells our ants from our bees 97.4 percent of the time, better than either network we trained. We replaced a last layer that already knew the answer, having learned it from more than a thousand photographs of each insect, with a random one, and then retrained it on 194. Worse, the PyTorch tutorial that supplies these photographs describes them as a small subset of ImageNet itself, so some of our "test" photographs may well be ones AlexNet was trained on.[1]

None of this makes the chapter's recipe wrong; it works the same way for categories that are truly new, and you can check that with a

[1] "This dataset is a very small subset of imagenet": Sasank Chilamkurthy, "Transfer Learning for Computer Vision Tutorial," PyTorch Tutorials.

dataset of your own. But it does mean that this dataset flatters any network that starts from ImageNet, and it is a lesson worth more than the recipe. Whenever you borrow a pretrained network, ask what it was trained on, and whether your problem, or even your test set, was part of it. A borrowed network that scores well may be remembering rather than generalizing, and the only way to know is to look.

16.8 Looking at the answers

We keep the simplest of the two good networks, the one in which only the last layer learned, and look at its answers for eight test photographs:

tf_predictions

```
net.eval()                                  # last layer only
picks = range(0, len(test_data), 19)        # spread across both classes
x = torch.stack([test_data[k][0] for k in picks])
y = torch.tensor([test_data[k][1] for k in picks])
with torch.no_grad():
    guesses = net(x.to(device)).argmax(dim=1).cpu()

plt.figure(figsize=(4.6, 2.6))
for i in range(8):
    plt.subplot(2, 4, i + 1)
    plt.imshow(show(x[i]))
    color = 'black' if guesses[i] == y[i] else 'red'
    plt.title(classes[guesses[i]], color=color)
    plt.axis('off')
plt.show()
```

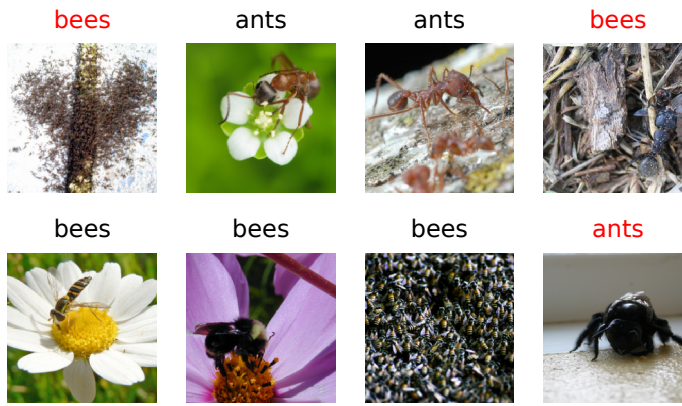


Figure 16.2: Predictions on test images. A red title marks a mistake.

Five of the eight are right, including an ant standing on a white flower. The mistakes are revealing. A dark, fuzzy bee on a bare floor is called an ant, and two ants among stalks and dry plant litter are called bees. Bees in the training photographs are usually on flowers, and ants usually on soil or bark, so the network may have learned something about the setting along with the insect, and these insects are in each other's settings. Nothing in the training data tells it which of the two we meant. This kind of *shortcut learning* is one of the most common ways that classifiers fail in the real world, and the only reliable defense is to look closely at the mistakes.

16.9 Saving and using the model

Saving the network works exactly as in Chapter 15:

tf_save

```
torch.save(net.state_dict(), 'data/ants_bees.pt')
```

To use it later, perhaps in a different program, we rebuild the same architecture, including the two-class last layer, load the weights, and switch to evaluation mode. A small `predict` function then prepares any image file *exactly* as the validation and test images were prepared and returns the most likely class and its probability:

tf_load

```
from PIL import Image

def load_model(path):
    net = alexnet()
    net.classifier[6] = nn.Linear(4096, len(classes))
    net.load_state_dict(torch.load(path, map_location=device))
    return net.to(device).eval()

def predict(net, image_file):
    image = Image.open(image_file).convert('RGB')
    x = val_tf(image)[None].to(device)
    with torch.no_grad():
        probs = net(x).softmax(dim=1)[0]
    return classes[probs.argmax()], round(probs.max().item(), 3)

model = load_model('data/ants_bees.pt')
folder = f'{data_dir}/val/bees'
photo = os.path.join(folder, sorted(os.listdir(folder))[0])
print(predict(model, photo))
```

OUTPUT

```
('bees', 1.0)
```

That is a complete, practical image classifier: a few hundred photographs, a network borrowed from someone else's much larger effort, and a few seconds of computing. The same recipe, with a different folder of images, will learn to recognize many things you can photograph, and with your own photographs you will be sure the network has not seen them before. The rover of Chapter 1 is this recipe exactly: three folders of camera frames, labeled *left*, *right* and *forward*, a borrowed network with a new three-way last layer, and a `predict` call for every frame the camera takes. But how much of the

borrowed network's success comes from its training, and how much from its shape? The next chapter answers that with networks whose hidden layers are never trained at all.

Try it

1. Train each of the three networks for 3 epochs instead of 10 and run `tf_compare` again. Which of them needs the extra epochs?
2. Give `tf_scratch` a fair chance: train it with `lr=0.01` and for 30 epochs. Does it ever beat calling everything an ant?
3. Remove the augmentation by setting `train_tf = val_tf` before `tf_loaders`, then run `tf_model` and `tf_train` again. What happens to validation accuracy, and to how much it jumps around?
4. Fine-tune with a ten times smaller learning rate: rebuild `tuned_net` from the pretrained weights as in `tf_finetune`, then train it with `train(tuned_net, tuned_net.parameters(), epochs=10, lr=0.0001)`. Does it now match or beat training only the last layer?
5. Make your own two-class dataset: a few dozen phone photographs of each of two things, in `train` and `val` folders with one subfolder per class. Point `data_dir` at it and run the chapter.

Chapter 17

Random Features

In the last chapter we froze AlexNet's convolutional layers and trained only a new last layer. The frozen layers did most of the work, but they were not just any layers: they had been trained for days on a million photographs. This chapter asks a stranger question. What if the frozen layers had never been trained at all? What if their weights were simply random?

We have seen a hint already. In Chapter 14, the first-layer kernels of a trained network were still 97 percent the random patterns they started as, and kernels started smaller, which moved more, did no better. Here we take the idea to its limit and build networks whose hidden layers are random and stay random. Only the last layer learns, and it does not even need gradient descent: its weights can be *solved* for in a single step. Then we will measure, honestly, how far random features go on digits, on clothing and on photographs, and meet the same idea in time: a random network that remembers a signal and learns to continue it.

17.1 A curve from random bends

Start small, with points scattered around a smooth bump:

rf_curve_data

```
rng = np.random.default_rng(0)
x = np.sort(rng.uniform(-10, 10, 60))
y = np.exp(-0.02 * (x - 4)**2) + 0.1 * rng.normal(size=60)
x_new = np.linspace(-10, 10, 400)          # where we want predictions
```

To fit them, send each x through a layer of h neurons with random weights and a tanh, the S-shaped bend that rises from -1 to 1. Each random neuron contributes a curve that bends at a different place and steepness. The output is a weighted sum of these h curves. Choosing the weights of the sum to make the squared error as small as possible is called a *least-squares* problem, and NumPy's `pinv` (short

for *pseudo-inverse*) solves it in one call. The next section shows what it computes.

rf_curve

```
def random_layer(x, W, b):
    return np.tanh(np.outer(x, W) + b)     # (points, h)

def fit(h, seed=1):
    r = np.random.default_rng(seed)
    W, b = r.normal(0, 0.3, h), r.uniform(-3, 3, h)  # random, then frozen
    H = random_layer(x, W, b)
    w_out = np.linalg.pinv(H) @ y          # solved, not trained
    return random_layer(x_new, W, b) @ w_out

fig, axes = plt.subplots(1, 3, figsize=(4.6, 1.6), sharey=True)
for ax, h in zip(axes, [2, 10, 60]):
    ax.plot(x, y, 'k.', ms=3)
    ax.plot(x_new, fit(h), color='#f97306')
    ax.set_ylim(-0.5, 1.5)
    ax.set_title(f'h = {h}')
plt.show()
```

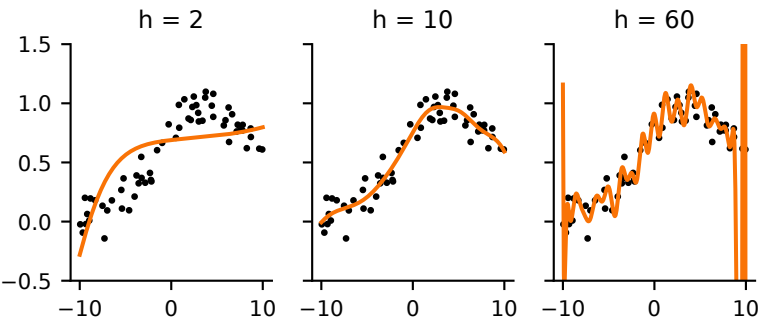


Figure 17.1: Fits made from h random tanh curves. Only the weights of the sum were chosen; the curves themselves are random.

Two random curves cannot make a bump. Ten can, and the fit is good (Figure 17.1). With sixty curves for sixty points, the fit chases every wiggle of the noise and swings wildly, worst of all near the edges, a textbook case of the memorizing we met in Chapter 12. Random neurons are raw material, and the output layer's only job is to choose a mixture of them.

17.2 Solving instead of training

Now the same idea on MNIST. We load the images as rows of 784 numbers, as in Chapter 10:

rf load

```
from torchvision.datasets import MNIST, FashionMNIST

def load(dataset):
    train = dataset('data', train=True, download=True)
    test = dataset('data', train=False, download=True)
    X = train.data.reshape(-1, 784).float().to(device) / 255
    X_test = test.data.reshape(-1, 784).float().to(device) / 255
    return X, train.targets.to(device), X_test, test.targets.to(device)

X, Y, X_test, Y_test = load(MNIST)
print(X.shape, X_test.shape)
```

OUTPUT

```
torch.Size([60000, 784]) torch.Size([10000, 784])
```

Start with no hidden layer at all: a single weight matrix `W` from the 784 pixels to ten outputs, trained to make each output 1 for the correct digit and 0 for the others. The targets `T` are rows of zeros with a 1 for the correct digit, and the loss is the squared error. We already know how to train that, by rolling downhill:

rf_bowl

```
T = F.one_hot(Y, 10).float()           # targets: a 1 for the right digit
W = torch.zeros(784, 10, device=device, requires_grad=True)
optimizer = torch.optim.SGD([W], lr=0.05)
for step in range(5001):
    loss = ((X @ W - T)**2).mean()     # squared error
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if step % 1000 == 0:
        print(step, round(loss.item(), 4))
```

OUTPUT

```
0 0.1
1000 0.0418
2000 0.0409
3000 0.0405
4000 0.0402
5000 0.0401
```

The loss falls fast at first and then creeps. After 5,000 steps it is still going down. Now the shortcut. For a linear output and a squared-error loss, the loss is a perfect bowl: plot it against any weight and you get a parabola, like the x^2 of Chapter 11. A bowl has a single lowest point, and where the slope is zero in every direction can be worked out directly. If H is the matrix of features, one row per image, the best weights W satisfy

$$(H^\mathsf{T}H + \lambda I)\, W = H^\mathsf{T}T.$$

You do not need to follow the algebra: it is just "slope equals zero" written for all the weights at once. It is a system of linear equations, one per weight, and `torch.linalg.solve` solves it in one call. The small number λ, called the *ridge*, adds a penalty for large weights; I is the *identity matrix*, ones on the diagonal and zeros elsewhere, which is how the penalty reaches every weight. We will see in a moment why the ridge matters. (The `pinv` of the last section computes the same answer with a vanishingly small λ.)

rf_solve

```
def solve(H, Y, ridge=1.0):
    """The output weights that best map features H to one-hot Y."""
    T = F.one_hot(Y, 10).cpu().double()
    Hd = H.cpu().double()                    # 64-bit numbers, see below
    G = Hd.T @ Hd + ridge * torch.eye(H.shape[1], dtype=torch.float64)
    W = torch.linalg.solve(G, Hd.T @ T)      # the bottom of the bowl
    return W.float().to(H.device)

def readout_accuracy(H, W, Y):
    return ((H @ W).argmax(dim=1) == Y).float().mean().item()

W = solve(X, Y)
print('loss ', round(((X @ W - T)**2).mean().item(), 4))
print('train', round(readout_accuracy(X, W, Y), 4))
print('test ', round(readout_accuracy(X_test, W, Y_test), 4))
```

OUTPUT

```
loss  0.0391
train 0.8519
test  0.8534
```

The solved weights reach a lower loss, 0.0391, than 5,000 steps of gradient descent managed (0.0401): they sit at the bottom of the bowl the descent was still rolling toward. They get 85 percent of the test digits right, less than the 92 percent the trained linear classifier

of Chapter 11 reached. That is because squared error is not the right loss for classifying. It cares about getting every output exactly 0 or 1, not just about which output is largest. But the shortcut is fast and exact, and from here on the features are the weak point, not the solver. (`solve` does its arithmetic in 64-bit numbers on the processor; we will see why that matters, too. The helper `readout_accuracy` takes features and output weights.)

17.3 A random hidden layer

So let's improve the features with a hidden layer of ReLU neurons whose weights are random and are never trained. Scaling the random weights by $1/\sqrt{784}$ keeps the hidden values a sensible size, the square-root law of Chapter 9 at work. We solve for the output layer, and try several widths:

rf_elm

```
torch.manual_seed(0)

def random_features(X, W1):
    return torch.relu(X @ W1)

for h in [100, 500, 1000, 2000, 4000]:
    W1 = torch.randn(784, h, device=device) / 784**0.5   # never trained
    H = random_features(X, W1)
    W2 = solve(H, Y)
    acc = readout_accuracy(random_features(X_test, W1), W2, Y_test)
    print(f'{h:5d} random neurons   test {acc:.4f}')
```

OUTPUT

```
  100 random neurons   test 0.8182
  500 random neurons   test 0.9181
 1000 random neurons   test 0.9446
 2000 random neurons   test 0.9565
 4000 random neurons   test 0.9679
```

A layer of 4,000 random neurons takes us to almost 97 percent. Nothing in that layer was learned. Each neuron computes some arbitrary mixture of the pixels, and among thousands of arbitrary mixtures there are enough useful ones for the output layer to choose from. Networks built this way are called *random-feature models*; the idea appears in the random vector functional-link networks of Yoh-Han Pao and colleagues in the early 1990s, was put on a firm mathematical

footing by Ali Rahimi and Benjamin Recht in 2007, and is also known by the name Guang-Bin Huang and colleagues gave it, *extreme learning machines*.

Compare that with the trained network of Chapter 12: two hidden layers of 256 neurons, every weight adjusted by gradient descent, 98.1 percent on the same test set. Training buys accuracy, and it buys economy. The trained network needed an eighth as many hidden neurons (512 against 4,000) to make fewer than two-thirds as many mistakes. But a random layer gets surprisingly far.

17.4 Excursion: too many features

This section and the precision lesson inside it are a closer look at the ridge; nothing later depends on them, and you can skip to *Random kernels*. Now the ridge. Here is what happens when the number of training images changes while the 4,000 random features stay the same. For each size we solve twice: with essentially no ridge ($\lambda = 10^{-8}$), reporting training and test accuracy and the size of the solved weights (their length, $\sqrt{\sum W^2}$), and with the ridge that does best on 10,000 *validation* images, never on the test set:

rf_threshold

```
# H and W1 are still the 4,000-neuron layer from the last pass above
H_test = random_features(X_test, W1)
H_val, Y_val = H[50_000:], Y[50_000:]       # for choosing the ridge

def best_ridge(H, Y):
    scores = {r: readout_accuracy(H_val, solve(H, Y, r), Y_val)
              for r in [0.01, 1, 10, 100]}
    return max(scores, key=scores.get)

print('images  train  test   weight size | ridge  test')
for n in [2000, 3000, 3800, 4000, 4200, 5000, 8000]:
    W2 = solve(H[:n], Y[:n], 1e-8)                # essentially no ridge
    train_acc = readout_accuracy(H[:n], W2, Y[:n])
    plain = readout_accuracy(H_test, W2, Y_test)
    r = best_ridge(H[:n], Y[:n])
    tuned = readout_accuracy(H_test, solve(H[:n], Y[:n], r), Y_test)
    print(f'{n:6d}  {train_acc:.3f}  {plain:.3f} {W2.norm():9.0f}   |'
          f' {r:<5g}  {tuned:.3f}')
```

OUTPUT

```
images  train  test   weight size | ridge  test
  2000  1.000  0.900         9   | 10     0.929
  3000  1.000  0.855        16   | 10     0.937
  3800  1.000  0.590        41   | 10     0.942
  4000  1.000  0.109      1573   | 10     0.943
  4200  1.000  0.595        47   | 10     0.945
  5000  1.000  0.855        22   | 10     0.948
  8000  1.000  0.940        13   | 10     0.955
```

With no ridge, every row classifies all of its training images correctly. With 2,000 images the test accuracy is still 90 percent: with fewer equations than unknowns, the solver picks the solution with the smallest weights, which is about as tame as an exact fit can be. But as

the number of images approaches the number of features, the weights grow, and at 4,000 images, exactly as many as there are features, they are more than a hundred times larger than at 2,000 and accuracy collapses to 11 percent, no better than guessing. With as many equations as unknowns there is exactly one exact fit, and to pass through every one of 4,000 noisy targets it needs enormous weights. Add more images and both recover. Is this a fluke of one random layer? Three other random layers, at exactly 4,000 images:

rf_threshold_seeds

```
for seed in [1, 2, 3]:                    # three other random layers
    torch.manual_seed(seed)
    W1_s = torch.randn(784, 4000, device=device) / 784**0.5
    Hs = random_features(X[:4000], W1_s)
    Hs_test = random_features(X_test, W1_s)
    W2 = solve(Hs, Y[:4000], 1e-8)
    print(seed, round(readout_accuracy(Hs_test, W2, Y_test), 3))
```

OUTPUT

```
1 0.152
2 0.124
3 0.121
```

All three collapse. A ridge of 10 avoids the whole problem; it never needs to fit the training set exactly. This is *regularization*: a preference for simple answers, which matters most when the data are only just enough. The dip at the point where images and features are equal has a name, *double descent*, and it has been found in large networks too.

There is a practical lesson hiding in `solve`. It converts the features to 64-bit numbers *before* multiplying them. Here is the same 2,000-image solve done the other way round, with the products added up in the 32-bit numbers that graphics cards prefer and converted afterwards:

rf_precision

```
def solve32(H, Y, ridge):                    # adds up in 32-bit numbers
    T = F.one_hot(Y, 10).float()
    G = (H.T @ H).cpu().double()             # converted too late
    G += ridge * torch.eye(H.shape[1], dtype=torch.float64)
    W = torch.linalg.solve(G, (H.T @ T).cpu().double())
    return W.float().to(H.device)

for name, f in [('32-bit', solve32), ('64-bit', solve)]:
    W2 = f(H[:2000], Y[:2000], 1e-4)
    print(name, round(readout_accuracy(H_test, W2, Y_test), 3))
```

OUTPUT

```
32-bit 0.14
64-bit 0.9
```

Same formula, same data: 14 percent instead of 90. A 32-bit number keeps about seven significant digits. The sums in $H^\mathsf{T}H$ are large, and their rounding errors swamp the tiny ridge of 10^{-4}, which changes the answer completely. The deeper cause is that multiplying H by its own transpose squares how sensitive the problem is to rounding, so there are three cures: 64-bit numbers, a ridge large enough to stand above the rounding errors, or a solver such as `torch.linalg.lstsq` that never forms $H^\mathsf{T}H$ at all. When results are mysteriously bad, check the precision as well as the code.

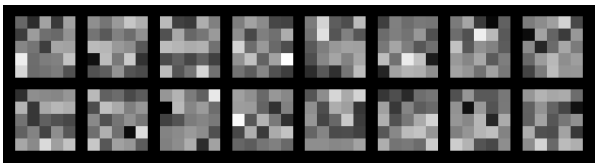


Figure 17.2: Sixteen of the 64 random 5×5 kernels. Gray is zero, white positive and black negative.

17.5 Random kernels

The random layer above ignores the shape of the image, as every fully connected layer does. What if the random features are *convolutions*? We make 64 random 5×5 kernels, each shifted to average zero, so that it responds to changes in brightness rather than to brightness itself, and scaled to the same length. Figure 17.2 shows 16 of them, drawn with the `montage_plot` helper of Chapter 8:

rf_kernels

```
torch.manual_seed(0)
kernels = torch.randn(64, 1, 5, 5, device=device)
kernels -= kernels.mean(dim=(2, 3), keepdim=True)   # average zero
kernels /= kernels.norm(dim=(2, 3), keepdim=True)   # all the same size
shown = kernels[:16, 0].cpu()
montage_plot(shown, size=4.0, grid_shape=(2, 8), symmetric=True)
```

Each image is convolved with all 64 kernels, passed through a ReLU, and max-pooled down to 7×7: the same kinds of operations as the network of Chapter 14, though it had two convolutional layers and this has one, and here nothing will be trained. That gives $64 \times 7 \times 7 = 3{,}136$ features per image, and we solve for the output layer as before:

rf_conv_features

```
def conv_features(X, batch=5000):
    out = []
    for i in range(0, len(X), batch):
        x = X[i:i + batch].reshape(-1, 1, 28, 28)
        z = torch.relu(F.conv2d(x, kernels, padding=2))  # (n,64,28,28)
        z = F.max_pool2d(z, 4)                           # (n,64,7,7)
        out.append(z.flatten(1))
    return torch.cat(out)

H = conv_features(X)
H_test = conv_features(X_test)
print(H.shape)
W2 = solve(H, Y)
print('test', round(readout_accuracy(H_test, W2, Y_test), 4))
```

OUTPUT

```
torch.Size([60000, 3136])
test 0.9764
```

Random convolutions do better than 4,000 random neurons with fewer features, 97.6 percent against 96.8. The structure of a convolution, local and shared across the image, is worth something even when its weights are nonsense. Now clothing:

rf_fashion

```
Xf, Yf, Xf_test, Yf_test = load(FashionMNIST)
W = solve(Xf, Yf)
print('pixels         ', round(readout_accuracy(Xf_test, W, Yf_test), 4))
Hf, Hf_test = conv_features(Xf), conv_features(Xf_test)
W2 = solve(Hf, Yf)
print('random kernels ', round(readout_accuracy(Hf_test, W2, Yf_test), 4))
```

OUTPUT

```
pixels          0.8086
random kernels  0.8936
```

Solving on the raw pixels gives 81 percent; random kernels give 89. The *trained* two-layer network of Chapter 14 reached 90.9 percent on the same test set, in a single untuned run. On small grayscale images, then, a single layer of random convolutions and a solved readout get within a point and a half of a network that was trained throughout. Counted as mistakes, that is 10.6 percent against 9.1, so the trained network still makes about a seventh fewer errors, from one run each and with no tuning on either side. Using random convolutions as features has a history of its own: Kevin Jarrett and colleagues in 2009

and Andrew Saxe and colleagues in 2011 found that random-weight convolutional networks did surprisingly well, and that the architecture accounted for much of it.

17.6 Random features on photographs

Does the same hold for real photographs? We can test it on the ants and bees of the last chapter, with the setup of that chapter: AlexNet with every layer frozen except the last. Instead of training the last layer, we solve for it, choosing the ridge on the same 50 validation photographs as before. The only difference between the runs is whether the frozen weights are AlexNet's trained ones or random ones.

rf_photos_data

```
from torchvision import transforms
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader

tf = transforms.Compose([
    transforms.Resize(256), transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])
train_set = ImageFolder('data/hymenoptera_data/train', tf)
test_set = ImageFolder('data/hymenoptera_data/val', tf)

def last_layer_inputs(net, dataset):    # what Chapter 16's new layer saw
    net.classifier[6] = torch.nn.Identity()
    net = net.to(device).eval()
    H, Y = [], []
    with torch.no_grad():
        for x, y in DataLoader(dataset, batch_size=64):
            H.append(net(x.to(device)))
            Y.append(y.to(device))
    return torch.cat(H), torch.cat(Y)
```

rf_photos

```
from torchvision.models import alexnet, AlexNet_Weights

g = torch.Generator().manual_seed(0)
order = torch.randperm(len(train_set), generator=g)
val, fit_on = order[:50], order[50:]        # as in Chapter 16

def photo_test(net):
    H, Y = last_layer_inputs(net, train_set)
    H_t, Y_t = last_layer_inputs(net, test_set)
    def val_score(r):
        W = solve(H[fit_on], Y[fit_on], r)
        return readout_accuracy(H[val], W, Y[val])
    scores = {r: val_score(r) for r in [1e-6, 1e-4, 1e-2, 1]}
    r = max(scores, key=scores.get)
    return readout_accuracy(H_t, solve(H[fit_on], Y[fit_on], r), Y_t)

trained = alexnet(weights=AlexNet_Weights.DEFAULT)
print('pretrained', round(photo_test(trained), 3))
for seed in range(3):
    torch.manual_seed(seed)
    print('random    ', round(photo_test(alexnet()), 3))
```

OUTPUT

```
pretrained 0.895
random     0.667
random     0.693
random     0.627
```

The trained features give 89.5 percent on the test photographs, in line with the 86.9 percent of the trained last layer in Chapter 16. Three different random AlexNets give 63 to 69 percent. That is far better than the 46 percent of the network trained from scratch in the last chapter, which never learned anything, so random features carry real information about photographs. But here, unlike on digits, the learned features are worth twenty to thirty points. Photographs are far more varied than small grayscale digits, and the patterns that separate an ant from a bee are not found by a random AlexNet as built here. Remember, though, what the last chapter found: ants and bees are among AlexNet's own thousand categories, so its trained features were made for exactly this question, and the gap overstates what learned features are worth on a category that is truly new. For that, try the comparison on photographs of your own.

17.7 Excursion: a random memory

Random features work in time, too. This section moves from images to signals, and nothing later depends on it. Here the task is to learn a signal well enough to continue it on our own. We use three signals (Figure 17.3). The first is three waves added together. The second is the swing of a *Van der Pol oscillator*, a simple nonlinear rule first used to model electronic circuits and heartbeats, which produces a lopsided, spiky rhythm. The third is the *Mackey–Glass* equation, which Michael Mackey and Leon Glass proposed in 1977 as a model of the production of blood cells: the rate of production depends on the level of cells some time ago, and that delay makes it *chaotic*. It never repeats, and, like the tripling recipe of Chapter 9, it turns a tiny difference now into a large one later. The code simulates the first two in tiny steps, exactly as Chapter 13 simulated a drumhead, and the third the same way, keeping a list of its past to look back into.

rf_signal

```
t = np.arange(5000) * 0.05
waves = np.sin(t) + 0.5 * np.sin(2 * t + 1) * np.cos(0.5 * t)

def van_der_pol(mu=3.0, dt=0.05, n=5000):
    x, v, out = 1.0, 0.0, []
    for i in range(n + 2000):                  # skip 2,000 to settle
        for _ in range(10):                    # ten small steps per sample
            a = mu * (1 - x**2) * v - x        # the oscillator's rule
            x, v = x + dt / 10 * v, v + dt / 10 * a
        out.append(x)
    out = np.array(out[2000:])
    return out / np.abs(out).max()

def mackey_glass(delay=17, n=5000):
    x, out = [1.2] * (10 * delay + 1), []      # a steady past to start
    for i in range(n + 1000):                  # skip 1,000 to settle
        for _ in range(10):                    # ten small steps per sample
            old = x[-10 * delay - 1]           # the level 17 samples ago
            x.append(x[-1] + 0.1 * (0.2 * old / (1 + old**10)
                                    - 0.1 * x[-1]))
        out.append(x[-1])
    out = np.array(out[1000:])
    out = out - out.mean()
    return out / np.abs(out).max()

signal = van_der_pol()
chaos = mackey_glass()
plt.figure(figsize=(4.6, 2.8))
for i, (s, name) in enumerate([(waves, 'three waves'),
                               (signal, 'Van der Pol oscillator'),
                               (chaos, 'Mackey-Glass (chaotic)')]):
    plt.subplot(3, 1, i + 1)
    plt.plot(s[:600])
    plt.title(name)
    plt.axis('off')
plt.show()
```

Before anything clever, a control. The simplest way to continue a signal is a *linear* rule: predict the next value as a weighted sum of the last few, choose the weights by least squares, then feed the predictions back in and let the rule run on its own for 400 steps. We give it 6, 30, 100 and 500 past values to work with:

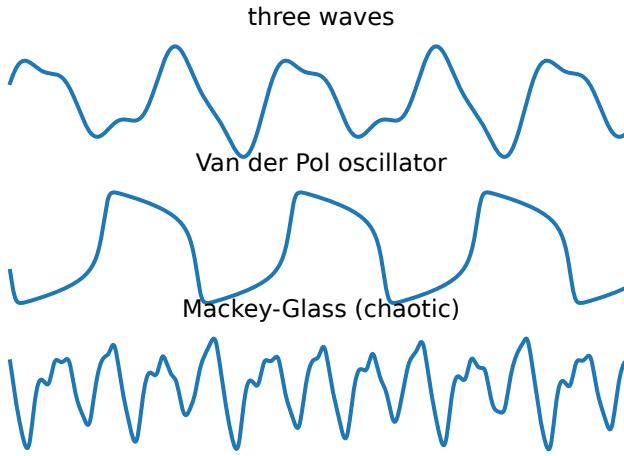


Figure 17.3: The three signals: three waves (top), a Van der Pol oscillator (middle), and the chaotic Mackey–Glass equation (bottom).

rf_linear

```
def continue_linear(s, order=6, train_len=3000, steps=400):
    rows = np.array([s[i - order:i] for i in range(order, train_len)])
    c = np.linalg.lstsq(rows, s[order:train_len], rcond=None)[0]
    history = list(s[train_len - order:train_len])
    for i in range(steps):                    # run on its own
        history.append(np.dot(c, history[-order:]))
    future = np.array(history[order:])
    return np.abs(future - s[train_len:train_len + steps]).mean()

for name, s in [('three waves', waves), ('Van der Pol', signal),
                ('Mackey-Glass', chaos)]:
    print(f'{name:12s}', [f'{continue_linear(s, k):.1e}'
                          for k in (6, 30, 100, 500)])
```

OUTPUT

```
three waves  ['1.7e-07', '1.0e-11', '2.1e-14', '6.1e-15']
Van der Pol  ['6.4e-01', '5.4e-01', '4.4e-10', '4.4e-13']
Mackey-Glass ['3.1e-01', '2.7e-01', '2.5e-01', '2.2e-01']
```

Six numbers continue the three waves almost perfectly, to seven decimal places, and better as the rule gets longer. That is no accident: a sum of three waves obeys an exact linear rule of length six, and least squares finds it. The oscillator fails with 6 or 30 numbers but is continued almost perfectly with 100. Its rhythm repeats about every 180 samples, and a rule that can see far enough back can simply copy

the shape of the last cycle; any signal that repeats can be continued that way. The chaotic signal is another matter. Even with 500 past values the error is about a fifth of the signal's size: it never repeats, so there is no cycle to copy, and its rule is nonlinear. That is the signal worth giving to a network.

The network for this job is a *reservoir*: a few hundred neurons connected to each other at random, with the incoming signal fed to all of them. At each time step, every neuron's new value is a tanh of the input and of all the neurons' old values. Because they feed into one another, the current state is a mixture of everything the signal has done lately: an echo of the past. (Herbert Jaeger's *echo state networks* of 2001 and Wolfgang Maass's *liquid state machines* of 2002 are the two original versions of the idea, and Mackey–Glass has been their favorite test from the start.)

How long the echo lasts depends on how strongly the neurons pass activity around the loop. Here is an experiment: give 500 randomly connected neurons a kick, then no input at all, and watch the size of their activity at three different strengths of the connections:

rf_echo

```
r = np.random.default_rng(0)
n = 500
W_raw = r.normal(size=(n, n))
radius = np.max(np.abs(np.linalg.eigvals(W_raw)))

for strength in [0.5, 0.95, 1.2]:
    W_res = W_raw * strength / radius
    state = np.tanh(r.uniform(-1, 1, n))    # a kick, then no input
    sizes = []
    for step in range(201):
        if step in (0, 10, 50, 100, 200):
            sizes.append(f'{np.sqrt(np.mean(state**2)):.4f}')
        state = np.tanh(W_res @ state)
    print(strength, sizes)
```

OUTPUT

```
0.5 ['0.4846', '0.0003', '0.0000', '0.0000', '0.0000']
0.95 ['0.4909', '0.1293', '0.0038', '0.0001', '0.0000']
1.2 ['0.5011', '0.3806', '0.3918', '0.4259', '0.4342']
```

At strength 0.5 the kick is gone within ten steps, too quickly to remember anything. At 1.2 it never dies: the network keeps itself going and forgets the input in its own chatter. At 0.95 the echo lingers

for dozens of steps and then fades, which is what a memory should do. The strength is measured by a number called the *spectral radius* of the connection matrix. You do not need its definition (it is the largest size among the matrix's eigenvalues) to use it: dividing a random matrix by its spectral radius and multiplying by 0.95 puts it just below the edge, as `W_raw * strength / radius` did above.

Now the reservoir. Its weights are random and are never trained. Only the readout, a weighted sum of the 500 neurons that should equal the next value of the signal, is solved for, exactly as before. The function `fit_readout` runs the training signal through the reservoir, collects its states, and solves; `step` takes the bend `f` as an argument, so that we can later swap it out.

rf_reservoir

```
r = np.random.default_rng(1)
W_in = r.uniform(-0.5, 0.5, n)                  # all three are random
b = r.uniform(-0.2, 0.2, n)                     # and never trained
W_res = W_raw * 0.95 / radius                   # just below the edge

def step(u, state, f=np.tanh):
    return f(W_in * u + W_res @ state + b)

def fit_readout(s, f=np.tanh, train_len=3000):
    state, states = np.zeros(n), []
    for u in s[:train_len]:
        state = step(u, state, f)
        states.append(state)
    S = np.array(states[100:])                  # skip the warm-up
    targets = s[101:train_len + 1]              # the value that came next
    w_out = np.linalg.solve(S.T @ S + 1e-9 * np.eye(n), S.T @ targets)
    print(f'one-step error on the training signal '
          f'{np.abs(S @ w_out - targets).mean():.1e}')
    return w_out, state

w_out, state = fit_readout(chaos)
```

OUTPUT

```
one-step error on the training signal 5.5e-06
```

On the training signal the readout predicts the next value almost perfectly. The real test is to let the reservoir run on its own: feed each prediction back in as the next input and see whether it can continue the chaotic signal with no further help (Figure 17.4).

rf_generate

```
def run_on_its_own(w_out, state, u, f=np.tanh, steps=400):
    generated = []
    for i in range(steps):
        state = step(u, state, f)
        u = state @ w_out                    # its own prediction
        generated.append(u)                  # becomes the next input
    return np.array(generated)

generated = run_on_its_own(w_out, state, chaos[3000])
truth = chaos[3001:3401]
print(f'average error {np.abs(truth - generated).mean():.1e}')
plt.figure(figsize=(4.6, 1.6))
plt.plot(truth, 'k', lw=5, alpha=0.25, label='true signal')
plt.plot(generated, color='#f97306', lw=1,
         label='reservoir, running on its own')
plt.legend(frameon=False, loc='lower center', ncol=2,
           bbox_to_anchor=(0.5, 1.0))
plt.xlabel('steps from the end of training')
plt.show()
```

OUTPUT

```
average error 1.3e-03
```

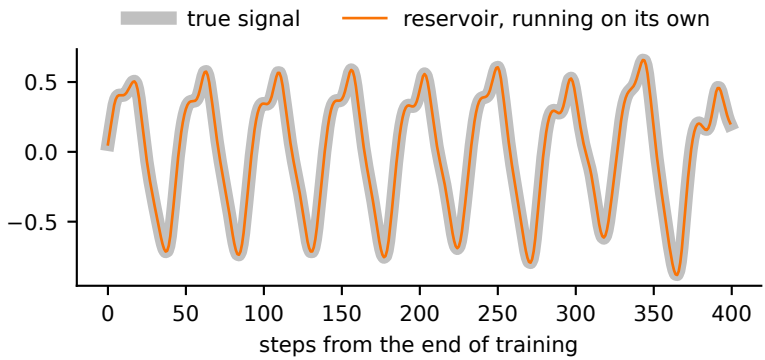


Figure 17.4: After training, the reservoir is fed its own predictions. For 400 steps it continues the chaotic Mackey–Glass signal so closely that the two curves can hardly be told apart.

Where the best linear rule, with as many weights as the readout, went wrong by a fifth of the signal's size, the random reservoir stays within about a thousandth of it for 400 steps. Is that the reservoir's memory, or its bends? The reservoir has the same 500 numbers to

work with as the longest linear rule. A fair control keeps everything else, the same random connections, the same readout and the same signal, and takes away only the tanh:

rf_no_tanh

```
def no_bend(z):                                  # tanh removed
    return z

w_flat, state_flat = fit_readout(chaos, no_bend)
flat = run_on_its_own(w_flat, state_flat, chaos[3000], no_bend)
print(f'average error {np.abs(truth - flat).mean():.1e}')
```

OUTPUT

```
one-step error on the training signal 1.6e-04
average error 2.6e-01
```

Without its bends the reservoir is no better than the linear rule, because it *is* a linear rule: every neuron is then a weighted sum of past inputs. Memory alone continued the repeating oscillator; the chaotic signal needs memory and nonlinearity together, and the random tanh neurons give the readout enough raw material to express its rule.

Be warned that the trick has limits. No predictor can follow a chaotic signal forever, because its own tiny errors grow just as the signal's differences do; run `rf_generate` for 1,500 steps and the reservoir eventually parts company with the truth. And with a much stronger ridge or a spectral radius far from 1, it drifts or settles into a rhythm of its own; the exercises invite you to find out when. The recurrent networks used for language and speech before the transformer had this same looping shape, but with every weight trained, at great effort.

17.8 How much must be learned?

Random features turn the usual question around. Instead of asking how to train every weight, they ask how few weights need training at all. The answer depends on the data. On small clothing images, a random layer of the right shape and a solved readout come within a point or two of a fully trained network, though the trained one still makes fewer mistakes. On photographs, learned features are worth twenty or thirty points, and that is where training, and borrowing someone else's training, pays. Two things hold everywhere: the shape of a network, convolutions for images and loops for signals, is worth a great deal even before any weight is learned; and training makes

features far more compact, since the trained networks needed a fraction of the units.

So far every network in this book has answered questions about images: what digit, what garment, ant or bee. In the next part we turn the question around. Instead of asking what is in an image, we will ask a network to *make* one.

Try it

1. In `rf_curve`, use 60 random neurons but replace `pinv` with a ridge solution, `np.linalg.solve(H.T @ H + 0.1 * np.eye(h), H.T @ y)`. What happens to the wild swings?

2. In `rf_elm`, replace the ReLU with `torch.tanh` and then with no nonlinearity at all. Explain the result without a nonlinearity. (Chapter 12 has the answer.)

3. In `rf_kernels`, try 16 and then 256 random kernels. With 256, change `max_pool2d(z, 4)` in `rf_conv_features` to `max_pool2d(z, 7)`, which leaves 4×4 positions per kernel ($256 \times 16 = 4{,}096$ features). How does accuracy depend on the number of kernels?

4. Do trained kernels make better features than random ones when only the last layer is solved? At the end of Chapter 14's code, save the trained first-layer kernels with `torch.save(w[0].detach().cpu(), f)`, where `f` is a file name such as `'data/k.pt'`. (`detach` drops their link to the gradient calculations, which would otherwise follow them and fill your memory.) Here, load them with `kernels = torch.load(f).to(device)`, use `padding=1` instead of 2 (they are 3×3), and compare on Fashion-MNIST.

5. In `rf_reservoir`, change 0.95 to 0.5 and to 1.2, and the ridge from `1e-9` to `1e-4`. When does the self-fed reservoir in `rf_generate` stay on track?

6. Run the reservoir on its own for 1,500 steps (`steps=1500`, and compare with `chaos[3001:4501]`). At which step does it first miss by more than 0.1? Change the reservoir's random seed: does the answer change much?

7. Give the reservoir the Van der Pol oscillator, `fit_readout(signal)`. Does it beat the linear rule of length 100, which copies the last cycle?

Part V

Machines That Imagine

Chapter 18

Autoencoders

Every network so far has needed answers. Each training image came with a label, and the loss measured how far the network's guess was from it. Labels are expensive: someone has to sit down and write "ankle boot" sixty thousand times. Yet a child learns most of what it knows about the visual world without anyone labeling anything. Can a network learn something useful from images alone?

One beautifully simple answer is the *autoencoder*. Ask a network to reproduce its input, but force the information through a narrow bottleneck on the way. To pass a 28×28 image through 32 numbers and rebuild it on the other side, the network has to discover what matters about the images: which details can be dropped and which patterns recur. The "label" is the image itself. This is our first step from networks that *recognize* toward networks that *generate*.

18.1 A linear autoencoder

We use Fashion-MNIST again, kept as images of shape (1, 28, 28) with values between 0 and 1, and hold back the last 10,000 training images for validation. We load the labels too, but the autoencoder will never look at them; they come out only at the end of the chapter, to test what it learned. `get_batch` draws a random batch of images (without labels, so it takes one argument fewer than the version of Chapter 11), and the `montage_plot` helper from Chapter 8 draws grids of them.

ae_load

```
import torch
import torch.nn.functional as F
from torchvision.datasets import FashionMNIST
torch.manual_seed(0)

device = ('cuda' if torch.cuda.is_available()
          else 'mps' if torch.backends.mps.is_available() else 'cpu')

def load(train):
    ds = FashionMNIST('data', train=train, download=True)
    X = ds.data[:, None].float() / 255.0      # add a channel axis
    return X.to(device), ds.targets.to(device)

X, Y = load(train=True)             # (60000, 1, 28, 28), values 0-1
X_test, Y_test = load(train=False)
X, X_val = X[:50000], X[50000:]     # 10,000 held out for validation
Y, Y_val = Y[:50000], Y[50000:]

def get_batch(X, batch_size):
    return X[torch.randint(len(X), (batch_size,), device=X.device)]

print(tuple(X.shape), len(X_val), len(X_test))
```

OUTPUT

```
(50000, 1, 28, 28) 10000 10000
```

The simplest autoencoder is a single weight matrix used twice. The *encoder* multiplies the 784 pixels by a 784×32 matrix to get a 32-number *code*; the *decoder* multiplies the code by the same matrix transposed to get 784 numbers back. The loss is the mean squared difference between the input and its reconstruction, and every thousand steps we print it for the current batch and for the validation images.

ae_linear

```
def encode(x, W):
    return x.reshape(len(x), 784) @ W          # 784 numbers -> 32

def decode(code, W):
    return (code @ W.T).reshape(-1, 1, 28, 28)  # 32 numbers -> 784

def error(x, y):                                # mean squared error
    return ((x - y) ** 2).mean()

W = (0.01 * torch.randn(784, 32, device=device)).requires_grad_()
optimizer = torch.optim.Adam([W], lr=0.001)

for step in range(5001):
    x = get_batch(X, 128)
    loss = error(decode(encode(x, W), W), x)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if step % 1000 == 0:
        with torch.no_grad():
            val = error(decode(encode(X_val, W), W), X_val)
        print(step, round(loss.item(), 4), round(val.item(), 4))
```

OUTPUT

```
0 0.2047 0.1993
1000 0.016 0.0159
2000 0.0148 0.0155
3000 0.0156 0.0155
4000 0.0146 0.0154
5000 0.0153 0.0154
```

Each image is compressed $784/32 \approx 25$ times. How much survives? The average squared error over all 10,000 test images is one number; the first sixteen test images, each followed by its reconstruction, show what it means (Figure 18.1):

ae_linear_test

```
def compare(originals, reconstructions):
    pairs = torch.stack([originals, reconstructions], dim=1)
    montage_plot(pairs.reshape(-1, 28, 28).cpu().numpy(),
                 grid_shape=(len(originals) // 4, 8), size=4.6)

with torch.no_grad():
    x_linear = decode(encode(X_test, W), W)
    print(round(error(x_linear, X_test).item(), 4))
compare(X_test[:16], x_linear[:16])
```

OUTPUT

```
0.0154
```

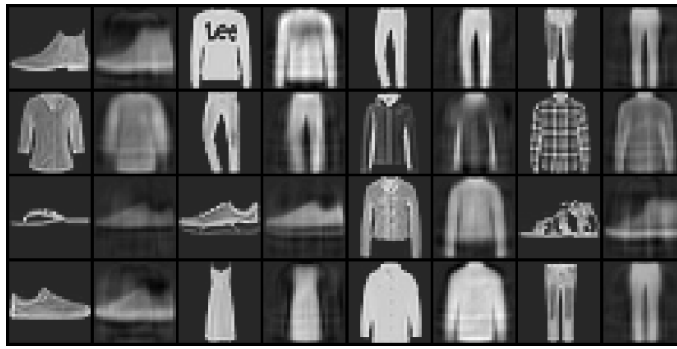


Figure 18.1: Test images (odd columns) and their reconstructions from 32 numbers by the linear autoencoder (even columns).

The overall shapes survive, a boot, a shirt, a pair of trousers, but the details blur away: the printed logo and the plaid pattern are smeared. The 32 columns of the weight matrix show what the encoder measures (Figure 18.2, with zero drawn as middle gray):

ae_linear_weights

```
weights = W.T.reshape(32, 28, 28).detach().cpu().numpy()
montage_plot(weights, grid_shape=(4, 8), symmetric=True)
```

These are broad, overlapping patterns, ghostly blends of shirt, trouser and shoe outlines. Each image is rebuilt as a weighted mixture of these 32 patterns, so whatever they cannot express is lost. Mathematically, the best a linear autoencoder can do is capture the same 32-dimensional family of images that the classical statistical technique called *principal component analysis* finds. (Strictly, the same 32-dimensional subspace, not the same 32 patterns, and without first

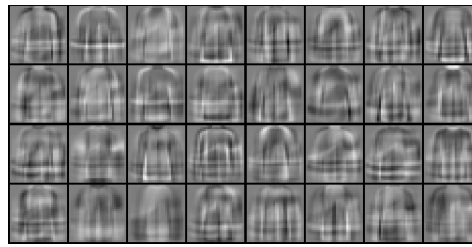


Figure 18.2: The 32 columns of the linear autoencoder's weight matrix. Middle gray is zero; lighter is positive, darker negative.

subtracting the average image, which the classical method does.) The rest of this paragraph is for readers who know some linear algebra; the numbers that matter come after the listing. We can check that claim. The best 32 directions can be computed directly with a *singular value decomposition*, `torch.linalg.svd`, one of the workhorses of numerical linear algebra; then we measure how well the autoencoder's 32 directions line up with them, where 1 means perfectly:

ae_pca

```
with torch.no_grad():
    data = X.reshape(len(X), 784).cpu()
    _, _, Vt = torch.linalg.svd(data, full_matrices=False)
    V = Vt[:32].T                          # the best 32 directions (PCA)
    Q, _ = torch.linalg.qr(W.cpu())        # the autoencoder's 32 directions
    overlap = torch.linalg.svdvals(Q.T @ V)   # 1 means the same direction
    t = X_test.reshape(len(X_test), 784).cpu()
    pca_error = ((t @ V @ V.T - t) ** 2).mean()
print(round(overlap.mean().item(), 3), round(overlap.min().item(), 3))
print(round(pca_error.item(), 4))
```

OUTPUT

```
0.975 0.498
0.0152
```

On average the two sets of directions agree to 0.975, and the best possible 32-dimensional reconstruction has an error of 0.0152, against the autoencoder's 0.0154. The weakest agreement, 0.50, shows that training had not quite finished: gradient descent settles the most important directions first, and the error barely depends on the last few. To do better we need the tools of Part IV.

18.2 Going up: transposed convolution

A convolutional encoder is easy: it is the front half of the network of Chapter 14, with stride-2 convolutions that halve the image size in place of pooling. The decoder has to go the other way, from a small stack of maps to a large image. The tool for this is the *transposed convolution*. Where an ordinary convolution gathers a patch of inputs into one output, a transposed convolution spreads each input value out over a patch of outputs, weighted by the kernel. The clearest way to see it is to feed it an input that is all zeros except for a single 1 (Figure 18.3):

ae_spread

```
x = torch.zeros(1, 1, 4, 4)
x[0, 0, 1, 1] = 1                                  # a single 1
kernel = torch.arange(1., 17).reshape(1, 1, 4, 4)
full = F.conv_transpose2d(x, kernel, stride=2)             # no crop
out = F.conv_transpose2d(x, kernel, stride=2, padding=1)   # cropped
print(full.shape, out.shape)

def show_grid(a, title):
    n = a.shape[-1]
    plt.imshow(a[0, 0], cmap='Greys', vmin=0, vmax=a.max())
    plt.xticks(np.arange(-0.5, n), [])
    plt.yticks(np.arange(-0.5, n), [])
    plt.tick_params(length=0)
    plt.grid(color='0.75', linewidth=0.4)
    plt.title(f'{title}\n{n}x{n}')

plt.figure(figsize=(4.6, 1.6))
for i, (a, title) in enumerate([(x, 'input'), (kernel, 'kernel'),
                                (full, 'padding=0'), (out, 'padding=1')]):
    plt.subplot(1, 4, i + 1)
    show_grid(a, title)
    if i == 2:                                     # the part padding=1 keeps
        plt.gca().add_patch(plt.Rectangle((0.5, 0.5), 8, 8, fill=False,
                                          ec='tab:red', lw=1.5, zorder=3))
plt.show()
```

OUTPUT

```
torch.Size([1, 1, 10, 10]) torch.Size([1, 1, 8, 8])
```

The 1 becomes a copy of the kernel, stamped into the output. Each input position stamps its own copy, scaled by its value, and the stride of 2 means that neighboring inputs stamp their copies two pixels apart

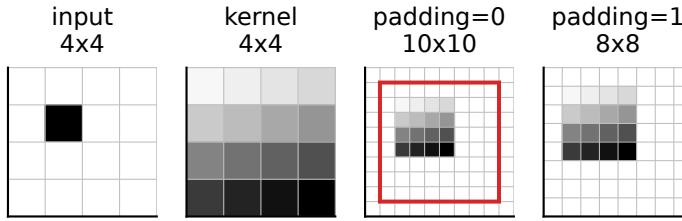


Figure 18.3: A transposed convolution with stride 2. The single 1 in the input is replaced by a copy of the 4×4 kernel in the output. With `padding=1`, the outer ring of the output (outside the red frame) is cropped away.

in the output. Because the kernel is 4 wide, neighboring copies overlap and add up, which blends them smoothly. With no padding, a 4×4 input gives a 10×10 output: $(4-1) \times 2$ steps plus the kernel's width of 4. Here `padding` means the opposite of what it means for an ordinary convolution: instead of adding a border, `padding=1` *crops* one pixel from every edge of the output, leaving 8×8, exactly twice the input. So a stride-2 transposed convolution with a 4×4 kernel and `padding=1` doubles the size of a stack of maps:

ae_upsample

```
small = torch.randn(1, 64, 7, 7)
kernel = torch.randn(64, 32, 4, 4)           # (in, out, height, width)
print(F.conv_transpose2d(small, kernel, stride=2, padding=1).shape)
```

OUTPUT

```
torch.Size([1, 32, 14, 14])
```

The weights of a transposed convolution are stored as (*input channels*, *output channels*, *height*, *width*), the reverse of an ordinary convolution.

18.3 A convolutional autoencoder

Three small helpers create the weights with the He scaling of Chapter 14: random numbers with standard deviation $\sqrt{2/n}$, where n is the *fan-in*, the number of inputs that feed each output: input channels times kernel area for a convolution. A transposed convolution with stride 2 is different. Neighboring copies of the kernel are stamped two pixels apart, so each output pixel receives contributions through only a quarter of the kernel's positions, and its fan-in is a quarter of that. To keep the code short, this network has no biases.

ae_conv_weights

```
def conv(n_in, n_out, k=4):                  # (out, in, k, k)
    std = (2 / (n_in * k * k)) ** 0.5
    W = torch.randn(n_out, n_in, k, k, device=device) * std
    return W.requires_grad_()

def conv_t(n_in, n_out, k=4):                # transposed: (in, out, k, k)
    fan_in = n_in * k * k // 4      # stride 2: each output gets 1/4
    std = (2 / fan_in) ** 0.5
    W = torch.randn(n_in, n_out, k, k, device=device) * std
    return W.requires_grad_()

def linear(n_in, n_out):                     # (out, in)
    std = (2 / n_in) ** 0.5
    W = torch.randn(n_out, n_in, device=device) * std
    return W.requires_grad_()

w = [conv(1, 32),              # encoder: 1 -> 32 channels, 28 -> 14
     conv(32, 64),             #          32 -> 64 channels, 14 -> 7
     linear(64 * 7 * 7, 32),   #          3136 -> 32 numbers
     linear(32, 64 * 7 * 7),   # decoder: 32 -> 3136 numbers
     conv_t(64, 32),           #          64 -> 32 channels, 7 -> 14
     conv_t(32, 1)]            #          32 -> 1 channel, 14 -> 28
```

The network, written as an `encoder` and a `decoder`. The final *sigmoid*, $1/(1+e^{-x})$, squashes the output into the range 0 to 1, the range of the pixels.

ae_conv_model

```
def encoder(x, w):
    x = F.relu(F.conv2d(x, w[0], stride=2, padding=1))
    x = F.relu(F.conv2d(x, w[1], stride=2, padding=1))
    return F.linear(x.flatten(start_dim=1), w[2])

def decoder(code, w):
    x = F.relu(F.linear(code, w[3])).reshape(-1, 64, 7, 7)
    x = F.relu(F.conv_transpose2d(x, w[4], stride=2, padding=1))
    return torch.sigmoid(F.conv_transpose2d(x, w[5], stride=2, padding=1))

def autoencoder(x, w):
    return decoder(encoder(x, w), w)

print(encoder(X[:5], w).shape, autoencoder(X[:5], w).shape)
```

OUTPUT

```
torch.Size([5, 32]) torch.Size([5, 1, 28, 28])
```

The code is still 32 numbers, so the comparison with the linear

autoencoder is fair. Training is the same as before:

ae_conv_train

```
optimizer = torch.optim.Adam(w, lr=0.001)

for step in range(5001):
    x = get_batch(X, 128)
    loss = error(autoencoder(x, w), x)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if step % 1000 == 0:
        with torch.no_grad():
            val = error(autoencoder(X_val, w), X_val)
        print(step, round(loss.item(), 4), round(val.item(), 4))
```

OUTPUT

```
0 0.2061 0.1812
1000 0.0129 0.0136
2000 0.012 0.0114
3000 0.0095 0.0103
4000 0.0102 0.0099
5000 0.0086 0.0096
```

ae_conv_test

```
with torch.no_grad():
    x_conv = autoencoder(X_test, w)
    print(round(error(x_conv, X_test).item(), 4))
compare(X_test[:16], x_conv[:16])
```

OUTPUT

```
0.0095
```

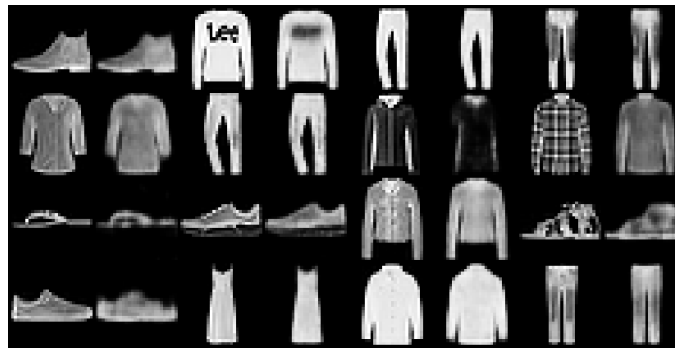


Figure 18.4: The same test images, reconstructed by the convolutional autoencoder from 32 numbers.

Over the whole test set the reconstruction error is 38 percent lower than the linear autoencoder's, and you can see it in Figure 18.4: the gray haze around each garment is gone, edges are sharper and the sneakers have soles. Fine detail is still lost; the logo and the plaid have vanished altogether. Thirty-two numbers are not enough to record them, and the network spends its budget on the shapes that all clothing shares.

18.4 What the codes are good for

The autoencoder never saw a label. Did it learn anything a classifier could use? A quick test is to take a handful of labeled images, turn each into its 32-number code, and train the simplest possible classifier, a single linear layer as in Chapter 11, on the codes. For comparison we train the same classifier on the raw 784 pixels. The function `probe` does both, with the first `n` training images as the only labeled data, and reports accuracy on the validation set and then, once, on the test set:

ae_probe

```
def probe(features, n):         # a linear classifier trained on n labels
    with torch.no_grad():
        f = features(X[:n])
        f_val, f_test = features(X_val), features(X_test)
    V = torch.zeros(f.shape[1], 10, device=device, requires_grad=True)
    b = torch.zeros(10, device=device, requires_grad=True)
    optimizer = torch.optim.Adam([V, b], lr=0.01)
    for step in range(500):
        loss = F.cross_entropy(f @ V + b, Y[:n])
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    with torch.no_grad():
        val = ((f_val @ V + b).argmax(dim=1) == Y_val).float().mean()
        test = ((f_test @ V + b).argmax(dim=1) == Y_test).float().mean()
    return round(val.item(), 3), round(test.item(), 3)

pixels = lambda x: x.flatten(start_dim=1)       # 784 numbers
encoded = lambda x: encoder(x, w)               # 32 numbers
for n in [100, 1000]:
    print(n, 'labels, pixels:', probe(pixels, n),
          ' codes:', probe(encoded, n))
```

OUTPUT

```
100 labels, pixels: (0.695, 0.685)  codes: (0.716, 0.706)
1000 labels, pixels: (0.797, 0.783)  codes: (0.793, 0.784)
```

With only 100 labels, the 32 numbers learned without labels do about as well as the 784 pixels they came from: 71 percent against 69 on the test set, a difference within the noise of a single draw of 100 labels. With 1,000 labels the two are level. The codes are 25 times smaller and, at these numbers of labels, gave up nothing a linear classifier could use. (With tens of thousands of labels we would expect the pixels to pull ahead; the exercises let you check.) This is the idea behind

self-supervised learning: learn from plentiful unlabeled data first, then use a few labels to put the result to work. In Chapter 16 we borrowed features learned from a million labeled photographs; modern systems learn their features mostly from data nobody labeled at all.

18.5 The space of codes

Every image now corresponds to a point in a 32-dimensional *latent space*, and the decoder can turn *any* point in that space into an image, not just the points that came from real images. What lies between two real images? Here we encode an ankle boot and a pair of trousers, then decode points along the straight line between their codes:

ae_blend

```
with torch.no_grad():
    a, b = encoder(X_test[[0, 2]], w)       # an ankle boot and a trouser
    steps = torch.linspace(0, 1, 8, device=device)[:, None]
    blend = decoder((1 - steps) * a + steps * b, w)
montage_plot(blend[:, 0].cpu().numpy(), grid_shape=(1, 8), size=4.6)
```

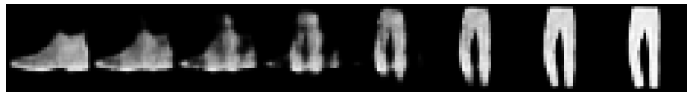


Figure 18.5: Decoding points on the line between the code of a boot (left) and the code of a pair of trousers (right).

The boot does not simply fade out while the trousers fade in, as it would if we blended the pixels. The shape itself deforms through a series of strange in-between garments. The network has built its own space of clothing, and we can walk around in it.

So can we make new clothing by picking random codes? Each of the 32 numbers in a code has its own typical value and spread, so we measure the average and standard deviation of each one over the validation images, and draw random codes to match:

ae_random

```
with torch.no_grad():
    codes = encoder(X_val, w)               # (10000, 32)
    mean, std = codes.mean(dim=0), codes.std(dim=0)  # one per dimension
    print(std[:8].cpu())
    random_codes = mean + std * torch.randn(16, 32, device=device)
    fakes = decoder(random_codes, w)
montage_plot(fakes[:, 0].cpu().numpy(), grid_shape=(2, 8), size=4.6)
```

OUTPUT

```
tensor([1.62, 1.78, 1.15, 1.61, 1.27, 1.29, 1.61, 1.65])
```

The spreads printed for the first eight numbers range from about 1.2 to 1.8, which is why each number gets its own. Most of the results

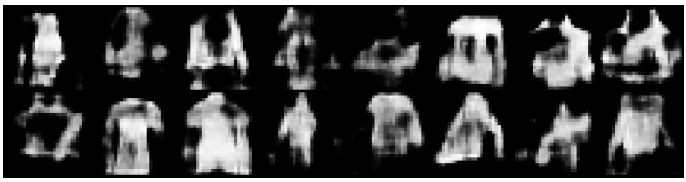


Figure 18.6: Decoding sixteen random codes.

in Figure 18.6 are recognizably tops, T-shirts and pullovers, and a few could be bags, but all of them are blurry and blotchy, and some are hard to name at all: a shirt melting into a bag, a garment no shop would sell. Not one is a shoe or a pair of trousers. The autoencoder was only ever asked to reconstruct real images. Nothing in its training asked that its latent space be easy to sample from, and it isn't. The codes of real images fill an irregular region of the 32-dimensional space, with gaps and bulges. Our random codes land roughly in the right neighborhood but often between the places real codes go, where the decoder has never been trained and can only guess.

18.6 Variational autoencoders

The principled fix is the *variational autoencoder*, or VAE, proposed in 2013. A VAE adds a second term to the loss that pushes the codes of the training images toward a simple, known distribution: each of the 32 numbers independently bell-shaped around zero with a spread of 1, the same distribution `torch.randn` draws from. The encoder also learns to make each image's code slightly fuzzy, so the decoder learns to turn a whole small cloud of codes, not one exact point, into a good image. The two pressures together leave few gaps. After training, a code drawn with `torch.randn` lands where the decoder has been trained, and comes out as a plausible image. The price is that reconstructions get a little blurrier still, because the codes are no longer free to sit wherever reconstruction alone would put them.

VAEs are more than a historical step. Image generators such as Stable Diffusion do not work on pixels directly. They first train a VAE to squeeze images into a much smaller latent space, do all of their generating inside that space, and use the VAE's decoder to turn the result into pixels. These are called *latent diffusion models*.

To get a network that turns *random* numbers directly into convincing images, we can also train for exactly that and nothing else. The trick for doing so is one of the cleverest ideas in deep learning: set two networks against each other.

Try it

1. Change the code size from 32 to 2 in both autoencoders (the `784, 32` matrix, and `linear(64 * 7 * 7, 32)` and `linear(32, 64 * 7 * 7)`). How much worse are the reconstructions? With two numbers per image you can plot the codes of the validation images as points with `plt.scatter`, colored by `Y_val`. Do the classes form separate clusters?

2. Run `probe` with `n` set to 30 and to 10,000. Where does the advantage of the codes appear, and where does it disappear?

3. Repeat the blend of Figure 18.5 between two sneakers (in the test set, class 7 sits at indices 9 and 12, among others) and between two ankle boots. Is the walk smoother within one kind of garment than across kinds?

4. In `ae_random`, multiply `std` by 0.5 and then by 2 before drawing the random codes. Which gives more convincing clothing, and why might staying near the middle of the codes help?

5. Add noise to the training inputs, but keep the clean images as the target: `loss = error(autoencoder(x + 0.2 * torch.randn_like(x), w), x)`. This *denoising* autoencoder learns to remove noise. Test it on noisy test images.

Chapter 19

Two Networks at Play

The autoencoder of the last chapter could rebuild a sneaker from 32 numbers, but it was never trained to *invent* one, and random codes came out as blurry, muddled garments. In 2014 Ian Goodfellow and his colleagues proposed a way to train a network whose whole job is to turn random numbers into realistic images. The idea is a contest. A *generator* network makes fake images from random noise. A *discriminator* network looks at an image and tries to say whether it is real, from the training set, or fake, from the generator. Each is trained to beat the other. The discriminator learns to spot fakes; the generator learns from the discriminator's verdicts how to make better ones. This is a *generative adversarial network*, or GAN.

A familiar analogy is a counterfeiter and a detective. Neither starts out any good. But every time the detective learns a new way to spot a forgery, the counterfeiter learns to fix that flaw, and the contest pushes both to get better.

19.1 One kind of thing

GANs are easiest to train on a narrow kind of image, so we take only the sneakers from Fashion-MNIST, class number 7. The generator will end with a `tanh`, whose outputs lie between −1 and 1, so we rescale the real images to that range as well. We also load the test set's sneakers, which the GAN never trains on; they come out only at the end, to check whether the generator is copying its training images.

gan_data

```
import torch
import torch.nn.functional as F
from torchvision.datasets import FashionMNIST
torch.manual_seed(0)

device = ('cuda' if torch.cuda.is_available()
          else 'mps' if torch.backends.mps.is_available() else 'cpu')

def load(train):                    # sneakers only, pixels in -1..1
    ds = FashionMNIST('data', train=train, download=True)
    X = ds.data[ds.targets == 7][:, None].float() / 255.0
    return (X * 2 - 1).to(device)

X = load(train=True)
X_test = load(train=False)          # only for a check at the very end
print(FashionMNIST.classes[7], X.shape, X_test.shape)
print(X.min().item(), X.max().item())

def get_batch(batch_size):
    return X[torch.randint(len(X), (batch_size,), device=device)]
```

OUTPUT

```
Sneaker torch.Size([6000, 1, 28, 28]) torch.Size([1000, 1, 28, 28])
-1.0 1.0
```

19.2 The two players

The two networks mirror each other. The discriminator is a small CNN, two stride-2 convolutions and a linear layer, ending in a sigmoid that gives the probability that its input is real. The generator is the decoder of the last chapter: a linear layer, then two transposed convolutions growing 7×7 to 28×28. Its input is a *latent vector* of 64 random numbers.

The weight helpers `conv`, `conv_t` and `linear`, and the `montage_plot` helper, are the ones from Chapter 18. (This `conv` makes weights; it is not the hand-written convolution of Chapter 13.) Here `get_batch` needs only a batch size, because there are no labels. The two networks are called `D` and `G`; single capital letters break this book's habit of lowercase function names, but they are what every GAN paper and program calls them.

gan_weights

```
latent_size = 64

d = [conv(1, 64),                 # discriminator: 28 -> 14
     conv(64, 128),               #                14 -> 7
     linear(128 * 7 * 7, 1)]      #                6272 -> 1 score

g = [linear(latent_size, 128 * 7 * 7),  # generator: 64 -> 6272
     conv_t(128, 64),                   #            7 -> 14
     conv_t(64, 1)]                     #            14 -> 28
```

gan_models

```
def D(x, d):
    x = F.leaky_relu(F.conv2d(x, d[0], stride=2, padding=1), 0.2)
    x = F.leaky_relu(F.conv2d(x, d[1], stride=2, padding=1), 0.2)
    return torch.sigmoid(F.linear(x.flatten(start_dim=1), d[2]))

def G(z, g):
    x = F.relu(F.linear(z, g[0])).reshape(-1, 128, 7, 7)
    x = F.relu(F.conv_transpose2d(x, g[1], stride=2, padding=1))
    return torch.tanh(F.conv_transpose2d(x, g[2], stride=2, padding=1))

z = torch.randn(5, latent_size, device=device)
print(G(z, g).shape, D(G(z, g), d).shape)
```

OUTPUT

```
torch.Size([5, 1, 28, 28]) torch.Size([5, 1])
```

The quarter in `conv_t`'s fan-in matters more here than it did for

the autoencoder. When we tried this generator with the fan-in of an ordinary convolution, which makes its starting weights half as large, its hidden units all switched off within a few hundred steps, its output went to a flat gray, and it never recovered.

The discriminator uses a *leaky* ReLU, which lets a small fraction (here 0.2) of each negative value through instead of zeroing it. The generator learns entirely from the gradients that flow back through the discriminator, and a plain ReLU blocks the gradient wherever its input is negative. The leak keeps information flowing.

19.3 The contest

Each player has its own optimizer, and each will update only its own weights. The learning rate and the `betas` setting of Adam are values that experience has shown to work well for GANs; we also fix 16 latent vectors so that we can watch the same 16 fakes improve over time.

gan_setup

```
batch_size = 64
real = torch.ones(batch_size, 1, device=device)    # target: "real"
fake = torch.zeros(batch_size, 1, device=device)   # target: "fake"

d_optimizer = torch.optim.Adam(d, lr=0.0002, betas=(0.5, 0.999))
g_optimizer = torch.optim.Adam(g, lr=0.0002, betas=(0.5, 0.999))

fixed_z = torch.randn(16, latent_size, device=device)  # to watch progress
```

Every training step has two halves:

1. **The discriminator's turn.** Show it a batch of real sneakers, with target 1, and a batch of fakes, with target 0. The loss is *binary cross-entropy*, the two-class version of the cross-entropy from Chapter 11. The fakes are `detach()`ed: in this half, only the discriminator learns.

2. **The generator's turn.** Make a fresh batch of fakes and pass them to the discriminator, but this time with target 1, "real." The loss is large when the discriminator sees through the fakes, and its gradient flows back through the discriminator into the generator, telling each generator weight how to change to make the fakes more convincing. Only the generator's optimizer steps.

The target in the second half deserves a closer look. The contest, as Goodfellow first wrote it down, has the generator *minimize* the discriminator's loss on fakes: make $\log(1-D(G(z)))$ as small as possible. But early in training the fakes are terrible, the discriminator rejects them with confidence, $D(G(z))$ is close to 0, and that expression is nearly flat there. Its gradient is tiny exactly when the generator most needs to learn. Training with target "real" instead means minimizing $-\log D(G(z))$, which is steep when $D(G(z))$ is near 0. Both losses reward the same thing, fakes that the discriminator believes, but this *non-saturating* version, which Goodfellow's paper itself recommends, gives the generator a strong signal from the start.

One practical note: applying a sigmoid and then taking a logarithm, as `F.binary_cross_entropy` does, can lose precision when the discriminator is very sure. `F.binary_cross_entropy_with_logits` takes the raw score before the sigmoid and combines the two steps in a numerically safer way. Larger projects use it, with the sigmoid removed from `D`. We keep the sigmoid so that `D`'s output reads as a probability.

gan_train

```
history, snapshots = [], {}

for step in range(6001):
    # 1. Teach the discriminator: real images -> 1, generated -> 0
    images = get_batch(batch_size)
    z = torch.randn(batch_size, latent_size, device=device)
    fakes = G(z, g).detach()             # don't update G in this step
    d_loss = (F.binary_cross_entropy(D(images, d), real)
              + F.binary_cross_entropy(D(fakes, d), fake))
    d_optimizer.zero_grad()
    d_loss.backward()
    d_optimizer.step()

    # 2. Teach the generator: make D say "real" for generated images
    z = torch.randn(batch_size, latent_size, device=device)
    g_loss = F.binary_cross_entropy(D(G(z, g), d), real)
    g_optimizer.zero_grad()
    g_loss.backward()
    g_optimizer.step()

    history.append((d_loss.item(), g_loss.item()))
    if step in (0, 200, 1000, 6000):
        with torch.no_grad():
            snapshots[step] = G(fixed_z, g)[:, 0].cpu().numpy()
    if step % 1000 == 0:
        print(step, round(d_loss.item(), 3), round(g_loss.item(), 3))
```

OUTPUT

```
0 1.829 0.682
1000 0.65 1.536
2000 0.946 1.935
3000 0.903 1.369
4000 0.911 1.309
5000 1.099 0.918
6000 1.164 0.969
```

19.4 Watching it learn

Figure 19.1 shows the 16 fixed latent vectors turned into images at four moments during training.

gan progress

```
plt.figure(figsize=(4.6, 1.4))
for i, (step, images) in enumerate(snapshots.items()):
    plt.subplot(1, 4, i + 1)
    plt.imshow(montage(images, padding_width=1, fill=-1),
               cmap='gray', vmin=-1, vmax=1)
    plt.title(f'step {step}')
    plt.axis('off')
plt.show()
```

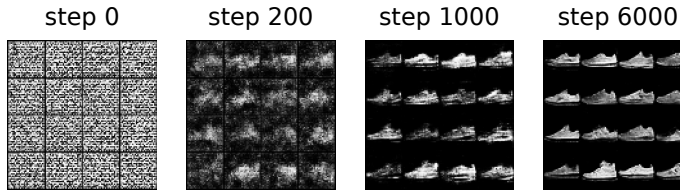


Figure 19.1: The same 16 random latent vectors, decoded by the generator after 0, 200, 1,000 and 6,000 training steps.

At step 0 the generator produces static. By step 200 the images have gone mostly dark, with faint streaky texture, because the discriminator's first easy lesson was that real images are mostly black. By step 1,000 they are recognizably sneakers, if ragged, and by step 6,000 they have cleaner outlines, soles and light and dark panels. Nothing in the generator's training ever showed it a real sneaker. Everything it knows came secondhand, through the discriminator's judgments.

The losses tell a story of their own (Figure 19.2):

gan_losses

```
d_hist, g_hist = np.array(history).T
print('discriminator:', round(d_hist[3000:].mean(), 2),
      ' at chance:', round(2 * np.log(2), 2))
print('generator:    ', round(g_hist[3000:].mean(), 2),
      ' at chance:', round(np.log(2), 2))

def smooth(h, n=100):                          # moving average of n steps
    return np.convolve(h, np.ones(n) / n, mode='valid')

plt.figure(figsize=(4.6, 2.4))
plt.plot(smooth(d_hist), 'C0-', label='discriminator')
plt.plot(smooth(g_hist), 'C1--', label='generator')
for y, style, text in [(2 * np.log(2), '-', '2 ln 2'),
                       (np.log(2), '--', 'ln 2')]:
    plt.axhline(y, color='gray', linestyle=style, linewidth=0.6)
    plt.text(len(history), y, text, ha='right', va='bottom')
plt.xlabel('step')
plt.ylabel('loss (average of 100 steps)')
plt.legend()
plt.show()
```

OUTPUT

```
discriminator: 0.98  at chance: 1.39
generator:     1.21  at chance: 0.69
```

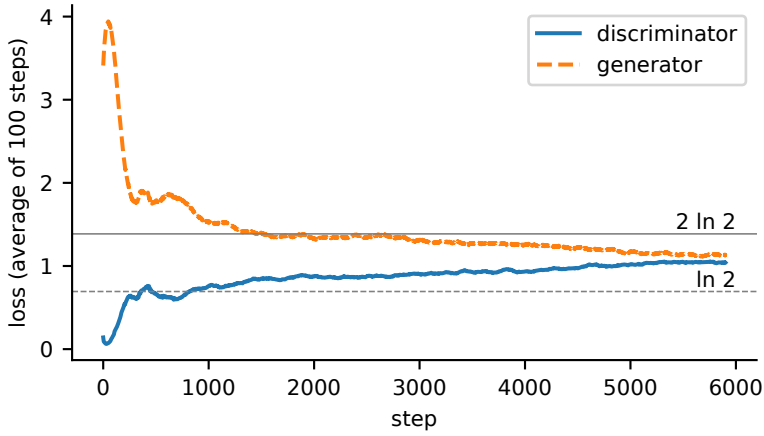


Figure 19.2: Discriminator (solid) and generator (dashed) losses, each averaged over 100 steps. The thin gray lines mark the losses when the discriminator can do no better than guess: $2\log 2$ for the discriminator, $\log 2$ for the generator.

Unlike every loss curve so far, these do not go to zero. If the generator were perfect, the discriminator could do no better than say 0.5 for every image. Its loss would then be $-\log 0.5 = \log 2 \approx 0.69$ on the real batch plus the same on the fake batch, $2 \log 2 \approx 1.39$, and the generator's loss would be $\log 2$. Those are the gray lines. In the first few hundred steps the discriminator wins easily and the generator's loss shoots up. After that the two curves drift slowly toward each other; over the second half of training the discriminator's loss averages 0.98 and the generator's 1.21. The discriminator still has the upper hand, but it is not running away with the contest. This is the best a GAN usually manages: not a minimum, not a settled equilibrium, but a balance that holds well enough for both players to keep learning. If one player gets too far ahead, the other stops learning, which is why GANs have a reputation for being temperamental to train.

19.5 New sneakers

Thirty-six brand new latent vectors, never seen before, give thirty-six new sneakers (Figure 19.3):

gan_samples

```
with torch.no_grad():
    z = torch.randn(36, latent_size, device=device)
    samples = G(z, g)
montage_plot(samples[:, 0].cpu().numpy(), size=3.2)
```

They vary the way real sneakers do: higher and lower, light and dark, plain and striped. Some are imperfect, smeared or lopsided. How convincing are they to the discriminator?

gan_judge

```
with torch.no_grad():
    print('real, training:', round(D(X[:1000], d).mean().item(), 3))
    print('real, test:    ', round(D(X_test, d).mean().item(), 3))
    print('fake:          ', round(D(samples, d).mean().item(), 3))
```

OUTPUT

```
real, training: 0.694
real, test:     0.668
fake:           0.432
```

On average the discriminator gives the real sneakers it trained on a 69 percent chance of being real, real sneakers it has never seen 67 percent, and the generator's sneakers 43 percent. (The test sneakers



Figure 19.3: Sneakers that do not exist.

matter: a discriminator that had merely memorized its training images would rate new real ones low.) It can still tell them apart more often than not, but it is far from certain, and neither player has won outright.

19.6 New, or copied?

A generator that simply memorized a few training images and reproduced them would also fool a discriminator for a while. The generator never sees a training image directly, but it is worth checking. For 1,000 generated sneakers we find the closest of the 6,000 training sneakers, measuring distance as the straight-line distance between the two lists of 784 pixels (`torch.cdist` computes it for every pair at once). As a yardstick we do the same for the 1,000 real test sneakers, which the GAN never saw:

gan_nearest

```
with torch.no_grad():
    many = G(torch.randn(1000, latent_size, device=device), g)
    dist = torch.cdist(many.flatten(1), X.flatten(1))   # (1000, 6000)
    test = torch.cdist(X_test.flatten(1), X.flatten(1))
    print('fakes:    ', round(dist.min(dim=1).values.median().item(), 2))
    print('real test:', round(test.min(dim=1).values.median().item(), 2))
    nearest = X[dist[:8].argmin(dim=1)]       # closest training images
pairs = torch.cat([many[:8], nearest])[:, 0].cpu().numpy()
montage_plot(pairs, grid_shape=(2, 8), size=4.6)
```

OUTPUT

```
fakes:     6.68
real test: 6.07
```

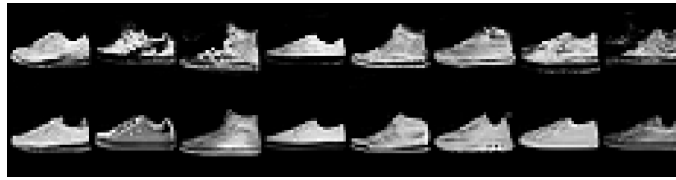


Figure 19.4: Top: eight generated sneakers. Bottom: the training image closest to each one.

The median generated sneaker is 6.68 away from its nearest training image, slightly farther than the 6.07 of a real sneaker the GAN never saw. If it were copying, that number would be close to zero. In Figure 19.4 each fake resembles its nearest neighbor in overall shape, as any sneaker would, but differs in the details: the stripes, the shading, the height of the collar. This check has a blind spot, though. A GAN can fail in a subtler way, called *mode collapse*: the generator finds a few kinds of sneaker that fool the discriminator and makes only those, new

in their details but far less varied than the real thing. Distance to the nearest training image would not notice; looking at many samples side by side, as in Figure 19.3, would.

Finally, since the generator turns *any* latent vector into a sneaker, we can walk in a straight line between two latent vectors, as we did with the autoencoder. We pick the two of our 36 samples whose images differ the most. To see what the walk adds, the second row blends the two end images' pixels instead, in the same eight steps (Figure 19.5):

gan_walk

```
with torch.no_grad():
    flat = samples.flatten(start_dim=1)
    far = torch.cdist(flat, flat).argmax().item()   # most different pair
    a, b = z[far // len(z)], z[far % len(z)]
    t = torch.linspace(0, 1, 8, device=device)[:, None]
    walk = G((1 - t) * a + t * b, g)            # blend the latent vectors
    ends = G(torch.stack([a, b]), g)
    t = t[:, :, None, None]
    fade = (1 - t) * ends[0] + t * ends[1]      # blend the pixels
both = torch.cat([walk, fade])[:, 0].cpu().numpy()
montage_plot(both, grid_shape=(2, 8), size=4.6)
```

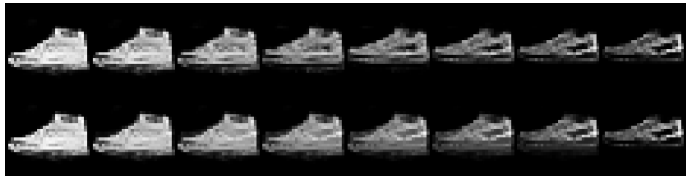


Figure 19.5: Top: a straight walk through the generator's latent space between two of its sneakers. Bottom: blending the pixels of the same two images.

Both rows start and end at the same images, a light high-top and a dark low sneaker with a bright stripe, and at this size they look alike at first glance. Look closely at the middle frames, enlarged side by side (Figure 19.6):

gan_walk_middle

```
middle = [both[3], both[8 + 3]]             # frame 4 of each row
montage_plot(middle, grid_shape=(1, 2), size=2.6)
```

The pixel blend is a double exposure: the faded outline of the high-top, collar and all, shows through underneath the dark sneaker, along with a faint second sole. The latent walk instead gives one

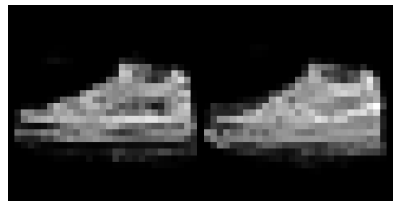


Figure 19.6: The fourth frame of each row, enlarged. Left: the latent walk. Right: the pixel blend.

sneaker in every frame, whose collar lowers and whose color darkens step by step. The difference is the one we saw with the autoencoder: blending pixels mixes two pictures, while moving through the latent space changes the object.

The generator was trained on nothing but random latent vectors drawn by `torch.randn`, so every region of its latent space that `torch.randn` is likely to reach was visited in training and judged by the discriminator. That is why a random latent vector gives a sneaker here, while a random code given to the autoencoder's decoder gave a blurry mixture.

The images here are tiny, but the core idea is the one behind the image generators you may have used: a network that maps random numbers to pixels. Most of today's systems are trained differently. They are *diffusion models*, which learn to remove noise from an image a little at a time, rather than to fool a discriminator. They are also vastly bigger. But they are built from the same parts you have now seen in action: weighted sums, nonlinearities, latent spaces and gradient descent.

Try it

1. Train on a different class: change the 7 in `load` (and in the `print` below it) to 1 (trousers) or 8 (bags). Which kind of image does the GAN learn fastest, judging by the snapshots at step 1,000?

2. Give `conv_t` the fan-in of an ordinary convolution, `n_in * k * k` (without the `// 4`), and retrain. Watch the printed losses: when both settle at exactly 0.693, the generator has died. Why that number? 0.693 is $\log 2 = -\log \frac{1}{2}$. Work out what score a network with no bias

terms gives an image that is all zeros, what probability the sigmoid turns that score into, and what the loss is for a probability of one half. Then try `F.leaky_relu(..., 0.2)` in place of the generator's two `F.relu`s instead.

3. Switch the generator to the original, saturating loss: replace `g_loss` with `-F.binary_cross_entropy(D(G(z, g), d), fake)`, which is $\log(1 - D(G(z)))$ averaged over the batch. How do the first thousand steps compare?

4. Remove the sigmoid from `D` and train with the numerically safer loss, `F.binary_cross_entropy_with_logits`, in place of `F.binary_cross_entropy`. Wherever a probability is printed, use `torch.sigmoid(D(...))`. Do the results change?

5. Change `latent_size` to 2 and retrain. Then decode a 10×10 grid of latent vectors with both numbers running from -2 to 2, and display it with `montage_plot`. What does each direction control?

Chapter 20

Onward

We began with a list of five numbers and an `@` sign. We have ended with networks that recognize photographs, learn new categories from a few hundred examples, and imagine sneakers that do not exist. Let's step back and see how few ideas that took, and then follow the same ideas to the large systems in use today.

20.1 What we built, and from what

Every network in this book was assembled from the same small set of parts:

- **Weighted sums.** The dot product of Chapter 6, organized as matrix multiplications (fully connected layers) or sliding windows (convolutions).
- **Nonlinearities.** The bend of a ReLU between layers, without which any stack of layers collapses into one.
- **A loss.** A smooth measure of how wrong the network is: cross-entropy for classifying, squared error for reconstructing, and the discriminator's verdict for generating.
- **Gradient descent.** Autograd computes which way is downhill for every weight at once, and an optimizer takes the step.
- **Data.** Lots of examples, ideally, and when there are few, the borrowed experience of a network trained on many.

Everything else is arrangement and scale. AlexNet is our Fashion-MNIST network with more layers and more kernels; transfer learning is AlexNet with a new last layer; the GAN generator is the autoencoder's decoder trained by a different teacher.

Since 2012 the field has added a handful of further parts, and two of them matter here. *Residual connections*, which made networks of more than a hundred layers trainable in 2015, let each block add its output

to its input instead of replacing it, so that information and gradients have a direct path through the whole stack. And *attention*, introduced in 2014 for translating text and made the whole of a network by the transformer of 2017, is the engine of every large language model since. Attention is the one piece we have not yet built, so let's build it.

20.2 Images as tokens

A transformer works on a sequence of *tokens*. In a language model a token is a piece of text, a word or part of a word, and each token is turned into a list of numbers. Images can be treated the same way. We go back to the MNIST digits, so that we can compare the result with the fully connected network of Chapter 12, and hold back 10,000 training images for validation. `get_batch` and `accuracy` are the helpers from Chapter 11, and the weight helper `layer` is the one from Chapter 14, which stores weights as (*outputs*, *inputs*) for `F.linear`.

on_load

```
import torch
import torch.nn.functional as F
from torchvision.datasets import MNIST
torch.manual_seed(0)

device = ('cuda' if torch.cuda.is_available()
          else 'mps' if torch.backends.mps.is_available() else 'cpu')

def load(train):
    ds = MNIST('data', train=train, download=True)
    return (ds.data.float() / 255.0).to(device), ds.targets.to(device)

X, Y = load(train=True)             # (60000, 28, 28), values 0-1
X_test, Y_test = load(train=False)
X, X_val = X[:50000], X[50000:]     # 10,000 held out for validation
Y, Y_val = Y[:50000], Y[50000:]
print(tuple(X.shape), len(X_val), len(X_test))
```

OUTPUT

```
(50000, 28, 28) 10000 10000
```

Cut an image into a 4×4 grid of 7×7 patches and flatten each patch, and the image becomes a sequence of 16 patches of 49 numbers each (Figure 20.1). That is how the *vision transformer* reads an image.

on_patches

```
def patches(x):                       # (n, 28, 28) -> (n, 16, 49)
    x = x.reshape(-1, 4, 7, 4, 7).permute(0, 1, 3, 2, 4)
    return x.reshape(-1, 16, 49)

tokens = patches(X_test[:1])[0]
print(tokens.shape)

plt.figure(figsize=(2.2, 2.2))
for i in range(16):
    plt.subplot(4, 4, i + 1)
    plt.imshow(tokens[i].reshape(7, 7).cpu(), cmap='gray', vmin=0, vmax=1)
    plt.axis('off')
plt.show()
```

OUTPUT

```
torch.Size([16, 49])
```

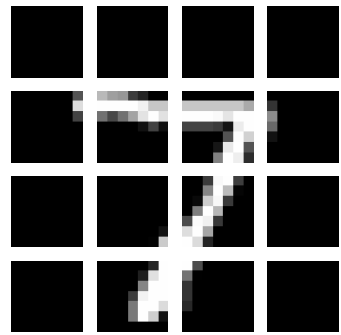


Figure 20.1: A handwritten 7 cut into 16 patches.

The two lines of `patches` are the only tricky code in this chapter, so here they are on a tiny 4×4 "image" of the numbers 0 to 15, cut into four 2×2 patches:

on_patch_small

```
a = torch.arange(16).reshape(1, 4, 4)     # a tiny 4x4 "image"
blocks = a.reshape(1, 2, 2, 2, 2)         # (image, block row, row,
                                          #  block column, column)
small = blocks.permute(0, 1, 3, 2, 4)     # the two block indices first
print(a[0])
print(small.reshape(1, 4, 4)[0])          # one 2x2 patch per row
```

OUTPUT

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11],
        [12, 13, 14, 15]])
tensor([[ 0,  1,  4,  5],
        [ 2,  3,  6,  7],
        [ 8,  9, 12, 13],
        [10, 11, 14, 15]])
```

`reshape` does not move any numbers; it only changes how they are counted. Reading the 16 numbers as (block row, row within the block, block column, column within the block) labels every pixel with the patch it belongs to. `permute` reorders those labels so that the two block indices come first, and a final `reshape` lays each patch out as one row: the top-left patch is 0, 1, 4, 5. For MNIST the same two lines use blocks of 7 instead of 2.

Two more steps turn patches into tokens. First, each patch of 49 pixels is multiplied by a learned weight matrix, a fully connected layer, to give a token of 64 numbers. Raw pixels are a poor description of a patch; the learned *projection* lets training decide which 64 measurements of a patch are worth having, much as the first layer of a CNN learns its kernels.

Second, each token gets a *position embedding* added to it: a list of 64 learned numbers that belongs to its place in the grid, one for each of the 16 positions. This is needed because attention, as we will see, treats its tokens as an unordered collection. Shuffle the 16 patches and, without position embeddings, the network could not tell the difference. A convolution knows where everything is, because it slides over a grid; attention must be told.

20.3 Attention

The central operation of a transformer is *attention*. Every token computes three things with three weight matrices: a *query* (what it is looking for), a *key* (what it has to offer), and a *value* (what it will pass on). Then four steps:

1. The dot product of every query with every key scores how relevant each token is to each other token.
2. The scores are divided by the square root of the number of numbers in each query and key, which keeps them in a range where the softmax is not too sharp (the square-root law of Chapter 9 again).
3. A softmax turns each token's scores into weights that add up to one.
4. Each token receives the weighted mixture of everyone's values.

Here are the four steps done by hand on three tokens of two numbers each. To keep it small, the query and key of each token are just the token itself, and each value is a single number:

on_toy

```
t = torch.tensor([[1.0, 0.0],            # three tokens of two numbers
                  [0.0, 1.0],
                  [1.0, 1.0]])
Q, K = t, t                      # as if query and key layers did nothing
V = torch.tensor([[10.0], [20.0], [30.0]])  # what each token passes on
scores = Q @ K.T / 2 ** 0.5      # every query against every key
A = scores.softmax(dim=1)        # each row adds up to 1
print(scores)
print(A)
print(A @ V)                     # each token's mixture of the values
```

OUTPUT

```
tensor([[0.71, 0.00, 0.71],
        [0.00, 0.71, 0.71],
        [0.71, 0.71, 1.41]])
tensor([[0.40, 0.20, 0.40],
        [0.20, 0.40, 0.40],
        [0.25, 0.25, 0.50]])
tensor([[20.00],
        [22.03],
        [22.55]])
```

Read the table of scores row by row. The first token, $(1, 0)$, has a positive dot product with itself and with the third token, $(1, 1)$, and zero with the second, so after the softmax it gives weights 0.40, 0.20 and 0.40 and receives the mixture $0.4 \times 10 + 0.2 \times 20 + 0.4 \times 30 = 20$. The third token matches everyone, itself most, and receives about 22.5. Every token has looked at every other token, decided how much each one matters to it, and taken a blend of what they offer. In a real transformer the query, key and value come from learned weight matrices, so training decides what each token looks for.

Our model has one attention layer. Its output is *added* to the tokens, a residual connection like the ones that made 2015's very deep networks trainable, so that each token keeps what it had and gains a mixture of what the others offer. A small two-layer network then processes each token on its own, again added to it. Finally, the 16 tokens are averaged into one list of 64 numbers, and a last linear layer turns that into ten class scores. The weights, in a list `w` as usual:

on_weights

```
d = 64                                          # numbers per token
position = 0.1 * torch.randn(16, d, device=device)

w = (layer(49, d)                       # w[0], w[1]: patch -> token
     + [position.requires_grad_()]      # w[2]: one vector per position
     + layer(d, d)                      # w[3], w[4]: query
     + layer(d, d)                      # w[5], w[6]: key
     + layer(d, d)                      # w[7], w[8]: value
     + layer(d, 2 * d)                  # w[9] to w[12]: a small network
     + layer(2 * d, d)                  #    applied to each token
     + layer(d, 10))                    # w[13], w[14]: 10 class scores
print(sum(p.numel() for p in w))
```

OUTPUT

```
33930
```

That is about 34,000 weights, a twelfth of the convolutional network of Chapter 14. The model itself:

on_model

```
def model(x, w):
    t = F.linear(patches(x), w[0], w[1]) + w[2]   # (n, 16, d) tokens
    Q = F.linear(t, w[3], w[4])         # what each token looks for
    K = F.linear(t, w[5], w[6])         # what each token offers
    V = F.linear(t, w[7], w[8])         # what each token passes on
    scores = Q @ K.transpose(1, 2) / d ** 0.5     # (n, 16, 16)
    A = scores.softmax(dim=2)           # each row sums to 1
    t = t + A @ V                       # add a mix of the others
    t = t + F.linear(F.relu(F.linear(t, w[9], w[10])), w[11], w[12])
    return F.linear(t.mean(dim=1), w[13], w[14]), A

scores, A = model(X[:5], w)
print(scores.shape, A.shape)
```

OUTPUT

```
torch.Size([5, 10]) torch.Size([5, 16, 16])
```

Look at what the attention part contains: three fully connected layers, a matrix of dot products scaled by a constant, a softmax, and one more matrix multiplication. Every one of them appeared in Chapters 6 to 12, and in the toy example above. Two things are new. Our tensors now have a batch axis, (*images*, 16, 64), and `@` on such tensors multiplies the last two axes separately for every image. `transpose(1, 2)` swaps the last two axes of `K`, the batched version of `K.T`, so that `Q @ K.transpose(1, 2)` gives, for each image, the 16×16 table of dot products between every query and every key. Training is the loop we have used since Chapter 11:

on_train

```
optimizer = torch.optim.Adam(w, lr=0.001)

for step in range(8001):
    x, y = get_batch(X, Y, 128)
    loss = F.cross_entropy(model(x, w)[0], y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if step % 1000 == 0:
        with torch.no_grad():
            acc = accuracy(model(X_val, w)[0], Y_val)
        print(step, round(loss.item(), 3), round(acc, 4))
```

OUTPUT

```
0 3.31 0.1079
1000 0.403 0.9125
2000 0.333 0.9422
3000 0.101 0.9509
4000 0.105 0.9586
5000 0.066 0.9603
6000 0.074 0.9636
7000 0.045 0.964
8000 0.061 0.9633
```

on_test

```
with torch.no_grad():
    scores, _ = model(X_test, w)
    print('test accuracy:', round(accuracy(scores, Y_test), 4))
```

OUTPUT

```
test accuracy: 0.9601
```

20.4 How good is it, and what does it look at?

96.0 percent on the test set, from 33,930 weights and well under a minute of training on a laptop. The fully connected network of Chapter 12 reached 98.1 percent with eight times as many weights, so our tiny transformer is not the better digit reader. That should not be a surprise. Apart from cutting the image into patches and treating every patch the same way, attention, like a fully connected layer, assumes almost nothing about images: it has to learn from data which patches matter to which and, through the position embeddings, even where each patch is. A convolution, by contrast, comes with built-in assumptions that suit images: nearby pixels matter together, and the same pattern

means the same thing anywhere in the picture. The flexibility of attention pays off at scale. The original vision transformer beat the best convolutional networks only after being pretrained on about 300 million images. Here, the point is not the score but that you have built one.

We can look inside. `A` holds, for every patch, the attention weights it gives to all 16 patches. Here is where patch 5 (second row, second column) looks in the second test image, a 2, laid out as the 4×4 grid. (`np.kron` in the helper `heat` blows each of the 16 weights up into a 7×7 block, so that the grid can be drawn at the size of the image.)

on_attention

```
with torch.no_grad():
    _, A = model(X_test[:2], w)
print(A[1, 5].reshape(4, 4).cpu())   # where patch 5 looks, in image 1

def heat(a):                         # 16 weights -> 28x28 picture
    return np.kron(a.reshape(4, 4).cpu().numpy(), np.ones((7, 7)))

plt.figure(figsize=(4.6, 2.4))
for row in range(2):
    image, a = X_test[row].cpu(), A[row]
    panels = [(image, 'image'), (heat(a[5]), 'patch 5 looks at'),
              (heat(a.mean(dim=0)), 'average attention')]
    for col, (p, title) in enumerate(panels):
        plt.subplot(2, 3, 3 * row + col + 1)
        plt.imshow(p, cmap='gray' if col == 0 else 'magma')
        if col == 1:
            plt.gca().add_patch(plt.Rectangle((6.5, 6.5), 7, 7,
                                fill=False, ec='white', lw=1))
        if row == 0:
            plt.title(title)
        plt.axis('off')
plt.show()
```

OUTPUT

```
tensor([[    0.02,     0.00,     0.02,     0.02],
        [    0.03,     0.21,     0.01,     0.04],
        [    0.04,     0.00,     0.20,     0.01],
        [    0.05,     0.02,     0.01,     0.33]])
```

In the 2, patch 5 spreads its attention: a fifth on itself, a fifth on the patch diagonally below it, and a third on the bottom-right corner, where the 2's base stroke ends. In the 7, patch 5 attends only to itself. The averages on the right show that attention is concentrated on a few

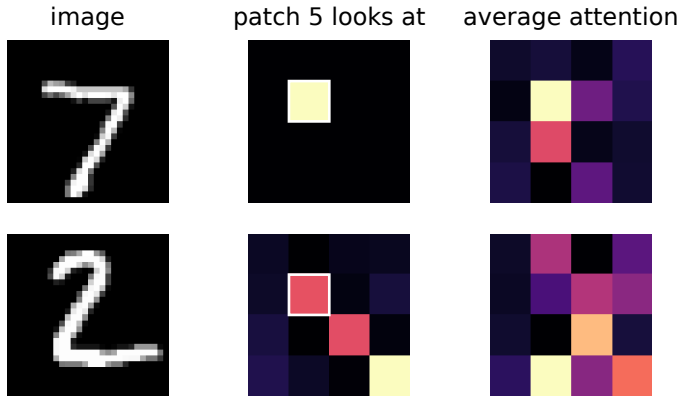


Figure 20.2: Attention in two test images. Middle: the weights that patch 5 (framed) gives to each of the 16 patches; brighter means more. Right: the attention each patch receives, averaged over all 16 queries. Each panel has its own color scale.

patches, but they are not simply the patches with the most ink. In the 7, the patch below the left end of the bar is empty, yet it receives the second-most attention; in the 2, the patch that holds most of the diagonal stroke receives almost none. The network has found its own use for the mechanism, and with one layer and one head, that use is not easy to put into words. Interpreting what large transformers attend to is an active research problem.

A real transformer stacks dozens of such layers, each with several attention computations side by side (*heads*), each free to look for something different. It is the same layer, many times over, with one addition that helps very deep stacks train: *layer normalization*, which rescales each token to a standard size before each attention and each small network.

20.5 From attention to language models

How is a language model like ChatGPT trained? To predict the next token of text. Given the tokens so far, the model produces a score for every token in its vocabulary, and the loss is the cross-entropy between those scores and the token that actually came next. That is the classifier of Chapter 11, with a vocabulary of tens to hundreds of thousands of tokens in place of ten digits, and with attention, instead of a single weight matrix, between the input and the scores. One more detail: each token may attend only to itself and the tokens before it, so the model cannot peek at the answer.

The overall recipe is Chapter 16's. First *pretrain* a very large model on next-token prediction over an enormous amount of text, which forces it to absorb grammar, facts and patterns of reasoning, just as AlexNet absorbed edges and textures from ImageNet. Then *fine-tune* it on a much smaller, carefully chosen dataset. For a chat assistant, that means *instruction tuning* on examples of requests and good responses, followed by learning from judgments of which of two answers is better, often called reinforcement learning from human feedback, or RLHF, and, in recent models, by reinforcement learning on problems whose answers can be checked, such as mathematics and code, which is what teaches them to reason step by step. The models have billions of weights and are trained on trillions of tokens of text, but the recipe is one you already know.

20.6 Questions to invent with

This book covered the basics. If it has done its job, you can read the code of modern models and recognize what you see. But reading is not the point. Many chapters ended on something nobody fully understands, and you now have the tools to go after it. A few questions, each small enough for a laptop:

- **How little needs to be learned?** In Chapter 14 reshaping the first-layer kernels made no difference to accuracy, and in Chapter 17 random kernels came within a point and a half of a trained network on clothing, but not on photographs. Where exactly is the line, and why?

- **How few examples are enough?** You learned digits from 60,000 examples; a child learns from a handful. Combine the unlabeled codes of Chapter 18 with a few labels and see how far down you can push the number.
- **Can a network learn a rule exactly?** The network of Appendix A learned the Game of Life without a single mistake. What is the smallest network that can? Which rules can it not learn?
- **What is attention looking at?** The patterns in Figure 20.2 were not the ones we expected. Train on a task where you know the right answer and find out whether attention finds it.
- **Build the rover.** Chapter 1's rover is a camera, three actions and a convolutional network trained on labeled frames. Every part of it is in this book.

Pretrained models for pose, segmentation, detection and image generation can be downloaded in a few lines, and they are worth opening up: inside each are the parts you know. And whatever you build, remember the ants and bees: a network that already knew the answer, and mistakes that pointed to the background rather than the insect. Ask what your model was trained on, what your data contains, who is missing from it, and what your model is really responding to.

Try it

1. Remove the position embeddings: replace `+ w[2]` in `model` with nothing and retrain. How much accuracy is lost when the network no longer knows where each patch came from?

2. Feed the trained model a test image with its 16 patches shuffled (`patches(x)[:, torch.randperm(16)]`, inside a copy of `model`). This is the harder half: to shuffle the same 7×7 blocks for the fully connected network of Chapter 12, cut the images into patches, shuffle them, and undo `patches` by running its `reshape` and `permute` backwards (the `permute` is its own inverse). Which network is more upset?

3. Use 4×4 patches instead: 49 tokens of 16 numbers each (change `patches`, the 49 in `layer(49, d)`, and the 16 positions). Smaller patches mean more tokens and finer attention, at what cost in time?

4. Add a second attention layer with its own query, key and value weights, between the first one and the small per-token network. Append three more `layer(d, d)` to `w`, which become `w[15]` to `w[20]`, and repeat the four attention lines of `model` with them. Does a second round of mixing help on this small dataset?

5. Train the same model on Fashion-MNIST (change `MNIST` to `FashionMNIST` in `on_load`) and compare with the 88.3 percent of the fully connected network of Chapter 12 and the 90.9 percent of the CNN of Chapter 14.

20.7 Final thoughts

There is a temptation, once you have seen how simple the parts are, to conclude that there is nothing to it: just multiplication and addition, repeated. I hope you draw the opposite conclusion. Out of multiplication and addition, repeated, came machines that learn to see, to draw and to write, and nobody fully understands why it works as well as it does. There is a great deal left to find out.

You are equipped to help. Every model you will read about, however large, is a list of weights, a function that uses them, a loss and a loop. When the next one is announced, you can find its code,

look for the parts you know, and build a small version yourself, on your own laptop, to see what it really does. Better still, you can build the one nobody has announced yet. The best way to understand these machines, and the best way to decide what they will become, is to invent them. Thank you for building these machines with me.

Happy coding!

Appendix

Appendix A

A Project of Your Own

The best way to finish learning deep learning is to apply it to something you care about. This appendix is a plan for a first project: a classifier for your own images, trained with the recipe of Chapter 16, tested with honest experiments, and presented to other people. It grew out of the final project of the author's Mathematics of Data Science course, where students have used it to sort oranges by quality and to tell bats from bugs.

A.1 Warming up

Before starting, check that you can do each of these without looking back. Each is a few lines of code from an earlier chapter.

1. Load MNIST and draw a montage of the first 36 digits (Chapter 8).
2. Score MNIST with a random 784×10 matrix, then improve it by random search with a population to at least 75 percent on the test set (Chapter 10). Hill climbing alone reached about 67.
3. Read a color photograph from a file or URL, resize it, and draw a grayscale copy (Chapter 7).
4. Correlate the grayscale image with ten random 3×3 kernels and draw each kernel beside its feature map (Chapter 13).
5. Write up what you did in a short Markdown file with your figures, and post it on GitHub. (Markdown is plain text with a few marks for formatting, such as # at the start of a heading; GitHub is a free website for sharing code, and shows Markdown files as formatted pages. Its own guides take a few minutes to follow.)

A.2 Choosing a problem

Choose something you can photograph or download in quantity, where the answer is a category: ripe or unripe, healthy leaf or diseased, one bird species or another. Two to five classes is plenty. A hundred images of each class is enough to start, thanks to transfer learning. Ask two questions before you collect anything. Would a person be able to tell the classes apart from the image alone? And does anything other than the class differ between your folders, such as the background, the lighting or the camera? The mistakes of the ants-and-bees network in Chapter 16 suggest it partly learned the background. Yours will, if you let it.

Put the images in one folder per class, exactly as in the ants-and-bees data, so that `ImageFolder` can read them. Keep a separate test folder that you do not look at until the very end.

pi_folders

```
data_dir = 'data/hymenoptera_data'      # replace with your own folder
for split in ['train', 'val']:
    counts = {name: len(os.listdir(f'{data_dir}/{split}/{name}'))
              for name in sorted(os.listdir(f'{data_dir}/{split}'))}
    print(split, counts)
```

OUTPUT

```
train {'ants': 124, 'bees': 121}
val {'ants': 70, 'bees': 83}
```

In the ants-and-bees data the folder called `val` plays the part of the test folder, exactly as in Chapter 16; validation images are taken from `train`. (The code downloads the data first, as in that chapter.)

A.3 Experiments, done fairly

Your first model will work, and then the temptation is to try things: a different learning rate, more augmentation, another architecture. The trouble is that every training run is partly luck. The random order of the batches and the random crops change the result, and so does which images end up in the validation set. Put everything that defines a run into one function, with the settings as arguments and a seed for the luck. Two details matter. The validation split is fixed with a generator of its own, so that every run is scored on the same 50 photographs; and the function reports the accuracy after the last epoch, not the best of all

epochs, because picking the best of several noisy scores makes every setting look better than it is. `evaluate` is the helper of Chapter 16.

pi_transforms
```
normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                 std=[0.229, 0.224, 0.225])
val_tf = transforms.Compose([transforms.Resize(256),
                             transforms.CenterCrop(224),
                             transforms.ToTensor(), normalize])
train_tf = transforms.Compose([transforms.RandomResizedCrop(224),
                               transforms.RandomHorizontalFlip(),
                               transforms.ToTensor(), normalize])
```

pi_experiment
```
def run_experiment(lr=0.001, batch_size=16, augment=True, epochs=5,
                   seed=0):
    train_all = ImageFolder(f'{data_dir}/train',
                            train_tf if augment else val_tf)
    val_all = ImageFolder(f'{data_dir}/train', val_tf)
    g = torch.Generator().manual_seed(0)     # the same split every run
    split = torch.randperm(len(train_all), generator=g).tolist()
    torch.manual_seed(seed)                  # the luck that varies
    train_loader = DataLoader(Subset(train_all, split[50:]),
                              batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(Subset(val_all, split[:50]), batch_size=64)

    net = alexnet(weights=AlexNet_Weights.DEFAULT)
    net.requires_grad_(False)
    net.classifier[6] = nn.Linear(4096, len(train_all.classes))
    net = net.to(device)
    head = net.classifier[6].parameters()
    optimizer = torch.optim.SGD(head, lr=lr, momentum=0.9)

    for epoch in range(epochs):
        net.train()
        for x, y in train_loader:
            loss = nn.functional.cross_entropy(net(x.to(device)),
                                               y.to(device))
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
    return evaluate(net, val_loader)         # after the last epoch

print(run_experiment())
```

OUTPUT
```
0.9
```

Now an experiment is a loop. Change one thing at a time, repeat every setting with several seeds, and write the results to a file as you go so that nothing is lost:

pi_grid

```
import itertools
import csv

grid = {'lr': [0.01, 0.001], 'augment': [True, False]}
seeds = [0, 1, 2]

rows = []
for lr, augment in itertools.product(grid['lr'], grid['augment']):
    accs = [run_experiment(lr=lr, augment=augment, seed=s)
            for s in seeds]
    rows.append({'lr': lr, 'augment': augment,
                 'mean': np.mean(accs), 'min': min(accs),
                 'max': max(accs)})
    print(f'lr {lr:<6} augment {augment!s:5}  '
          f'{np.mean(accs):.3f}  ({min(accs):.2f} to {max(accs):.2f})')

with open('data/experiments.csv', 'w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=rows[0].keys())
    writer.writeheader()
    writer.writerows(rows)
```

OUTPUT

```
lr 0.01   augment True   0.887  (0.80 to 0.96)
lr 0.01   augment False  0.920  (0.90 to 0.94)
lr 0.001  augment True   0.880  (0.82 to 0.92)
lr 0.001  augment False  0.913  (0.90 to 0.92)
```

Then draw the results with their spread (Figure A.1).

pi_plot

```
labels = [f"lr {r['lr']}\n{'aug' if r['augment'] else 'no aug'}"
          for r in rows]
means = np.array([r['mean'] for r in rows])
low = means - [r['min'] for r in rows]
high = [r['max'] for r in rows] - means

plt.figure(figsize=(4.6, 2.0))
plt.errorbar(labels, means, yerr=[low, high], fmt='o', capsize=4)
plt.ylim(0.5, 1.0)
plt.ylabel('validation accuracy')
plt.show()
```

Look at the whiskers before the dots. Three runs of the *same*

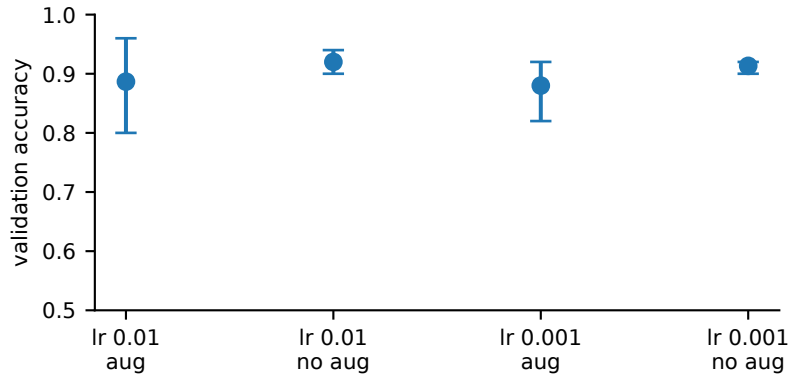


Figure A.1: Validation accuracy for four settings, three runs each. Dots show the mean; whiskers run from the worst run to the best.

setting, learning rate 0.01 with augmentation, scored anywhere from 80 to 96 percent. The differences between settings are a few points, much smaller than that. With 50 validation images, one image is two percentage points, so a "2 percent improvement" is a single photograph. If anything, augmentation looks slightly worse here, which would make sense after only five epochs, since randomly cropped photographs are harder to learn quickly; but these numbers cannot tell. On this data, none of these choices clearly matters. That is a real finding, and a common one. Had we run each setting once, we would have declared a winner by luck. The square-root law of Chapter 9 says what to do about it: to halve the noise in an average, use four times as many runs or four times as many validation images.

Here are experiments worth running on your own data:

- Train only the last layer, fine-tune everything, and train from scratch, as in Chapter 16.
- Compare two learning rates a factor of ten apart, and two batch sizes such as 16 and 64.
- Train with augmentation and without it.
- Replace AlexNet with a newer network from `torchvision.models`, such as `resnet18`. (Its last layer is called `fc`, not `classifier[6]`, and has 512 inputs, not 4,096: `net.fc = nn.Linear(512, len(classes))`.)

- Use random features instead of trained ones, as in Chapter 17, as a baseline that any trained model should beat.

Experiment-tracking services such as Weights & Biases record every run's settings and curves automatically and draw the comparison plots for you. A CSV file and a plotting script do the same job for a small project.

Only when you have chosen your final settings do you run the model once on the test folder. That number is the one you report.

A.4 A second project: learning a law of nature

Not every project needs a camera. Here is one that runs in seconds and asks a real question: can a network discover the rule of a world just by watching it? In Chapter 13 we *gave* a convolution the rule of Conway's Game of Life. Now we hide the rule and show a network only pairs of worlds, before and after one step, and ask it to work out what happens next. The helpers below compute the true next step, as in Chapter 13 but with PyTorch, and make random worlds, each with its own density of living cells so that the network sees crowded and empty worlds alike:

pi_life_data

```
import torch.nn.functional as F

neighbors = torch.ones(1, 1, 3, 3, device=device)
neighbors[0, 0, 1, 1] = 0

def wrap(x):                            # join opposite edges of the grid
    return F.pad(x, (1, 1, 1, 1), mode='circular')

def life_step(world):                   # the true rule, from Chapter 13
    n = F.conv2d(wrap(world), neighbors)
    return ((n == 3) | ((world == 1) & (n == 2))).float()

def random_worlds(k, size=32):          # each with its own density
    density = 0.1 + 0.6 * torch.rand(k, 1, 1, 1, device=device)
    return (torch.rand(k, 1, size, size, device=device) < density).float()
```

The network has the shape the rule itself suggests. A 3×3 convolution with 8 channels looks at each cell's neighborhood, a ReLU, and then a 1×1 convolution, which mixes the 8 channels at each cell

without looking at the neighbors, produces one score per cell: positive for alive, negative for dead.

pi_life_net

```
torch.manual_seed(0)
w = [(0.5 * torch.randn(8, 1, 3, 3, device=device)).requires_grad_(),
     torch.zeros(8, device=device, requires_grad=True),
     (0.5 * torch.randn(1, 8, 1, 1, device=device)).requires_grad_(),
     torch.zeros(1, device=device, requires_grad=True)]

def life_net(x, w):
    h = F.relu(F.conv2d(wrap(x), w[0], w[1]))   # 8 detectors, 3x3 each
    return F.conv2d(h, w[2], w[3])              # mixed into one score
```

Each training step makes 64 fresh random worlds, so the network never sees the same example twice:

pi_life_train

```
optimizer = torch.optim.Adam(w, lr=0.01)
for step in range(3001):
    before = random_worlds(64)
    after = life_step(before)
    loss = F.binary_cross_entropy_with_logits(life_net(before, w), after)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if step % 1000 == 0:
        print(step, f'{loss.item():.5f}')
```

OUTPUT

```
0 0.59461
1000 0.00328
2000 0.00069
3000 0.00029
```

Now the test, on 500 worlds it has never seen:

pi_life_test

```
with torch.no_grad():
    before = random_worlds(500)               # worlds it has never seen
    guess = (life_net(before, w) > 0).float()
    wrong = (guess != life_step(before)).sum().item()
print(f'{wrong} wrong cells out of {before.numel():,}')
```

OUTPUT

```
0 wrong cells out of 512,000
```

Not one mistake in half a million cells. But is that the rule itself, or only a very good imitation of it? Here we can do better than any

number of random tests. A cell's next state depends only on its own 3×3 neighborhood, and there are just $2^9 = 512$ of those, so we can try every one. Each neighborhood, wrapped around on itself, is a whole 3×3 world, and the network's answer for its middle cell is its verdict:

pj_life_all

```
codes = torch.arange(512, device=device)
bits = (codes[:, None] >> torch.arange(9, device=device)) & 1
patches = bits.float().view(512, 1, 3, 3)   # every 3x3 neighborhood
with torch.no_grad():
    guess = (life_net(patches, w) > 0).float()[:, 0, 1, 1]
truth = life_step(patches)[:, 0, 1, 1]     # the middle cell's fate
print(f'{(guess != truth).sum().item()} wrong out of {len(codes)}')
```

OUTPUT

```
0 wrong out of 512
```

None wrong: the network has learned the rule exactly, from examples alone, in a few seconds. (The half million random cells had almost certainly covered every neighborhood hundreds of times, which is why they found no mistakes either; testing them all says so for certain.) A network that has the rule can run any pattern, so its glider should walk (Figure A.2).

pj_life_glider

```
world = torch.zeros(1, 1, 12, 12, device=device)
world[0, 0, 1, 2] = world[0, 0, 2, 3] = 1
world[0, 0, 3, 1] = world[0, 0, 3, 2] = world[0, 0, 3, 3] = 1

plt.figure(figsize=(4.6, 1.0))
for i in range(6):
    plt.subplot(1, 6, i + 1)
    plt.imshow(world[0, 0].cpu(), cmap='gray_r', vmin=0, vmax=1)
    plt.title(f'step {4 * i}')
    plt.xticks([])
    plt.yticks([])
    plt.gca().spines[:].set_visible(True)
    for _ in range(4):                          # the network's rule, not
        with torch.no_grad():                   # Conway's
            world = (life_net(world, w) > 0).float()
plt.show()
```

It walks. Twenty steps of the network's own rule carry the glider five cells down and to the right, exactly as Conway's rule would. From here the project is yours. What is the smallest number of channels that can learn the rule? Can a network learn HighLife, or a rule you invent,

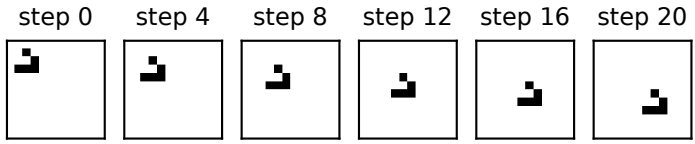


Figure A.2: A glider, run by the trained network instead of by Conway's rule.

as easily? Can it learn the drumhead of Chapter 13, a rule with real numbers instead of alive and dead? Learning the laws of a world from watching it is one of the open frontiers of machine learning, and this is a small corner of it you can explore on a laptop.

A.5 Telling others

A result nobody hears about does not exist. Two forms matter most. The first is a *notebook* that runs from top to bottom and reproduces every number you report, so that someone else can check your work and build on it. The second is a short *write-up*: the problem and why it matters, what the data look like, the method, the experiments with their error bars, and what you would try next. Keeping it to a page or two forces you to decide what matters.

In the course this appendix comes from, students also present each project as a slide deck, a public web page, a conference-style poster and a short video pitch. Each is the same story told for a different audience, and each is worth trying once.

Whatever the form, report what did not work as well as what did. A table of experiments in which most choices made no difference, like Figure A.1, is more useful to the next person than a single lucky number.

Notes and Further Reading

These notes give the main sources for the results and ideas named in each part, and a few places to read further. Sources quoted directly are cited in footnotes where they appear.

Part I: Why Deep Learning

The first model neuron is Warren S. McCulloch and Walter Pitts, "A Logical Calculus of the Ideas Immanent in Nervous Activity," *Bulletin of Mathematical Biophysics* 5 (1943): 115–133. Frank Rosenblatt's perceptron is in "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review* 65, no. 6 (1958): 386–408, and Oliver Selfridge's Pandemonium in "Pandemonium: A Paradigm for Learning," in *Mechanisation of Thought Processes* (London: Her Majesty's Stationery Office, 1959). Arthur Samuel's checkers program is described in "Some Studies in Machine Learning Using the Game of Checkers," *IBM Journal of Research and Development* 3, no. 3 (1959): 210–229, and Donald Michie's matchbox learner MENACE in "Experiments on the Mechanization of Game-Learning," *The Computer Journal* 6, no. 3 (1963): 232–236. The critique that cooled the field is Marvin Minsky and Seymour Papert, *Perceptrons* (Cambridge, MA: MIT Press, 1969).

The revival: John J. Hopfield, "Neural Networks and Physical Systems with Emergent Collective Computational Abilities," *PNAS* 79 (1982): 2554–2558; David E. Rumelhart, Geoffrey E. Hinton and Ronald J. Williams, "Learning Representations by Back-Propagating Errors," *Nature* 323 (1986): 533–536; Kunihiko Fukushima, "Neocognitron," *Biological Cybernetics* 36 (1980): 193–202; and Yann LeCun, Léon Bottou, Yoshua Bengio and Patrick Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE* 86, no. 11 (1998): 2278–2324, which also describes MNIST. Spiking networks as the "third generation" are from Wolfgang Maass, "Networks of Spiking Neurons: The Third Generation of Neural Network Models," *Neural Networks* 10, no. 9 (1997): 1659–1671. AlphaGo is David Silver et al., "Mastering the Game of Go with Deep Neural Networks and Tree Search," *Nature* 529 (2016): 484–489.

On computers for thinking with: J. C. R. Licklider, "Man-Computer Symbiosis," *IRE Transactions on Human Factors in Electronics* HFE-1 (1960): 4–11; Douglas C. Engelbart, *Augmenting Human Intellect: A Conceptual Framework* (Menlo Park, CA: Stanford Research Institute, 1962); and Seymour Papert,

Mindstorms: Children, Computers, and Powerful Ideas (New York: Basic Books, 1980). Turing's own early thoughts on learning machines are collected in B. Jack Copeland, ed., *The Essential Turing* (Oxford: Clarendon Press, 2004).

Part II: Images Are Numbers

Error diffusion is Robert W. Floyd and Louis Steinberg, "An Adaptive Algorithm for Spatial Greyscale," *Proceedings of the Society for Information Display* 17, no. 2 (1976): 75–77. Fashion-MNIST is Han Xiao, Kashif Rasul and Roland Vollgraf, "Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms," arXiv:1708.07747 (2017). The NumPy documentation at https://numpy.org/doc is the best reference for the array operations of Chapters 6–8.

Part III: Learning from Data

Initializing weights by the square-root rule follows Xavier Glorot and Yoshua Bengio, "Understanding the Difficulty of Training Deep Feedforward Neural Networks," *AISTATS* (2010), and, for ReLU networks, Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, "Delving Deep into Rectifiers," *ICCV* (2015). Adam is Diederik P. Kingma and Jimmy Ba, "Adam: A Method for Stochastic Optimization," *ICLR* (2015). The hand-built autograd of Chapter 11 follows the spirit of Andrej Karpathy's *micrograd*, https://github.com/karpathy/micrograd. For the mathematics behind all of Part III, Marc Peter Deisenroth, A. Aldo Faisal and Cheng Soon Ong, *Mathematics for Machine Learning* (Cambridge University Press, 2020), is free online.

Part IV: Seeing with Convolutions

The Game of Life reached the public in Martin Gardner, "Mathematical Games: The Fantastic Combinations of John Conway's New Solitaire Game 'Life'," *Scientific American* 223 (October 1970): 120–123. Reaction–diffusion is A. M. Turing, "The Chemical Basis of Morphogenesis," *Philosophical Transactions of the Royal Society of London B* 237 (1952): 37–72; the Gray–Scott patterns were mapped by John E. Pearson, "Complex Patterns in a Simple System," *Science* 261 (1993): 189–192.

AlexNet is Alex Krizhevsky, Ilya Sutskever and Geoffrey E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *NeurIPS* 25 (2012); the version in PyTorch follows Alex Krizhevsky, "One Weird Trick for Parallelizing Convolutional Neural Networks," arXiv:1404.5997 (2014). Dropout is Nitish Srivastava et al., "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research* 15 (2014):

1929–1958. How features grow more specific with depth is measured in Jason Yosinski, Jeff Clune, Yoshua Bengio and Hod Lipson, "How Transferable Are Features in Deep Neural Networks?" *NeurIPS* 27 (2014). The images in the web edition that show what a unit "wants to see" are made by the method surveyed in Chris Olah, Alexander Mordvintsev and Ludwig Schubert, "Feature Visualization," *Distill* (2017), https://distill.pub/2017/feature-visualization/. Shortcut learning is surveyed in Robert Geirhos et al., "Shortcut Learning in Deep Neural Networks," *Nature Machine Intelligence* 2 (2020): 665–673.

Random features: Ali Rahimi and Benjamin Recht, "Random Features for Large-Scale Kernel Machines," *NeurIPS* 20 (2007); Guang-Bin Huang, Qin-Yu Zhu and Chee-Kheong Siew, "Extreme Learning Machine: Theory and Applications," *Neurocomputing* 70 (2006): 489–501; Kevin Jarrett, Koray Kavukcuoglu, Marc'Aurelio Ranzato and Yann LeCun, "What Is the Best Multi-Stage Architecture for Object Recognition?" *ICCV* (2009); and Andrew M. Saxe et al., "On Random Weights and Unsupervised Feature Learning," *ICML* (2011). Double descent is described in Mikhail Belkin, Daniel Hsu, Siyuan Ma and Soumik Mandal, "Reconciling Modern Machine-Learning Practice and the Classical Bias–Variance Trade-Off," *PNAS* 116 (2019): 15849–15854. Reservoirs: Herbert Jaeger, "The 'Echo State' Approach to Analysing and Training Recurrent Neural Networks," GMD Report 148 (2001); Wolfgang Maass, Thomas Natschläger and Henry Markram, "Real-Time Computing Without Stable States," *Neural Computation* 14 (2002): 2531–2560; and Herbert Jaeger and Harald Haas, "Harnessing Nonlinearity: Predicting Chaotic Systems and Saving Energy in Wireless Communication," *Science* 304 (2004): 78–80, which uses the Mackey–Glass equation of Michael C. Mackey and Leon Glass, "Oscillation and Chaos in Physiological Control Systems," *Science* 197 (1977): 287–289.

Part V: Machines That Imagine

Deep autoencoders are Geoffrey E. Hinton and Ruslan R. Salakhutdinov, "Reducing the Dimensionality of Data with Neural Networks," *Science* 313 (2006): 504–507; the link between linear autoencoders and principal component analysis is Pierre Baldi and Kurt Hornik, "Neural Networks and Principal Component Analysis," *Neural Networks* 2 (1989): 53–58. Variational autoencoders are Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," arXiv:1312.6114 (2013). The arithmetic of convolutions and transposed convolutions is laid out in Vincent Dumoulin and Francesco Visin, "A Guide to Convolution Arithmetic for Deep Learning," arXiv:1603.07285 (2016).

GANs are Ian J. Goodfellow et al., "Generative Adversarial Nets," *NeurIPS* 27 (2014); the convolutional design of Chapter 19 follows Alec Radford, Luke

Metz and Soumith Chintala, "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks," arXiv:1511.06434 (2015).

Attention began with Dzmitry Bahdanau, Kyunghyun Cho and Yoshua Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," arXiv:1409.0473 (2014), and became the whole network in Ashish Vaswani et al., "Attention Is All You Need," *NeurIPS* 30 (2017). Residual connections are Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, "Deep Residual Learning for Image Recognition," *CVPR* (2016); layer normalization is Jimmy Lei Ba, Jamie Ryan Kiros and Geoffrey E. Hinton, "Layer Normalization," arXiv:1607.06450 (2016). The vision transformer is Alexey Dosovitskiy et al., "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale," *ICLR* (2021). Instruction tuning with human feedback is Long Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," *NeurIPS* 35 (2022).

Appendix

How hard it can be for networks to learn the Game of Life, and how much easier it is with more channels than the rule strictly needs, is studied in Jacob M. Springer and Garrett T. Kenyon, "It's Hard for Neural Networks to Learn the Game of Life," arXiv:2009.01398 (2020).

For a broader textbook treatment of everything in this book, Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning* (Cambridge, MA: MIT Press, 2016), and Simon J. D. Prince, *Understanding Deep Learning* (Cambridge, MA: MIT Press, 2023), are both free online.

Index

Adam, 115
AlexNet, 10, 167
AlphaGo, 26
argmax, 40
artificial intelligence, 16
asynchronous execution, 146
attention, 18, 255
    key, 255
    query, 255
    value, 26
attention heads, 260
autoencoder, 221
automatic differentiation, 100

Babbage, Charles, 14
backpropagation, 17, 20, 101
batch, 145
bias, 111
binary cross-entropy, 240
birthday problem, 85
bit depth, 54
broadcasting, 40
Butler, Samuel, 15

central limit theorem, 82
chain rule, 101
channel, 47
chaos, 75
code (latent), 222
confusion matrix, 162
convolution, 141
convolutional neural network, 155
Conway, John, 148
cosine similarity, 163
coupon collector's problem, 86
cropping, 49
cross-correlation, 141
cross-entropy, 108

data augmentation, 183
decision boundary, 5
decoder, 222
deep learning, 10, 20, 119
`default_rng`, 75
denoising, 235
derivative, 100
diffusion model, 249
Dijkstra, Edsger, 13
discriminator, 237
dithering, 54
dot product, 37
double descent, 202
dropout, 168

echo state network, 212
encoder, 222
Engelbart, Douglas, 18
epoch, 112
evolutionary search, 96
exponential, 107
extreme learning machine, 200

fan-in, 227
Fashion-MNIST, 69
feature map, 139

fine-tuning, 188
flattening, 65
Floyd–Steinberg dithering, 54
freezing weights, 185
Fukushima, Kunihiko, 6

Galton board, 80
Galton, Francis, 80
Game of Life, 148
generative adversarial network, 237
generator, 237
glider, 149
Goodfellow, Ian, 237
GPU, 91
gradient, 99
gradient descent, 99
Gray–Scott model, 152

He initialization, 125
hidden layer, 125
hill climbing, 94
Hinton, Geoffrey, 17
Hopfield, John, 17

identity matrix, 198
ImageNet, 167
instruction tuning, 261

Kay, Alan, 18
kernel, 137
Knuth, Donald, 13
Krizhevsky, Alex, 167

Laplacian, 149
latent diffusion model, 234
latent space, 233
latent vector, 239
layer normalization, 260
leaky ReLU, 240
learning rate, 104
least squares, 195
LeCun, Yann, 5
Leibniz, Gottfried Wilhelm, 14
Licklider, J. C. R., 18
liquid state machine, 212
logarithm, 107
loss, 106
Lovelace, Ada, 15

machine learning, 16
mask, 36
matrix, 33
matrix multiplication, 38
max pooling, 157
McCarthy, John, 16
McCulloch–Pitts neuron, 20
mechanical kingdom, 15
MENACE, 16
Michie, Donald, 16
Minsky, Marvin, 10
MNIST, 5, 61
mode collapse, 247
module, 131, 168, 185
momentum, 115
montage, 64
Monte Carlo method, 83
Monty Hall problem, 86

neural network, 10
neurons as an ecosystem, 22
`nn.Sequential`, 131
non-saturating, 241
nonlinearity, 122
normal distribution, 42, 80
NumPy, 31

one-hot, 110

optimizer, 115
overfitting, 129

Pandemonium, 17
Papert, Seymour, 18
Pascal's triangle, 78
Pascal, Blaise, 9
Pascaline Wager, 9
perceptron, 17
pixel, 3, 45
pooling, 4
position embedding, 254
pretrain, 261
principal component analysis, 224
projection, 254
pseudo-inverse, 195
pseudo-random numbers, 74
PyTorch, 89

random features, 199
random walk, 77
reaction–diffusion, 153
regularization, 202
ReLU, 122
reservoir computing, 212
reshape, 34
residual connection, 251
ridge regression, 198
Rosenblatt, Frank, 17

sample rate, 58
scikit-image, 48
seed, 42, 74, 92
self-supervised learning, 232
Selfridge, Oliver, 17
shape, 32
shortcut learning, 192
sigmoid, 228
singular value decomposition, 225
Sobel filter, 142
softmax, 107
spectral radius, 213
square-root law, 81
standard deviation, 78
stochastic gradient descent, 111
stride, 168
supervised learning, 4

template, 67
tensor, 89
test set, 61
token, 252
training set, 61
transfer learning, 181
transformer, 10, 18, 252
transpose, 33
transposed convolution, 226
Turing, Alan, 7, 13

uint8, 46

validation set, 111
value network, 26
variational autoencoder, 234
vectorization, 32
vision transformer, 252
von Neumann, John, 14

wave equation, 151
Wizard of Oz, 25